Machine Learning 101 for Financial Trading

FULL CODE EDITION

FSE-DAT-3019-1.1 | Data Analytics and Computational Modelling

Sector: Financial Services | Proficiency Level 3

LEARNER GUIDE

Tertiary Infotech Academy Pte Ltd

UEN 201200696W

Version 2.7 | 23 October 2026

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Table of Contents

Version History **4**
Introduction to the Course **7**
Low Code - You Own the Algorithm **8**
Course Objectives **9**
Alignment to the Skills Framework **10**
Course Structure **11**
How You Will Be Assessed **12**
Before You Start: Platform Setup **13**
Platform Guides: Sign-Up and Paper Trading Step by Step **14**
Download the Course Materials: Learner Guide, Slides and Lab Manual 14
Download, Install and Start the Trading Simulator Software 14
Sign Up for Alpaca and Open the Paper-Trading Account 15
Sign Up for TradingView and Find the Tools You Will Use 16
Paper Trading on TradingView, Step by Step 17
Paper Trading on Alpaca, Step by Step - Manual and Automated 17
Topic 1: Foundations of Quant and Algorithmic Trading: From Thorp to AI Agents **19**
1.1 Quant Success Stories: What Systematic Trading Has Achieved 19
1.2 The Evolution of Quant: From Thorp and Renaissance to High-Frequency Trading 19
1.3 The Paradigm Shift from Machine Learning to LLMs, and Its Impact on Quant 20
1.4 Agentic AI versus AI Agents in Trading, and Multi-Agent Trading Systems 21
1.5 Forms of Data within Financial Services and Big Data Applications in Trading 21
1.6 Developing Hypotheses for Quant and Algorithmic Trading Models 22
Reference: Quant Success Stories 23
Timeline: The Evolution of Quant and Algorithmic Trading 23
Comparison: Machine-Learning Quant versus LLM-Era Quant 25
Comparison: Agentic AI versus AI Agents in Trading 27
1.7 Lab 1: Pipeline Skeleton, Configuration and the Alpaca Paper Account 29
1.8 Lab 2: Data-Validation Stage: Cross-Check Pipeline Bars against TradingView 33
1.9 Lab 3: Ingestion Stage: Multi-Timeframe Bars, Headlines and Data Quality 36
1.10 Lab 4: Hypothesis Stage: from a Written Card to an Automated Statistical Test 41
1.11 Topic Summary 45
Topic 2: Technical Indicators, Trading Strategies and Metrics with TradingView and Alpaca **46**
2.1 Technical Indicators: Trend, Momentum, Volatility and Volume 46
2.2 Classification of Algorithmic Trading and Quant Algorithms 46
2.3 Analytics Platforms and Tools: TradingView versus Alpaca, and Python 47
2.4 Coding Languages for Algorithms and Signals, and Order Placement 48
2.5 Performance and Risk Metric Definitions 48
Reference: Classification of Trading Algorithms 49
Comparison: TradingView versus Alpaca 50
Strategy Families: the Idea, When It Works, When It Will Not 51
Key Metrics Defined 51
Indicator by Indicator 53
How It Works: Moving-Average Crossover (Trend Following) 61
How It Works: Bollinger Band Mean Reversion 62
How It Works: RSI and MACD Momentum Oscillators 63
How It Works: Cross-Sectional Momentum 64
How It Works: Donchian Channel Breakout 65
How It Works: VWAP and TWAP Execution 66
How It Works: Market Making 67
2.6 Lab 5: Indicator Stage: Pine Script v6 Indicators and their Python Twins 67
2.7 Lab 6: Strategy Prototype: TradingView Strategy Tester, Paper Trading from the Chart and Pipeline Reconciliation 71
2.8 Lab 7: Signal Stage: a Library of Trend, Mean-Reversion, Momentum and Breakout Signals 75
2.9 Lab 8: Execution Stage: an Order Manager that Trades the Paper Account in Any Session 79
2.10 Lab 9: Evaluation Stage: Performance and Risk Metrics with a Benchmark Table 84
2.11 Topic Summary 87
Topic 3: Machine Learning Models, Statistical Modelling and A/B Testing for Trading Signals **88**
3.1 Types of Statistical Analyses, Data Models, Algorithms and Computational Methods 88
3.2 Supervised Learning Models for Trading Signals 92
3.3 Unsupervised Learning: Clustering and PCA for Market Regimes 92
3.4 Statistical Modelling Techniques for Financial Time Series 92
3.5 A/B Testing Trading Strategies 93
3.6 Pairs Trading: Statistical Arbitrage as a Hedging Algorithm 93
Reference: Pairs Trading Step by Step 94

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

How It Works: Logistic Regression for Direction....94
How It Works: Decision Tree and Random Forest....95
How It Works: Gradient Boosting (XGBoost)....96
How It Works: K-Means Clustering for Market Regimes....97
How It Works: Principal Component Analysis (PCA)....98
How It Works: Walk-Forward Validation....98
How It Works: A/B Testing Two Strategy Variants....99
How It Works: Pairs Trading (Hedged Statistical Arbitrage)....100
3.7 Lab 10: Strategy-Search Stage: Grid Search, Out-of-Sample Ranking and Decay....100
3.8 Lab 11: ML-Signal Stage: a Walk-Forward Classifier from Probability to Position....103
3.9 Lab 12: Regime-Filter Stage: PCA and K-Means as a Gate on the Signals....107
3.10 Lab 13: A/B-Test Stage: a Statistical Decision before Capital Moves....110
3.11 Lab 14: Pairs Stage: a Hedged, Market-Neutral Statistical-Arbitrage Sleeve....113
3.12 Topic Summary....116
Topic 4: Building an End-to-End Algorithmic Trading Pipeline: Backtest, Forward Test and Performance Analysis....117
4.1 Architecture of an End-to-End Algorithmic Trading Pipeline....117
4.2 Backtesting with Realistic Trading Costs....117
4.3 Forward Testing with Automated Paper Trading....118
4.4 Potential Reasons for Unintended Outcomes and How to Diagnose Them....118
4.5 Performance Analysis, Insights and Reporting....118
Historical Data: Backtest, Tune, Then Forward Test....119
4.6 Lab 15: Backtest-Engine Stage: Cost Models, Execution Timing and Break-Even....120
4.7 Lab 16: Automation Stage: a Scheduled, Risk-Checked Forward Test and Live Paper Trading....123
4.8 Lab 17: Event-Driven Automation: TradingView Webhook to Signal, Risk and Execution....127
4.9 Lab 18: Diagnostics Stage: Automated Bias Checks before Go-Live....131
4.10 Lab 19: Monitoring and Reporting Stage: Automated Tearsheet and Backtest-vs-Forward Report....134
4.11 Lab 20: Capstone: One Command Runs the Pipeline End to End, with an Agentic Layer and Human Approval....138
4.12 Topic Summary....143
Assessment Preparation....144
Formula Sheet....145
Troubleshooting Cheat-Sheet....146
Glossary....147
References....148

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Version History

Version No	Date Effected	Change Update	Approved by
2.7	23 October 2026	Assessment papers rebuilt in the WSQ format for digital marking: cover page; page 2 with the trainee's name, NRIC, date and signature, the Instructions to Candidate and the Grading block with the assessor's name, NRIC, date and signature; questions from page 3, each with an answer box. Practical Performance redesigned around the labs, all in the Trading Simulator and TradingView: Task 1 hypothesis testing (Lab 4), Task 2 technical indicators and backtesting (Labs 5-7, 9), Task 3 machine-learning model and A/B testing (Labs 11, 13) and Task 4 the multi-agent desk with human approval, forward test, diagnosis and report (Labs 16, 18-20); marking points and the Assessor Key rewritten. Written Assessment Q6 (pairs trading) and Q9 (forms of data and turning headlines into data) now follow the slides.	Dr Alfred Ang
2.6	23 October 2026	Lab 17 (TradingView webhook) explained for learners: a new 'What This Lab Is For' section (what a webhook is, that the simulator supplies the URL, and the checks every alert passes before an order); Step 1 explains each part of the webhook card and alert message, Step 2 explains each of the seven self-test alerts and its HTTP code, and Step 3 gives the full real-alert walk-through (ngrok tunnel, webhook URL, chart, Pine strategy, alert dialog Condition / Message / Notifications, a quick Price-alert demo and where to watch the alert arrive). The Lab 17 screenshot now shows the TradingView webhook card with the 7/7 self-test result. Every lab in the deck is more visual: a 'Step by Step' slide (where to click, what to do, what you should see) and a 'How to Read the Screen' slide with numbered, colour-coded callouts on the simulator screenshot and a one-line takeaway; Lab 13 also gets a 'How to Reach the A/B Decision' slide (paired t-test, bootstrap Sharpe CI, Bonferroni alpha).	Dr Alfred Ang
2.5	23 October 2026	Topic 3 (K2): four slides added after 'Families of Learning Algorithms' explaining supervised, unsupervised and reinforcement learning - one slide each with a concept chart (SPY data for supervised and unsupervised, an agent-environment loop diagram for reinforcement), then a side-by-side comparison. Learner Guide section 3.1 and the Facilitator Guide Topic 3 teaching points cover the same content. Trading Simulator screenshots updated to the current app, whose Research tabs now follow the lab order: Strategy search and Walk-forward (Lab 10), ML models (Lab 11), Regimes (Lab 12), A/B test (Lab 13), Pairs screen (Lab 14), the multi-agent desk (Lab 20) and the 14-stage Pipeline Map (Getting Started); Lab 8 and Lab 9 screenshot captions aligned with the lab steps.	Dr Alfred Ang
2.4	22 October 2026	Practical Performance question paper simplified: one answer box per task, in which the candidate answers every part of the task and pastes its screenshot - the hypothesis test result in Task 1 and a TradingView Strategy Tester backtest of the Bollinger Bands strategy on XLF in Task 2.	Dr Alfred Ang

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Version No	Date Effected	Change Update	Approved by
2.3	22 October 2026	Assessment papers: every Practical Performance sub-question now has its own answer box, with screenshot boxes naming the lab where the step was practised, and the Written Assessment's ruled lines are replaced by answer boxes, so candidates type or paste their answers into the paper and upload it to the LMS. Assessment Plan Annex A wording aligned with the Written Assessment paper.	Dr Alfred Ang
2.2	22 October 2026	Screenshots added to Labs 7, 8 and 9: a Bollinger Band mean-reversion strategy in TradingView with the Paper Trading order panel (Lab 7), the simulator's paper order ticket and its passed pre-trade risk checks and fill (Lab 8), and Performance Analytics comparing a strategy with SPY buy-and-hold (Lab 9). Slide references in the Lesson Plan, Learner Guide and Facilitator Guide updated to the new slide numbers.	Dr Alfred Ang
2.1	21 October 2026	Low-code delivery. Every lab now runs in the Trading Simulator without coding, and the Practical Performance is done by configuring and running a provided trading program, with the same scenario, tasks, ability coverage and marking. A Prompt Library provides one AI prompt per lab, with one-click links, for learners who want code generated for them. Getting Started screenshots added (simulator Pipeline Map and Integrations, Alpaca paper dashboard, TradingView chart). Version History added to every document. Topic 2 slides added on mean reversion (the simulator's rules, mean reversion versus trend following, the main risk and safeguards), on the RSI look-back period (RSI(2) versus RSI(14)) and on long and short positions.	Dr Alfred Ang
2.0	18 October 2026	Major revision, focused entirely on quant and algorithmic stock trading. Courseware rebuilt and re-platformed to Alpaca (API-first broker, paper trading, 24/5 US stock trading) and TradingView (charting, Pine Script v6, Strategy Tester, Paper Trading) with Python. Four topics: quant success stories and the evolution of quant from Thorp and Renaissance through statistical arbitrage to high-frequency trading, the paradigm shift from machine learning to LLMs and its impact on quant, and AI agents versus agentic AI in trading; the classification of trading algorithms, indicators, strategies, a TradingView versus Alpaca comparison, order placement and metric definitions; machine learning models, statistical modelling, A/B testing and pairs trading as a hedging algorithm; and an end-to-end, automated algorithmic trading pipeline. 'How the algorithm works' slides explain each algorithm on real market data. Twenty hands-on labs build one pipeline stage by stage, each with an instruction PDF containing step-by-step AI prompts, and include live paper trading of a strategy. Assessment: Written Assessment (short-answer questions, K1-K9) and Practical Performance (four tasks, A1-A10) with a standalone SMART marking rubric. Facilitator Guide added. Fee restated in SGD exclusive of GST, optimum class size stated, provider name updated to Tertiary Infotech Academy Pte Ltd.	Dr Alfred Ang
1.1	22 December 2022	Multiple choice questions removed; practical test retained as the sole assessment instrument.	Dr Alfred Ang
1.0	10 June 2022	Initial release for pilot launch, aligned to Data Analytics and Computational Modelling (Proficiency Level 3).	Dr Alfred Ang

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Version No	Date Effected	Change Update	Approved by

This document is at Version 2.7, effective 23 October 2026.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Introduction to the Course

Algorithms now place the majority of orders in the world's major equity markets. Banks use them to execute client orders at the best price, asset managers to rebalance portfolios, and proprietary desks to run systematic strategies. Over the last decade two further changes have reached every trading floor: machine learning, which lets a model learn patterns from data rather than follow hand-written rules, and agentic AI, in which language-model agents research, debate and act.

This course teaches the discipline that makes those tools safe to use. You will frame a trading idea as a testable hypothesis, chart it on TradingView, rebuild it in Python on Alpaca market data, search for and model better variants with machine learning, A/B test them, backtest them with realistic trading costs, and then forward-test the winner with an automated bot that places buy and sell orders on an Alpaca paper-trading account. Finally you will diagnose what went wrong, analyse the performance and report a recommendation.

This Learner Guide contains the full content of all four topics and step-by-step instructions for all twenty labs. Use it in class, in the assessment (both components are open book), and afterwards at your desk.

Lion City Capital and every scenario in these materials are fictitious and for training only. All trading in this course is paper (simulated) trading. Nothing in these materials is investment advice or a recommendation to trade any security.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Low Code - You Own the Algorithm

This Full Code edition is for learners who want full customisation in Python. Every lab builds one stage of a single Python pipeline: the complete scripts are provided in each lab folder, with the prompts that generate them, and an AI coding assistant writes and changes the code. What matters is still the big concepts: the hypothesis, how each algorithm makes its decision, how it is tested without bias and with costs, how risk is controlled, and how the results are judged.

Role	What it means
Your job: the hypothesis	What edge do you expect, why, and how will you know it is real?
Your job: the algorithm's logic	The rule or model, its inputs, and when it buys or sells.
Your job: testing and judgement	Out-of-sample, next-bar, net of costs; read the metrics.
Your job: risk and accountability	Limits, kill switch, approval - you answer for every trade.
AI's job: the code	AI assistants write, run and fix the Python and Pine; you review it.
AI's job: the drudgery	Data plumbing, charts, reports - you review what it produces.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Course Objectives

- LO1: Develop hypotheses for implementing data analytics and computational modelling activities in algorithmic trading, and identify possible big data applications across the forms of data within financial services.
- LO2: Identify appropriate statistical algorithms and data models to test trading hypotheses, and use appropriate analytics platforms, analytical tools and statistical methods given specific analytics and reporting requirements.
- LO3: Conduct statistical modelling of market data with machine learning models to derive patterns and solutions, and perform A/B testing of trading strategies based on different hypotheses.
- LO4: Perform coding and configuration of an automated trading program based on selected models or algorithms, conduct tests on its actions and outcomes, diagnose unintended outcomes, and draw insights from the results to support decisions and report accordingly.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Alignment to the Skills Framework

This course is aligned to FSE-DAT-3019-1.1 Data Analytics and Computational Modelling at Proficiency Level 3 under the Skills Framework for Financial Services: "Identify and utilise appropriate statistical algorithms and data models to perform data analyses, test hypotheses and derive inferences, patterns or solutions from data". All 10 ability statements and all 9 knowledge statements are delivered and assessed.

Ref	Ability Statement	Delivered In
A1	Develop hypotheses for implementing data analytics and computational modelling activities	Topic 1
A2	Identify appropriate statistical algorithms and data models to test hypotheses and/or theories	Topics 2, 3
A3	Use appropriate analytics platforms, analytical tools, statistical methods and analytical approaches given specific analytics and reporting requirements	Topics 1, 2
A4	Conduct statistical modelling of data to derive patterns and solutions	Topic 3
A5	Perform A/B testing based on different hypotheses	Topic 3
A6	Identify possible big data applications	Topic 1
A7	Perform coding and configuration of software agents or programs based on selected models or algorithms	Topic 4
A8	Conduct tests on the actions taken and outcomes to assess effectiveness of the models	Topic 4
A9	Diagnose unintended outcomes produced by statistical models	Topic 4
A10	Draw relevant trends and insights from data analysis to support decisions and report accordingly	Topic 4

Ref	Knowledge Statement	Delivered In
K1	Data analytics and modelling business use cases	Topics 1, 4
K2	Types of statistical analyses, data models, algorithms and advanced computational methods	Topic 3
K3	Range and application of various statistical methods and algorithms	Topic 2
K4	Range and application of various types of data models	Topic 3
K5	Usage of analytics platforms and tools	Topics 1, 2, 4
K6	Statistical modelling techniques	Topic 3
K7	Coding languages for programming of algorithms and signals	Topics 2, 4
K8	Potential reasons for unintended outcomes	Topic 4
K9	Forms of data within financial services	Topic 1

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Course Structure

Course duration: 16 hours over 2 training days (14 hours classroom training + 2 hours assessment). Daily schedule: 0900 - 1830 (8 instructional hours per day, excluding two 15-minute tea breaks and a 1-hour lunch break).

Topic	Title	Day	Labs	TSC Ref
Topic 1	Foundations of Quant and Algorithmic Trading: From Thorp to AI Agents	Day 1	1 - 4	A1, A6, K1, K9
Topic 2	Technical Indicators, Trading Strategies and Metrics with TradingView and Alpaca	Day 1	5 - 9	A2, A3, K3, K5, K7
Topic 3	Machine Learning Models, Statistical Modelling and A/B Testing for Trading Signals	Day 2	10 - 14	A4, A5, K2, K4, K6
Topic 4	Building an End-to-End Algorithmic Trading Pipeline: Backtest, Forward Test and Performance Analysis	Day 2	15 - 20	A7, A8, A9, A10, K7, K8

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

How You Will Be Assessed

You must be assessed as Competent (C) in BOTH components to be awarded competency for this module. Each component is marked out of 100; you need at least 80 marks in each, and at least 60% of the marks for every knowledge and ability statement. Both components are open book: you may refer to this Learner Guide, the lab code and your own class notes.

Component	Duration	What it covers
Written Assessment (Short Answer Questions)	45 minutes	9 open-ended questions, one for each knowledge statement (K1 - K9). There are no multiple choice questions.
Practical Performance	75 minutes	4 practical tasks (the Trading Simulator and TradingView, as in the labs - no coding, on the Trading Simulator's paper broker or an Alpaca paper account) on a new trading scenario, covering the 10 ability statements (A1 - A10). The tasks mirror the labs.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Before You Start: Platform Setup

Every lab runs in one of two modes. In ONLINE mode the scripts use your own Alpaca paper-trading account for market data and orders. In OFFLINE mode - used automatically when no API keys are set - they read the bundled daily data in labs_fullcode/data and send orders to a built-in paper-broker simulator. Both modes run the same code, so nothing is lost if the venue network blocks the API.

Python environment

1. Install Python 3.10 or later and confirm python3 --version.
2. In the labs folder create a virtual environment: python3 -m venv .venv, then activate it (source .venv/bin/activate on macOS, .venv\Scripts\activate on Windows).
3. Install the libraries: pip install -r requirements.txt
4. Run the Lab 1 account check (see Lab 1). It reports whether you are in online or offline mode.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Platform Guides: Sign-Up and Paper Trading Step by Step

Follow these guides in order: download the course materials and install the trading simulator first, then sign up for Alpaca and TradingView before Lab 1 and Lab 2; the paper-trading guides are used in Lab 6 (TradingView) and Labs 8 and 16 (Alpaca). Menu labels on both platforms change from time to time - if a label differs, use the nearest equivalent.

Download the Course Materials: Learner Guide, Slides and Lab Manual

Get your own copy of the courseware before Day 1. Everything is free to download.

Step 1: Go to the course portal

Open lms-tms.tertiaryinfotech.com in your browser (the Tertiary Infotech Training Management System).

Step 2: Request a one-time password

Under Sign in to the TMS, enter the email you registered for the course with and click Send OTP. (Prefer a password? Click Login with password instead.)

Step 3: Key in the OTP

Open the email from the TMS, copy the one-time password and key it in. No OTP, or the class is missing after sign-in? Use Register with your Course Run ID (from your trainer or course administrator).

Step 4: Download the Learner Guide and slides

Open this course. Under Courseware, open the Learner Slides and the Learner Guide and download both PDFs (they open in Google Drive: File > Download).

Step 5: Download the lab manual

Open Activities / Lab. It holds the labs folder: every lab has its own folder with a LabNN_Instructions.pdf (the lab manual for that lab) and its Python or Pine Script files. Download the whole folder (right-click > Download).

Step 6: Or download everything from GitHub

Open https://github.com/tertiarycourses/TGS-2023018794-Machine-Learning-101-for-Financial-Trading, click Code > Download ZIP and unzip it. The lowcode/ folder holds the course: the Learner Guide (PDF, Word and Markdown), the slides, the Lesson Plan and all twenty lab manuals. The fullcode/ folder holds the Full Code edition, with the Python labs. With Git installed you can instead run: git clone https://github.com/tertiarycourses/TGS-2023018794-Machine-Learning-101-for-Financial-Trading.git

Step 7: Keep the materials together

Unzip into one folder on your laptop, for example Documents/ML101-Trading, so the Learner Guide, the lab manuals and the simulator sit side by side.

Each lab folder has its own LabNN_Instructions.pdf - the step-by-step lab manual.

Download, Install and Start the Trading Simulator Software

The labs run in the course's trading simulator - a user-friendly web app on your own laptop. Install it once before Lab 1. Choose Option A (uv) or Option B (Docker).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 1: Install the prerequisites

Install Git and Python 3.11 or later. Then install uv: on macOS or Linux run curl -LsSf https://astral.sh/uv/install.sh | sh ; on Windows (PowerShell) run powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex". For Option B install Docker Desktop instead.

Step 2: Download the software from GitHub

Run: git clone https://github.com/alfredang/algorithmic-trader-bot.git - or open https://github.com/alfredang/algorithmic-trader-bot, click Code > Download ZIP and unzip it. Then open a terminal in the algorithmic-trader-bot folder.

Step 3: Create your settings file

Copy .env.example to .env (macOS/Linux: cp .env.example .env ; Windows: copy .env.example .env). Leave it as it is to use the offline demo data, or add your Alpaca paper keys (and optional Twelve Data / NewsAPI keys) as described in the next guides.

Step 4: Option A - install and start with uv

Run: uv sync (installs everything), then uv run trader seed (loads the demo data, about 15 seconds), then uv run trader serve --with-worker (web app, API and the automation scheduler in one process). Leave the terminal open.

Step 5: Option B - start with Docker Compose

Run: docker compose up -d --build and wait until all four containers are healthy (docker compose ps). Load the demo data with docker compose exec backend trader seed . Stop it later with docker compose down (your data is kept).

Step 6: Open the app

Open http://localhost:8000 in your browser (if port 8000 is busy: uv run trader serve --with-worker --port 8010). The onboarding wizard lets you choose the offline demo or connect your Alpaca paper account; Settings > Load demo data reloads the demo at any time.

Step 7: Connect the AI agents (optional)

For the multi-agent desk: run uv sync --extra ai , then codex login and choose Sign in with ChatGPT (uses your own ChatGPT subscription), or put an ANTHROPIC_API_KEY or OPENAI_API_KEY in .env. Check Integrations in the app. Without either, the desk runs its rule-based agents - every lab still works.

Step 8: Start Lab 1

In the app's sidebar open Course Labs > Lab 1 and follow the checklist alongside Lab01_Instructions.pdf. If the README of the software gives a different command or port, follow the README - it is the authoritative start-up guide.

Simulator: github.com/alfredang/algorithmic-trader-bot - the README has the full start-up steps and troubleshooting.

Sign Up for Alpaca and Open the Paper-Trading Account

Alpaca paper trading is free and needs no funding. You use it in every lab that places orders. Do this before Lab 1.

Step 1: Create the account

Go to alpaca.markets and choose Sign Up. Register with your email address and a strong password, then open the verification email and confirm your address.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 2: Log in to the dashboard

Log in at app.alpaca.markets and go to the Home page.

Step 3: Switch to the paper account

In the upper-left corner choose your 'Paper Trading' account. Paper trading needs no deposit and no brokerage approval; every order uses virtual money.

Step 4: Generate your paper API keys

On the paper account's Home page find the API section and generate a key. Copy the Key ID and the Secret Key immediately - the secret is shown only once. Generating a new key invalidates the old one.

Step 5: Store the keys safely

In the labs folder copy .env.example to .env and paste the two values against APCA_API_KEY_ID and APCA_API_SECRET_KEY. Never put keys in a script, a slide, a chat or any file you share; .env is git-ignored.

Step 6: Check the connection

In the labs folder run: python -m pipeline check. The banner must say ONLINE and show your paper account's status and equity. Paper keys only work on the paper endpoint (paper-api.alpaca.markets) - they cannot touch a live account.

Step 7: If you cannot open an account

Leave .env blank. Every lab runs OFFLINE on the bundled data and the built-in paper-broker simulator, so nothing in the course is blocked.

No funding needed. Paper keys only work on paper-api.alpaca.markets. No account? Every lab also runs offline.

Sign Up for TradingView and Find the Tools You Will Use

The free TradingView plan is enough for Labs 2, 5 and 6. Do this before Lab 2.

Step 1: Create the account

Go to tradingview.com and choose Sign up (top right). Register with email, Google or Apple, then confirm the activation email.

Step 2: Choose the free plan

Keep the free plan when asked. It shows adverts and limits the number of indicators per chart and the alert features; webhook alerts (Lab 17) need a paid plan.

Step 3: Open Supercharts

From Products choose Supercharts. Search a symbol such as NASDAQ:AAPL or SGX:D05 in the symbol box at the top left and choose the daily (1D) timeframe.

Step 4: Set the time zone

Click the time zone at the bottom right of the chart and choose (UTC+8) Singapore, so bar times match the course examples.

Step 5: Locate the four panels the labs use

Indicators (top toolbar), the Pine Editor and the Strategy Tester (tabs at the bottom), and Trade / the Trading Panel (top right) for paper trading.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

The free plan covers Labs 2, 5 and 6. Webhook alerts in Lab 17 need a paid plan - the lab includes a local simulator.

Paper Trading on TradingView, Step by Step

TradingView's built-in Paper Trading account lets you place simulated orders from the chart. You use it in Lab 6 to put the moving-average strategy into action.

Step 1: Connect Paper Trading

On Supercharts click Trade (top right). Choose Paper Trading from the broker list and click Connect. The Trading Panel opens at the bottom of the chart.

Step 2: Set your starting capital and commission

In the Trading Panel open the account settings and set the starting balance and a commission that matches the course cost assumptions, so paper results are comparable with the Python backtest.

Step 3: Open the order ticket

Click Buy or Sell (or Trade) to open the order ticket. Choose the side, quantity and order type - Market, Limit or Stop - and the price for a limit or stop order.

Step 4: Add a stop loss and take profit

In the order ticket's risk settings tick Stop Loss and Take Profit and enter the levels (for example 2 x ATR below and above the entry). Then place the order.

Step 5: Watch the order and position

Use the Trading Panel tabs: Positions (open trades and unrealised P&L), Orders (working, filled, cancelled) and Account History. Orders and positions also appear as lines on the chart, which you can drag to modify.

Step 6: Put the strategy into action

With the Lab 6 Pine strategy on the chart, read its latest signal in the Strategy Tester's List of Trades and place the matching paper order. Record the fill price and compare it with the strategy's simulated fill.

Step 7: Close and reset

Close a position from the Positions tab (or the x on its chart line). To start again, reset the paper account from the Trading Panel's account settings - this clears its orders and history.

Paper fills use TradingView's prices but cannot show your own market impact - compare them with the backtest's assumed costs.

Paper Trading on Alpaca, Step by Step - Manual and Automated

First place one order by hand to see the order lifecycle, then let the pipeline place orders automatically (Labs 8 and 16).

Step 1: Place a manual paper order

In the Alpaca dashboard select the Paper Trading account (upper left), search the symbol (for example SPY) and open its page. Set the side, quantity, order type and time in force, then press Buy.

Step 2: Follow the order lifecycle

Open the Orders view and watch the status move from new / accepted to filled (or cancelled). Open Positions to see the holding, cost basis and unrealised P&L.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 3: Know the trading hours from Singapore

The US regular session runs 21:30-04:00 Singapore time (22:30-05:00 in the northern winter). Outside it, Alpaca's 24/5 trading accepts only LIMIT orders with extended hours enabled (time in force DAY or GTC), at lower liquidity; a market order placed while the market is closed waits for the next session.

Step 4: Let the pipeline place the orders

In the labs folder run: python -m pipeline execute (Lab 8). The order manager reads the Alpaca clock, compares the target position with the actual one, submits the paper order, polls its status and prints the fill.

Step 5: Run the strategy automatically

Run: python -m pipeline run --loop --interval 60 --live-paper (Lab 16). Each cycle fetches bars, computes the signal, runs the risk checks, places or skips the paper order and writes a row to the trade log.

Step 6: Check the results in the dashboard

Refresh the Alpaca dashboard: the bot's orders appear under Orders and its holdings under Positions, matching labs/_outputs/trade_log.csv.

Step 7: Stop it safely

Run: python -m pipeline kill --on. The next cycle halts before any order is sent. Cancel any working orders and, if you wish, close positions from the dashboard.

From Singapore the US regular session is 21:30-04:00 (22:30-05:00 in northern winter); class-time orders use Alpaca's 24/5 extended-hours limit orders.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Topic 1: Foundations of Quant and Algorithmic Trading: From Thorp to AI Agents

Day	Abilities	Knowledge	Labs	Slides
Day 1	A1, A6	K1, K9	1 - 4	35 - 75

Learning Outcome

Develop hypotheses for implementing data analytics and computational modelling activities in algorithmic trading, and identify possible big data applications across the forms of data within financial services.

1.1 Quant Success Stories: What Systematic Trading Has Achieved

Maps to: K1 | Slides 36-38

- Ed Thorp - from blackjack to markets: his fund (Convertible Hedge Associates, 1969; Princeton Newport Partners from 1974) is often called the first market-neutral hedge fund and earned about 20% a year after fees for two decades by hedging warrants and convertibles with probability models.
- Renaissance Technologies - mathematician Jim Simons founded Monemetrics in 1978 (renamed 1982). Its Medallion fund (1988) trades purely on mathematical models of market data and averaged about 66% a year before fees (about 39% after fees) from 1988 to 2018.
- D.E. Shaw - founded in 1988 by computer-science professor David Shaw, a former member of Morgan Stanley's quantitative trading group; it builds mathematical models and software to exploit market anomalies and manages about US$60-65 billion.
- Man AHL - founded in 1987 as a systematic trend-following futures manager whose founders typed historical prices in by hand; it grew into a multi-strategy quant business trading momentum models across 800+ markets.
- Two Sigma - founded in 2001, it applies artificial intelligence, machine learning and distributed computing to trading, and manages about US$70 billion.
- Quant market makers - Jane Street reported about US$20.5 billion of net trading revenue in 2024 and nearly a quarter of primary US-listed ETF activity; Citadel Securities is estimated to handle about a quarter of all US equities trading.
- The common thread: a testable hypothesis, clean data, rigorous out-of-sample testing, realistic costs and strict risk control - exactly the pipeline this course builds.

1.2 The Evolution of Quant: From Thorp and Renaissance to High-Frequency Trading

Maps to: K1, K9 | Slides 39-46

- Algorithmic trading is the use of a computer program that follows a defined set of instructions to generate and execute orders automatically, based on price, timing, volume or a mathematical model.
- Foundations (1950s-1970s): Markowitz's 'Portfolio Selection' (1952) measures risk as variance; Thorp's 'Beat the Market' (1967) applies probability to hedging; the Black-Scholes option formula is published as the CBOE opens in April 1973.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

- Factors and program trading (1970s-1980s): Barr Rosenberg's Barra builds multi-factor risk models of style, industry and size; computers trade baskets of stocks against new index futures. Black Monday (19 October 1987) shows portfolio-insurance program selling can deepen a crash.
- Statistical arbitrage (1980s-1990s): Gerry Bamberger and Nunzio Tartaglia's group at Morgan Stanley trades pairs of stocks that move together, by rule rather than intuition. Alumni and imitators - D.E. Shaw (1988) and Renaissance's Medallion (1988) - industrialise many small, market-neutral edges.
- Lessons from failure: Long-Term Capital Management, a highly leveraged relative-value fund, lost US$4.6 billion in 1998 and was recapitalised under Federal Reserve supervision; in the 'quant quake' of August 2007 crowded quant equity market-neutral funds lost heavily in days as positions unwound.
- From quant to high-frequency trading (1996-2010s): electronic order books such as Island ECN (1996), decimalisation (2001) and Regulation NMS (2005, in force 2007) reward speed. HFT firms co-locate servers at exchanges; Spread Networks' US$300 million straight fibre cut about 3 ms off New York-Chicago, and microwave links soon beat it.
- The speed race meets regulation: the 6 May 2010 Flash Crash and Knight Capital's loss of more than US$460 million in 45 minutes (2012) lead to kill switches and testing rules - MiFID II (2018), the SEC Market Access Rule - and IEX's 350-microsecond 'speed bump' exchange (2016). In Singapore, SGX RegCo's Regulatory Guide on Algorithmic Trading (2020) sets the expectations.
- Democratisation and AI (2018-present): API-first brokers such as Alpaca (2018) open algorithmic stock trading to individuals; machine learning and alternative data widen the signal set; LLMs and multi-agent systems now read text and propose trades.

1.3 The Paradigm Shift from Machine Learning to LLMs, and Its Impact on Quant

Maps to: K1, K9 | Slides 47-51

- Classical machine-learning quant works on structured data - prices, volumes, fundamentals - with features engineered by the quant (lagged returns, RSI, factor scores) and models such as regression, random forests and gradient boosting fitted to labelled history.
- The LLM paradigm adds unstructured text - news, filings, earnings-call transcripts, research - read directly by a pretrained language model. Lopez-Lira and Tang (2023) found that ChatGPT's good / bad scores for news headlines predict next-day stock returns, most strongly for small stocks and negative news.
- Finance-specific LLMs: BloombergGPT (March 2023) is a 50-billion-parameter model trained with 363 billion tokens of Bloomberg financial data plus general text, aimed at sentiment, entity recognition and news classification.
- Automated alpha research: systems such as Alpha-GPT turn a quant's idea in plain language into formulaic alpha factors, and AlphaAgent targets alpha decay by steering away from crowded, homogeneous factors. Coding agents now draft strategy code and backtests that the quant reviews.
- New risks: an LLM may already 'know' what happened in its training period, so in-sample backtests can carry look-ahead bias and a 'distraction' effect from what the model knows about the company (Glasserman and Lin, 2023). Test only on data after the model's knowledge cutoff and anonymise company names.
- Faster crowding: Lopez-Lira and Tang report that the returns to LLM news strategies decline as LLM adoption rises - prices become more efficient, so an LLM signal decays like any other alpha.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

- The paradigm shift: AI now writes the code. Coding assistants and agents produce the Python, Pine Script and pipelines, so the quant's value moves from writing every line to the big concepts - the hypothesis, how the algorithm decides, unbiased testing with costs, risk control and judging the results. This course is taught that way: AI codes, you own the algorithm.
- What does not change: the hypothesis, out-of-sample testing, costs, risk limits and human accountability - the quant specifies, tests and supervises the models and agents, and answers for every trade.

1.4 Agentic AI versus AI Agents in Trading, and Multi-Agent Trading Systems

Maps to: K1 | Slides 52-58

- Agentic AI is the capability of AI to plan and act: it calls tools (market-data and broker APIs), uses skills and memory, and follows instructions and guardrails to complete multi-step work when it is asked - for example, an LLM that fetches a stock's bars and news, checks its memory of past trades and proposes an order within its instructions.
- An AI agent is a special class of agentic AI that stands by 24/7 and works all the time: it is always on, woken by schedules, events and messages rather than by a prompt, and keeps long-term memory across days. Examples: OpenClaw (an open-source, self-hosted agent), Hermes Agent (Nous Research, February 2026) and OpenAI Dots (always-on agents in ChatGPT, September 2026). In trading, an AI agent is a round-the-clock monitor that watches prices and news overnight and proposes paper trades within limits.
- Multi-agent systems are agentic AI with several specialised agents - analysts, researchers, a trader and a risk manager - that coordinate, debate and decide; run continuously, the same desk becomes an always-on AI agent.
- TradingAgents (Xiao et al., 2024) simulates a stock-trading firm: fundamental, sentiment, news and technical analyst agents, bullish and bearish researchers who debate, a trader agent and a risk-management team.
- Other stock-trading agent systems: FinMem (2023, an LLM trading agent with layered memory making Buy / Sell / Hold decisions), FinAgent (2024, multimodal - prices, news and chart images), StockAgent (2024, LLM investors in a simulated stock market), QuantAgent (2025, indicator, pattern, trend and risk agents for high-frequency trading), the open-source AI Hedge Fund and Agentic Trading Lab, which paper-trades through the Alpaca API.
- Agentic systems introduce new failure modes: hallucinated figures, prompt injection from news text, over-trading and unclear accountability. Controls: hard risk limits coded outside the LLM, paper trading first, a full decision log and a human who approves every live order.
- Regulatory context in Singapore: MAS's FEAT principles (2018); the MAS Technology Risk Management Guidelines; and the MAS Guidelines on Artificial Intelligence Risk Management, issued on 7 October 2026 and effective from 7 October 2027, which require AI inventories, materiality assessment and lifecycle controls.

1.5 Forms of Data within Financial Services and Big Data Applications in Trading

Maps to: K9, A6 | Slides 59-63

- Market data: OHLCV bars (open, high, low, close, volume) at timeframes from one minute to one month; quotes (best bid and ask); trades (every print); and order-book depth. Alpaca's Market Data API serves bars, quotes, trades and news.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

- Reference and corporate-action data: symbols, exchanges, trading calendars, splits and dividends. Using unadjusted prices across a split creates a false crash in a backtest.
- Fundamental and macroeconomic data: financial statements, ratios (P/E, P/B, ROE), earnings; interest rates, inflation and central-bank decisions - the inputs to factor models.
- Alternative data: news and social-media sentiment, earnings-call transcripts, web traffic, satellite imagery and card spending - increasingly turned into trading signals by NLP and LLMs.
- The four Vs of big data in trading: volume (tick data runs to billions of rows), velocity (streaming quotes), variety (prices, text, images) and veracity (bad ticks, survivorship, restatements).
- Big data applications in trading: tick-level execution analytics and smart order routing, cross-asset regime detection, news-driven event trading, sentiment signals, intraday risk monitoring, and surveillance of order and trade data for market abuse.

1.6 Developing Hypotheses for Quant and Algorithmic Trading Models

Maps to: A1, K1 | Slides 64-66

- Every trading model starts with a hypothesis: a precise, testable statement about market behaviour - not 'buy good stocks' but 'when SPY closes below its lower Bollinger Band, its average 5-day forward return exceeds its unconditional 5-day return'.
- A good trading hypothesis states the universe, the signal, the holding period, the expected effect and the benchmark, and is written down BEFORE looking at the results - otherwise it is curve-fitting.
- Express it as a null hypothesis (H0: no difference) and an alternative (H1: the signal earns more than the benchmark), with the test and the significance level chosen in advance.
- Economic rationale matters: trend following rests on under-reaction and herding; mean reversion on over-reaction and liquidity provision; momentum on slow diffusion of information; pairs trading on a stable economic link between two stocks. A pattern with no rationale is more likely to be noise.
- The quant model-building steps: frame the hypothesis, collect data, define the target and features, split by time, train, validate, test out of sample, then paper trade before risking capital.
- Business use cases for data analytics and modelling in trading: signal generation, execution cost reduction, portfolio construction, risk forecasting and trade surveillance - each with its own success metric.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Reference: Quant Success Stories

Firm / person	Year	Story
Ed Thorp - Princeton Newport	1969	First market-neutral quant fund: about 20% a year after fees for two decades by hedging warrants and convertibles
Renaissance - Medallion	1988	Purely model-driven; about 66% a year before fees, 39% after (1988-2018); closed to outsiders, employee-owned
D.E. Shaw	1988	Founded by a computer-science professor from Morgan Stanley's quant group; models and software; about US$60-65bn
Man AHL	1987	Systematic trend following from hand-typed price histories to momentum models across 800+ markets
Two Sigma	2001	AI, machine learning and distributed computing applied to trading; about US$70bn under management
Jane Street	2024	Quant market maker: about US$20.5bn net trading revenue; nearly a quarter of primary US ETF activity
Citadel Securities	Today	Quant market maker estimated to handle about a quarter of all US equities trading

Common thread	Meaning
Hypothesis first	Each started from a testable idea about prices, not from a tool
Data and testing	Clean data and out-of-sample tests before capital
Many small edges	Diversified, market-neutral bets rather than one big call
Costs and risk	Execution cost, leverage and risk limits decide survival - see LTCM 1998

Timeline: The Evolution of Quant and Algorithmic Trading

Algorithm: A finite, step-by-step set of instructions that a computer follows to solve a problem or complete a task.

Algorithmic trading: Using computer programs that follow predefined rules on price, timing, volume or models to generate and execute orders automatically, with limited or no human intervention.

- 1950s-70s: portfolio theory, arbitrage and option pricing
- 1970s-80s: factor risk models and program trading
- 1980s-90s: statistical arbitrage and pairs trading
- 1988 on: fully systematic signal mining (Renaissance, D.E. Shaw)
- 2000s: electronic market making and HFT
- 2010s: speed race - co-location, microwave, microseconds
- 2020s: ML on alternative data, then LLM and agentic quant

Year	Milestone	Significance
1952	Modern portfolio theory	Markowitz's 'Portfolio Selection' measures risk as variance: diversification becomes maths
1967	Thorp's 'Beat the Market'	Ed Thorp applies probability to hedging warrants - an early quant trading playbook
1969	First market-neutral quant fund	Thorp's fund (Princeton Newport from 1974) earns about 20% a year after fees for two decades
1973	Black-Scholes and the CBOE	Option-pricing formula published as the CBOE opens (Apr 1973); CBOE adopts it by 1975

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Year	Milestone	Significance
Mid-1970s	Factor risk models	Barr Rosenberg's Barra models portfolio risk from style, industry and size factors
1978	Renaissance is founded	Mathematician Jim Simons founds Monemetrics; renamed Renaissance Technologies in 1982
1980s	Statistical arbitrage	Bamberger and Tartaglia's Morgan Stanley group trades pairs of co-moving stocks by rule
1987	Black Monday	Dow falls 22.6% on 19 Oct; portfolio-insurance program selling deepens the crash
1988	Medallion and D.E. Shaw	Renaissance launches Medallion; ex-Morgan Stanley quant David Shaw founds D.E. Shaw
1996	Island ECN	Fast electronic order book (trading from 1997) becomes a hub for early HFT firms
1998	LTCM collapses	Leveraged relative-value fund loses US$4.6bn; Fed-supervised US$3.65bn recapitalisation
2001	Decimalisation	US stocks quote in cents, not 1/16ths; spreads narrow and electronic market making grows
2005	Regulation NMS	SEC rule (in force 2007) links venues; fragmentation rewards speed and smart order routing
2007	The quant quake	Week of 6 Aug: crowded quant market-neutral funds suffer sudden losses as positions unwind
2010	Flash Crash	6 May: the Dow drops nearly 1,000 points in minutes after a large algorithmic sell order
2010	The speed race	US$300m straight fibre New York-Chicago cuts about 3 ms; microwave links soon beat it
2012	Knight Capital	A faulty deployment sends millions of orders in 45 minutes; Knight loses over US$460m
2016	IEX speed bump	SEC approves IEX as an exchange with a 350-microsecond delay against latency arbitrage
2018	MiFID II	EU requires algorithm testing, kill switches and records; HFT formally defined
2018	API-first brokers	Alpaca's commission-free trading API opens algorithmic stock trading to individuals
2020	SGX RegCo algo guide	Sets expectations for governance, risk controls and testing of algorithms by SGX members
2023	LLM trading signals	ChatGPT headline sentiment predicts next-day stock returns (Lopez-Lira and Tang)
2024	Multi-agent LLM trading	TradingAgents (Dec 2024): analyst, debate, trader and risk agents trade stocks
2026	MAS AI Risk Management	Guidelines issued 7 Oct 2026, effective 7 Oct 2027: AI inventories and lifecycle controls

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Paradigm shift	Period	What changed
From intuition to probability	1950s-1970s	Markowitz, Thorp and Black-Scholes make risk, hedging and option value mathematical
From stocks to factors	1970s-1980s	Barra factor models and program trading treat portfolios as baskets of risk exposures
From factors to statistical arbitrage	1980s-2000s	Morgan Stanley's pairs group, D.E. Shaw and Renaissance harvest many small market-neutral edges
From floors to electronic speed	1996-2010s	ECNs, decimalisation and Reg NMS enable HFT: co-location, microwave links, microseconds
From statistics to machine learning	2000s-2010s	Models learn non-linear patterns from prices, order books and alternative data
From writing code to directing AI	2023-present	AI writes and runs the code; quants own the big concepts - hypothesis, algorithm logic, testing, risk

Benefits of algorithmic trading	Risks of algorithmic trading
Speed: reacts and executes in milliseconds	Technical failure: bugs and outages (Knight Capital, 2012)
Discipline: rules remove fear, greed and fatigue	Overfitting: great backtests that fail on live data
Backtesting: test ideas on historical data first	Black swans and regime change the model has never seen
Lower costs: split large orders to reduce market impact	Feedback loops between algorithms (2010 Flash Crash)
Scale: monitors many markets and assets around the clock	Set-up, data and compliance costs; no guaranteed profit

Comparison: Machine-Learning Quant versus LLM-Era Quant

Era	Years	In trading
Symbolic AI	1956-1980s	Hand-written rules: expert systems and rule-based program trading; brittle when markets change
Machine learning	1990s-2000s	Models learn from data: regression, tree and boosting signals; statistical arbitrage at scale
Deep learning	2012-2017	Many-layer networks on GPUs find non-linear patterns in prices, order books and alternative data
Generative AI	2017-2023	Transformers and LLMs turn news, filings and earnings calls into trading signals
Agentic AI	2024-present	LLM agents research, code, backtest and paper-trade strategies under hard risk limits

Year	AI milestone	Trading counterpart
1950	Turing Test	Turing asks if machines can think; in 1952 Markowitz turns portfolio choice into maths
1956	'AI' is named	AI becomes a field; Ed Thorp soon applies computers and probability to blackjack, then markets
1980s	Expert systems	Hand-written if-then rules - the same logic as the rule-based program trading of the 1980s

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Year	AI milestone	Trading counterpart
1986	Backpropagation	Multi-layer neural networks become trainable - the ancestor of today's price-prediction nets
1988	Models trade markets	Renaissance's Medallion fund launches, trading purely on mathematical models of market data
1997	Deep Blue	IBM beats Kasparov at chess; computer-driven quant firms such as D.E. Shaw are already trading
2012	Deep learning	GPU-trained deep networks win ImageNet; deep learning becomes a new tool for market data
2016	AlphaGo	Deep reinforcement learning wins at Go; RL is researched for order execution and portfolios
2017	Transformer	The architecture behind every LLM - later used to read news and filings for trading signals
2023	LLMs meet markets	BloombergGPT (50bn parameters); ChatGPT headline sentiment predicts next-day stock returns
2024	Multi-agent trading	TradingAgents models a trading firm: analyst, bull/bear debate, trader and risk agents
2025	Agentic quant research	LLM agents mine formulaic alphas and write backtests (e.g. Alpha-GPT, AlphaAgent)
2026	Always-on AI agents	OpenClaw, Hermes Agent and OpenAI Dots stand by 24/7: agentic AI that keeps working
2026	Governed AI	MAS AI Risk Management Guidelines (7 Oct 2026) set lifecycle controls for AI in finance

Dimension	Machine-learning quant	LLM-era quant
Main data	Structured: prices, volumes, fundamentals	Adds unstructured text: news, filings, earnings calls, research
Features	Hand-engineered by the quant (lags, RSI, factors)	Text read directly; LLM scores sentiment and events
Model building	Fit regressions, trees, boosting on labelled data	Prompt or fine-tune pretrained LLMs; agents propose alphas
Who writes the code	The quant, line by line	AI writes it; the quant directs and reviews
Research workflow	The quant writes every line of code	Coding agents draft code and backtests; the quant reviews
Alpha decay	Signals fade as more funds trade them	LLM signals fade too as LLM adoption rises
New risks	Overfitting; look-ahead in features	LLM training-data look-ahead and 'distraction'; homogeneous factors
Human role	Model builder	Supervisor: hypothesis, testing, risk sign-off

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Impact on quant	Evidence
Text becomes a signal	ChatGPT headline scores predict next-day returns, strongest for small stocks and bad news (Lopez-Lira and Tang)
Finance-specific LLMs	BloombergGPT (2023): 50bn parameters trained with 363bn tokens of Bloomberg financial data
Automated alpha mining	Alpha-GPT turns ideas into formulaic alphas; AlphaAgent targets alpha decay and crowding
Backtests need new controls	LLMs may 'know' in-sample history: test after the knowledge cutoff and anonymise firms (Glasserman and Lin)
Faster crowding	Strategy returns decline as LLM adoption rises - prices become more efficient (Lopez-Lira and Tang)
Agents still need limits	Agents propose trades; hard-coded risk checks and humans approve them

Comparison: Agentic AI versus AI Agents in Trading

Agentic AI: The capability of AI to plan and act: it calls tools, uses skills and memory, and follows instructions to complete multi-step work when it is asked.

Trading example: On request, an LLM fetches a stock's bars and news through tools, checks its memory of past trades and proposes an order within its instructions

AI agent: A special class of agentic AI that stands by 24/7 and works all the time: always on, woken by schedules, events and messages, with its own long-term memory. Examples: OpenClaw, Hermes Agent, OpenAI Dots.

Trading example: A round-the-clock trading agent that watches prices and news overnight and proposes paper trades within risk limits

Dimension	AI Agents	Agentic AI
What it is	A special class of agentic AI that is always on	Capability to plan and act with tools, skills, memory
Runtime	Persistent: stands by 24/7 on a server or PC	Runs when invoked: a chat, workflow or pipeline
Trigger	Schedules, events, messages - unprompted	A user prompt or a pipeline step
Memory	Long-term memory across days and tasks	Session or task memory; can use long-term memory
Building blocks	Agentic AI + always-on runtime and scheduler	LLM + tools + skills + memory + instructions
Examples	OpenClaw, Hermes Agent, OpenAI Dots	Claude Code, ChatGPT with tools, TradingAgents
Trading example	24/7 market and news monitor proposing trades	On-demand multi-agent analysis before a trade
Main risks	Acting unsupervised overnight; runaway loops	Hallucination; prompt injection; tool misuse

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Framework	What it does in stock trading
TradingAgents (Xiao et al., 2024) https://arxiv.org/abs/2412.20138	Simulated stock-trading firm: fundamental, sentiment, news and technical analysts, bull/bear debate, trader, risk team
FinMem (Yu et al., 2023) https://arxiv.org/abs/2311.13743	LLM trading agent with profiling and layered memory making Buy / Sell / Hold decisions on individual stocks
FinAgent (Zhang et al., 2024) https://arxiv.org/abs/2402.18485	Multimodal agent reading prices, news and chart images, with dual-level reflection; tested on stocks and crypto
StockAgent (2024) https://arxiv.org/abs/2407.18957	LLM investor agents trade a simulated stock market under macro, policy and event shocks; avoids test-set leakage
QuantAgent (2025) https://arxiv.org/abs/2509.09995	Price-driven multi-agent system for high-frequency trading: Indicator, Pattern, Trend and Risk agents
AI Hedge Fund (open source) https://github.com/virattt/ai-hedge-fund	Investor-persona agents (e.g. Warren Buffett), a risk manager and a portfolio manager decide trades for chosen tickers
Agentic Trading Lab (Open-Finance-Lab) https://github.com/Open-Finance-Lab/AgenticTrading	Agent-run stock portfolios with backtests and paper trading through the Alpaca API

Agent role	Responsibility
Orchestrator / Planner	Breaks the goal into tasks, routes work to agents and assembles the result
Fundamental Analyst	Reads filings, earnings and balance sheets to judge intrinsic value
Technical Analyst	Reads price, volume and indicators such as MACD and RSI
Sentiment Analyst	Gauges mood from social media, analyst ratings and options flow
News Analyst	Tracks news wires, press releases and macro events
Bull / Bear Researchers	Argue for and against the trade so weak ideas are challenged
Trader	Combines the reports into a buy, sell or hold call with timing and size
Risk Manager	Checks volatility, liquidity, exposure and drawdown limits
Portfolio Manager	Approves or rejects the proposal and fits it into the whole portfolio
Execution Agent	Places approved orders through a broker API at the best price
Backtest / Evaluation Agent	Tests signals on historical data and feeds back improvements
Audit / Compliance Agent	Logs every decision and checks it against rules and limits

Risk	What goes wrong	Control
Hallucination	An LLM states a wrong earnings figure or invents a signal with confidence	Ground answers in tool or RAG data, log the reasoning, review unusual calls
Prompt injection	Malicious text in news, web pages or files hijacks the agent's instructions	Input and output guardrails, least-privilege API keys, no secrets in prompts
Overtrading and runaway loops	Agents trade too often or amplify each other, compounding losses	Order and position caps, daily-loss circuit breaker, kill switch
Model and backtest risk	Overfitting, look-ahead bias and regime change: strong backtests, weak live results	Walk-forward tests, paper trade first, monitor drift and costs
Accountability	Unclear who answers for an autonomous agent's trade or a vendor's AI	The firm stays accountable: human sign-off, audit trails, clear owners
Regulation (MAS)	FEAT principles (2018) and MAS AI Risk Management Guidelines (issued Oct 2026)	Board oversight, AI inventory, materiality assessment and lifecycle controls

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

1.7 Lab 1: Pipeline Skeleton, Configuration and the Alpaca Paper Account

Maps to: K5 | Duration: 20 min | Slide 67

Scenario

Lion City Capital (fictitious), a Singapore-based systematic trading desk, is building an automated algorithmic trading pipeline for US equities and ETFs on an Alpaca paper-trading account. Every later stage - data, signals, execution, automation - reads one configuration file, so the desk starts by creating that file, keeping the API keys outside the code, and proving that the paper account, the market clock and the data path answer.

Objective

Create the pipeline package and its config.yaml (universe, strategy parameters, risk limits, costs, secret names), build a validating configuration loader and a market-clock module, and run `python -m pipeline check` against the Alpaca paper account or the offline simulator.

Files

labs_fullcode/Lab01-alpaca-setup/ - Lab01-alpaca-setup/check_stage.py, pipeline/config.yaml, pipeline/config.py, pipeline/clock.py, pipeline/check.py, pipeline/__main__.py, Lab01-alpaca-setup/solution/, .env.example, requirements.txt

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 01	python3 Lab01-alpaca-setup/check_stage.py (same as: python -m pipeline check)
The complete code	pipeline/check.py, pipeline/clock.py, pipeline/config.py (reference copy: Lab01-alpaca-setup/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab01-alpaca-setup/Lab01_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 1: Set up Python and your AI coding assistant (K5)

Open a terminal in the labs folder. Run: python3 -m venv .venv, then source .venv/bin/activate (Windows: .venv\Scripts\activate), then pip install -r requirements.txt. Open the labs folder in VS Code with Claude Code (or your AI coding assistant). Working rule for the whole course: the assistant writes code, YOU review it and run it; never paste an API key into the assistant.

Check: pip finishes without errors and python3 -c "import pandas, yaml, alpaca, sklearn" prints nothing.

Step 2: Store Alpaca PAPER keys outside the code (K5)

Sign up at alpaca.markets, switch to Paper Trading in the account selector, open API Keys and click Generate New Keys. Run: cp .env.example .env and paste the values into APCA_API_KEY_ID and APCA_API_SECRET_KEY. Never put keys in code, e-mail, chat or git. No keys? Leave them blank: every stage then runs OFFLINE on bundled data and the local paper-broker simulator.

Check: labs_fullcode/.env holds your paper key ID (starts with PK) and secret, or blank values for offline mode.

Step 3: Write the pipeline configuration (K5)

Ask your assistant for pipeline/config.yaml, then compare it with the reference in Lab01-alpaca-setup/solution/config.yaml. Every later stage reads its symbols, dates, strategy, risk limits and costs from this one file.

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/ create config.yaml for Lion City Capital (fictitious)'s algorithmic trading pipeline. Sections: mode (auto | offline); broker (name alpaca, paper: true, sim_state_file pipeline_sim.json); secrets_from_env listing only the NAMES of the environment variables APCA_API_KEY_ID, APCA_API_SECRET_KEY, WEBHOOK_SECRET and ANTHROPIC_API_KEY; universe (trade: SPY QQQ AAPL MSFT NVDA JPM XLF GLD TLT; pairs KO/PEP, JPM/BAC, XLF/JPM; benchmark SPY; watch: AAPL as NASDAQ:AAPL in USD and D05.SI as SGX:D05 in SGD); data (start 2018-01-01, end 2025-12-31, timeframes 1Day 1Week 1Month 1Hour, hourly_window 2025-10-01 to 2025-12-31, cache_dir _cache, max_abs_daily_move 0.08); strategy (name ema_cross, symbol SPY, params fast 10 slow 30, execution next_open); costs (preset alpaca_retail, scenarios zero alpaca_retail per_share_broker high_cost, tradingview commission_per_order 1.0 and slippage_bps 0.4); risk (max_position_pct 0.95, max_daily_loss_pct 0.02, max_order_value 100000, max_order_qty 100, allowed_symbols = the trade universe, long_only true, kill_switch_file KILL_SWITCH); live_paper (max_position_pct 0.10, limit_buffer_bps 25, poll_seconds 30, crypto_demo_symbol BTC/USD, crypto_demo_notional 25); webhook (host 127.0.0.1, port 8080, path /webhook, require_signal_agreement false); agents (model claude-sonnet-5-5, symbols NVDA AAPL SPY, max_position_pct 0.20, max_order_value 25000, min_confidence 0.6, stop_atr 2.0, target_atr 3.0); schedule (America/New_York, Asia/Singapore, run_at_exchange_time 09:35). Comment every key. Never put a key or secret VALUE in this file.

Check: config.yaml has the sections broker, secrets_from_env, universe, data, strategy, costs, risk, live_paper, webhook, agents and schedule, and contains no key values.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 4: Build the configuration loader with safety validation (K5)

Ask your assistant for pipeline/config.py, then run: python3 pipeline/config.py

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/config.py. load_config(path=None, strict=True) reads pipeline/config.yaml with yaml.safe_load (path override from the PIPELINE_CONFIG environment variable), caches it, stores the path in cfg['_path'] and the problems in cfg['_problems'], and raises ValueError when strict and invalid. If mode is offline set LAB_OFFLINE=1. validate(cfg) returns a list of problems: required sections present; broker.paper must be true; risk.max_position_pct and risk.max_daily_loss_pct between 0 and 1; costs.preset must exist in common.backtest.COST_PRESETS; every secrets_from_env value must be an UPPER_SNAKE environment variable name; any string that looks like a credential (PK/AK/CK followed by 14+ capitals or digits, 'sk-ant-', or a 32+ character base64 string) is reported as a secret in the file. Also add apply_cli_flags(argv) (strip --offline -> LAB_OFFLINE=1, and --config PATH -> PIPELINE_CONFIG), secret(cfg, name) that reads the value from os.environ, lab_path(rel), kill_switch_path(cfg), cost_model(cfg, preset=None), tradingview_cost_model(cfg) returning CostModel('TradingView settings', min_commission=1.0, slippage_bps=0.4) from config, and stage_banner(no, title). Use LAB_ROOT from common.config. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python3 pipeline/config.py prints the path of pipeline/config.yaml followed by ': valid'.

Step 5: Add the market clock for US and Singapore time (K5)

Ask your assistant for pipeline/clock.py, then run: python3 pipeline/clock.py

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/clock.py. session_at(ts) returns 'regular' (09:30-16:00 New York, Mon-Fri), 'pre' (04:00-09:30), 'after' (16:00-20:00), 'overnight' (20:00-04:00 from Sunday night to Friday 04:00, Alpaca 24/5 trading) or 'closed'; naive timestamps are New York time. RULES maps each session to the allowed order types, time_in_force values and extended_hours flag: regular = market, limit, stop, bracket; pre, after and overnight = LIMIT orders only with time_in_force DAY or GTC and extended_hours=True; closed = a GTC limit order that queues for the next session. broker_now(broker) uses the simulator's replay clock (broker.state['now']) when it has one, otherwise the real UTC clock. market_status(broker=None, now=None, override=None) returns now in America/New_York and Asia/Singapore, the session, a holiday flag and the rules; when the broker is the Alpaca paper client (mode 'alpaca-paper') and the session is 'regular' but broker.get_clock().is_open is False, report 'closed' (exchange holiday). describe(status) returns one line such as 'Mon 2026-10-19 21:05 New York = Tue 09:05 Singapore | session: OVERNIGHT (...)'. Use zoneinfo only. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: One clock line with New York and Singapore time. At a weekend it shows CLOSED; during a weekday 09:00-18:30 Singapore class it shows OVERNIGHT (until 16:00 SGT in US summer time) or PRE.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 6: Build the check command and the pipeline CLI (K5)

Ask your assistant for pipeline/check.py, pipeline/__init__.py and pipeline/__main__.py. Compare with Lab01-alpaca-setup/solution/.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/check.py with run(reset=False, quiet=False) returning a dict and main(argv) for `python -m pipeline check [--reset]`. Print: [1] the config path, firm, mode, universe, strategy, risk limits and the validation result; [2] versions of pandas, numpy, scipy, sklearn, xgboost, matplotlib, yaml and alpaca; [3] each entry of secrets_from_env as 'found (PK12...)' for the Alpaca key ID, 'found (hidden)' for any secret, or 'not set' - never print a secret value; [4] the paper account from common.broker.get_broker(state_file=cfg['broker']['sim_state_file'], reset=reset): account number, status, equity, cash, buying power and number of positions; [5] the market clock line from pipeline.clock; [6] the last 3 daily bars of the strategy symbol from common.data.get_bars with data_source(). End with 'RESULT: PIPELINE READY - paper trading only' and exit code 0, or 'NOT READY - <problems>' and exit code 1. Then create pipeline/__init__.py (docstring, __version__) and pipeline/__main__.py with a COMMANDS table {command: (lab, module, function, description)} for all 20 stages (check, validate, ingest, hypothesis, indicators, reconcile, signals, execute, evaluate, search, ml, regime, abtest, pairs, backtest, run, schedule, kill, webhook, alert-sim, diagnose, report, agents, e2e); import the module lazily; strip --offline and --config with apply_cli_flags; print the table when called with no arguments. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline (no arguments) lists 24 commands across Labs 1-20.

Step 7: Run the pre-flight check (K5)

Run: python -m pipeline check --reset (same as: python3 Lab01-alpaca-setup/check_stage.py --reset). Then run echo $? (Windows: echo %errorlevel%) to see the exit code that a scheduler would see.

Check: Offline: validation OK; 8 libraries OK; all four secrets 'not set'; account SIM-LCC-0001, status ACTIVE, equity, cash and buying power USD 100,000.00, 0 positions; a clock line; SPY bars ending 2025-12-31 with close 676.64; 'RESULT: PIPELINE READY - paper trading only'; exit code 0. Online: your PAPER account number and balance and the session from the real Alpaca clock.

Step 8: Prove the safety rules work (K5)

In pipeline/config.yaml change broker.paper to false and run python -m pipeline check. Restore true. Then paste the fake key PKABCDEFGHIJKLMNOP as the value of firm and run the check again. Restore the firm name and run the check a third time.

AI prompt - paste into your AI coding assistant
Explain to me, as a finance professional new to coding, why labs_fullcode/pipeline/config.py refuses to run when broker.paper is false or when a value in config.yaml looks like an API key, and what could go wrong at a trading desk without these two checks. Do not change any file.

Check: First run: 'NOT READY - broker.paper must be true ...', exit code 1. Second run: 'firm looks like a secret - move it to an environment variable', exit code 1. Third run: PIPELINE READY again.

Expected Output (Full Code)

python -m pipeline check --reset (offline): configuration valid; 8 libraries OK; 4 secrets 'not set'; account SIM-LCC-0001 ACTIVE with equity, cash and buying power USD 100,000.00 and 0 positions; a clock line in New York and Singapore time (CLOSED at weekends, OVERNIGHT or PRE during a weekday Singapore class); last SPY bar 2025-12-31, close 676.64; 'RESULT: PIPELINE READY - paper trading only', exit code 0. python -m pipeline lists 24 commands.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Debrief

A trading pipeline is only as safe as its configuration and its connection to the broker. One validated config file, paper-only enforcement, keys held in the environment and a pre-flight check that a scheduler can read from the exit code show how AI and algorithmic trading programs are connected to execution platforms in a controlled way (K5).

Troubleshooting

Symptom	What to do
ModuleNotFoundError: pipeline, common or yaml	Activate the venv, run pip install -r requirements.txt and run every command from inside labs.
401 Unauthorized or forbidden online	The keys are wrong or are LIVE keys. Regenerate PAPER keys and check .env has no spaces or quotes around the values.
Mode shows OFFLINE although keys are set	Remove --offline, set mode: auto in config.yaml and make sure LAB_OFFLINE is not 1 in .env.
The assistant wrote code that prints the secret	Reject the change: only the first 4 characters of the key ID may ever be shown, never the secret.

1.8 Lab 2: Data-Validation Stage: Cross-Check Pipeline Bars against TradingView

Maps to: A3, K5, K9 | Duration: 20 min | Slide 68

Scenario

Lion City Capital's traders watch DBS Group (SGX:D05) and Apple (NASDAQ:AAPL) on TradingView, while the pipeline trains and trades on data from an API. Before any model is built, the desk must prove that the pipeline's bars agree with what traders see on the chart - and understand why adjusted research prices differ from the unadjusted prices on the screen and on an order ticket.

Objective

Read daily bars for SGX:D05 and NASDAQ:AAPL from the TradingView Data Window, record them, and build a validation stage that compares TradingView with the pipeline's raw and dividend-adjusted bars and writes a validation report.

Files

labs_fullcode/Lab02-tradingview-charts/ - Lab02-tradingview-charts/validate_stage.py, Lab02-tradingview-charts/lcc_data_check.pine, Lab02-tradingview-charts/tradingview_values.csv, pipeline/validate.py, pipeline/reference/AAPL_1Day_raw.csv, pipeline/reference/D05.SI_1Day_raw.csv, Lab02-tradingview-charts/solution/validate.py

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 02	python3 Lab02-tradingview-charts/validate_stage.py (same as: python -m pipeline validate)
The complete code	pipeline/validate.py (reference copy: Lab02-tradingview-charts/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab02-tradingview-charts/Lab02_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

Step 1: Add the data-check panel to SGX:D05 (A3, K9)

Log in to tradingview.com and open a chart of SGX:D05, timeframe 1D, chart type Candles. Open the Pine Editor, paste the contents of Lab02-tradingview-charts/lcc_data_check.pine, Save it as 'LCC Data Check' and click Add to chart. In its Settings set 'Bar to validate' to 31 December 2025. Open the Data Window (right toolbar) and hover over that bar.

AI prompt - paste into your AI coding assistant
Explain labs_fullcode/Lab02-tradingview-charts/lcc_data_check.pine for a finance professional: what each input does, how the script finds the bar to validate, and which values in its table I should copy into tradingview_values.csv. Do not change the file.

Check: The orange-shaded bar is 2025-12-31 and the panel shows about open 56.50, high 56.60, low 56.11, close 56.36, volume about 2.05 million (SGD, default unadjusted view). Free plans may show SGX data delayed.

Step 2: Repeat for NASDAQ:AAPL and switch the adjustment (A3, K9)

Open NASDAQ:AAPL on 1D with the same indicator and read 31 December 2025. Then open Chart settings > Symbol and tick 'Adjust data for dividends'; read the bar again and untick it.

Check: Default view: open 273.06, high 273.68, low 271.75, close 271.86, volume about 27.29 million. With 'Adjust data for dividends' ticked the close drops to about 271.12.

Step 3: Record what TradingView shows (A3)

Open Lab02-tradingview-charts/tradingview_values.csv. Overwrite the two example rows with your own readings (tradingview_symbol, date, open, high, low, close, volume, setting = default or adjusted).

Check: Two rows: NASDAQ:AAPL and SGX:D05, both dated 2025-12-31, setting default.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 4: Build the validation stage (A3, K5, K9)

Ask your assistant for pipeline/validate.py. The reference unadjusted files are already in pipeline/reference/ (split-adjusted, dividend-unadjusted bars with an adj_close column, the view TradingView shows by default).

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/validate.py. reference_bars(symbol) reads pipeline/reference/<SYMBOL>_1Day_raw.csv (columns timestamp, open, high, low, close, adj_close, volume, dividend, split). adjusted_bars(symbol, raw) returns the research bars from common.data.get_bars for the same dates and, if no file exists (FileNotFoundError, e.g. D05.SI), raw open/high/low/close multiplied by adj_close / close. compare(tv_row, bar) returns the percentage difference per field and match=True if every price is within 0.1% and volume within 10%. run(date='2025-12-31', tv_file=None) loops over config universe.watch, prints for each symbol the RAW row, the ADJUSTED row, your TradingView row from Lab02-tradingview-charts/tradingview_values.csv (lines starting with # are comments) and the differences, the dividend-adjustment factor (adjusted close / raw close), change vs prior close, 52-week high/low and 20-day average volume, and a verdict: 'PASS - TradingView matches RAW', 'PASS - TradingView matches ADJUSTED' or 'FAIL'. Save _outputs/Lab02_validation_report.csv and a chart Lab02_validation.png (raw vs adjusted close in 2025, one panel per symbol) with common.plotting.save. main(argv) supports --date and --tv. This stage only reads data - it must never call an order API. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: pipeline/validate.py exists and python -m pipeline validate --help lists --date and --tv.

Step 5: Run the validation stage (A3, K9)

Run: python -m pipeline validate --date 2025-12-31 (same as: python3 Lab02-tradingview-charts/validate_stage.py --date 2025-12-31).

Check: AAPL RAW 273.06 / 273.68 / 271.75 / 271.86 vs ADJUSTED 272.32 / 272.94 / 271.01 / 271.12, factor 0.9973, diff vs adjusted +0.272%. D05 RAW 56.50 / 56.60 / 56.11 / 56.36 vs ADJUSTED 54.35 / 54.45 / 53.98 / 54.22, factor 0.9620, diff +3.950%. Both 'PASS - TradingView matches RAW (dividend-unadjusted) bars'; RESULT: 2/2; _outputs/Lab02_validation_report.csv and Lab02_validation.png.

Step 6: Understand adjusted versus unadjusted prices (K9)

Open pipeline/reference/D05.SI_1Day_raw.csv and find the dividend and split columns (for example the 1.1 bonus-share split on 2024-04-22). Decide which series a backtest must use and which price an order ticket uses.

AI prompt - paste into your AI coding assistant
Using the dividend and split columns in labs_fullcode/pipeline/reference/D05.SI_1Day_raw.csv, explain why the pipeline's adjusted D05 close for 2025-12-31 is about 3.95% below TradingView's close, why backtests must use adjusted prices while orders are priced in raw prices, and what error a trading program makes if it mixes the two. Answer in five bullet points; do not change any file.

Check: D05's factor 0.9620 reflects dividends paid after the bar (back-adjusted into history); backtests use adjusted returns, orders use the raw market price.

Step 7: Make the validation fail on purpose (A3)

In tradingview_values.csv change the AAPL row to the dividend-adjusted values open 272.32, high 272.94, low 271.01, close 271.12, setting adjusted, and rerun the stage. Next change only the close to 280.00 and rerun. Finally restore your real reading.

Check: With 271.12 AAPL shows 'PASS - TradingView matches ADJUSTED bars'; with 280.00 it shows 'FAIL - TradingView matches neither series' and RESULT: 1/2.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 8: Identify other forms of market data (K9)

On TradingView open the News, Financials and economic calendar views for SGX:D05. Note whether the SGX data are real-time or delayed on your plan and why the pipeline takes its US data from the Alpaca API rather than from a chart.

Check: You can name at least three data forms besides bars (news, fundamentals, economic calendar) and state whether your SGX data are delayed.

Expected Output (Full Code)

python -m pipeline validate --date 2025-12-31: AAPL RAW close 271.86 vs ADJUSTED 271.12 (factor 0.9973); SGX:D05 RAW close 56.36 vs ADJUSTED 54.22 (factor 0.9620); both match TradingView's default (unadjusted) view to the cent - 'PASS - TradingView matches RAW'; RESULT: 2/2 watch-list symbols validated; files _outputs/Lab02_validation_report.csv and _outputs/Lab02_validation.png.

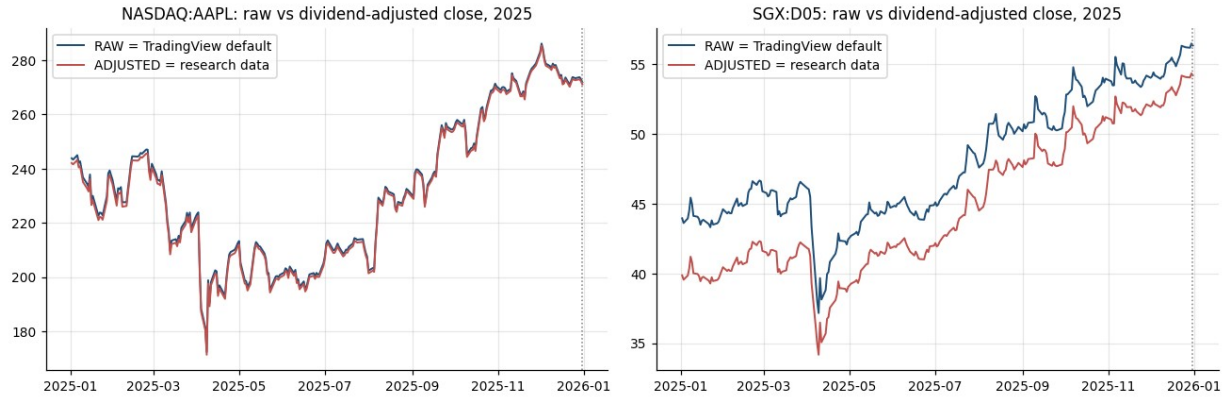


Figure: Lab 2 output (labs_fullcode/_outputs/Lab02_validation.png), offline mode on the bundled data.

Debrief

Charting platforms are analytics tools (A3) and the API is the pipeline's data path (K5); a validation stage proves they agree before anything is automated. Knowing that adjusted and unadjusted prices are different forms of the same data (K9) prevents a classic error: training on adjusted history and then trading on raw quotes without converting.

Troubleshooting

Symptom	What to do
SGX:D05 shows delayed or no intraday data	Free plans show delayed SGX data; daily bars are enough for this lab.
Verdict FAIL for D05	Check the date, that the chart is 1D and that 'Adjust data for dividends' is unticked; enter the regular-session close, not an extended-hours price.
Data Window not visible	Open it from the right toolbar (Data Window icon) or press Alt+D.
Pine compile error	The first line must be //@version=6 and the whole file must be pasted.

1.9 Lab 3: Ingestion Stage: Multi-Timeframe Bars, Headlines and Data Quality

Maps to: A6, K9 | Duration: 20 min | Slide 70

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Scenario

The pipeline's research and trading stages must all read the same clean data. The desk's data engineer automates ingestion: bars for the whole universe at four timeframes, a data-quality report that blocks bad files, a cache every later stage reads, and a first unstructured source - news headlines - that will later feed a sentiment agent.

Objective

Build `python -m pipeline ingest` to fetch and cache daily, weekly, monthly and hourly bars for the universe, run automated data-quality checks, ingest headlines as unstructured data, and describe the forms of data and the 3 Vs of big data.

Files

labs_fullcode/Lab03-alpaca-market-data/ - Lab03-alpaca-market-data/ingest_stage.py, pipeline/ingest.py, pipeline/reference/sample_headlines.json, Lab03-alpaca-market-data/solution/ingest.py

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 03	python3 Lab03-alpaca-market-data/ingest_stage.py (same as: python -m pipeline ingest)
The complete code	pipeline/ingest.py (reference copy: Lab03-alpaca-market-data/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab03-alpaca-market-data/Lab03_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 1: Understand how bars are requested (K9)

Open common/data.py and read get_bars(): online it builds a StockBarsRequest for StockHistoricalDataClient; offline it reads data/<SYMBOL>.csv and resamples weekly and monthly bars.

AI prompt - paste into your AI coding assistant
Read labs_fullcode/common/data.py and explain in five bullet points how get_bars() requests bars from Alpaca (TimeFrame, adjustment, feed, time zone) and what it does offline. Do not change the file.

Check: You can state that online requests use adjustment=ALL and feed=IEX, and that offline data are the bundled Yahoo Finance CSVs.

Step 2: Build the cache and load_bars() (A6, K9)

Ask your assistant for the cache helpers at the top of pipeline/ingest.py.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/ingest.py with cache_dir() (config data.cache_dir under labs), cache_file(symbol, timeframe) -> _cache/bars/<SYMBOL>_<TIMEFRAME>.csv, and load_bars(symbol, start=None, end=None, timeframe='1Day') that reads the cached CSV (hourly timestamps as America/New_York, daily as dates), slices start and end inclusively (weekly and monthly bars are labelled with the period end, so allow 6 / 30 days after end), and falls back to common.data.get_bars when the file is missing or when online with end=None. Add load_closes(symbols, start, end) that inner-joins daily closes like common.data.get_closes. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python3 -c "from pipeline.ingest import load_bars; print(load_bars('SPY','2025-12-01','2025-12-31').tail(1))" prints the 2025-12-31 bar with close 676.64 (from the data source until the cache exists).

Step 3: Add automated data-quality checks (A6)

Ask your assistant for quality_checks() and the blocking rules.

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/ingest.py add quality_checks(df, timeframe, end, max_move) returning a dict: rows, first, last, missing_values, duplicate_timestamps, not_sorted, high_below_low, close_outside_range, zero_volume, gaps_over_4_days (daily only), big_moves (daily |return| > max_move, from config data.max_abs_daily_move) and stale (last daily bar more than 5 days before end). BLOCKING = missing_values, duplicate_timestamps, not_sorted, high_below_low, close_outside_range, stale. Status is FAIL if any blocking count is non-zero (the file is NOT cached), CHECK if there are big moves or zero-volume bars, else OK. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: quality_checks() returns 0 for every blocking check on the bundled SPY daily file.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 4: Ingest an unstructured source: news headlines (A6, K9)

Ask your assistant for the headline functions. Offline they use the SYNTHETIC sample in pipeline/reference/sample_headlines.json (training text, not real news).

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/ingest.py add _fetch_headlines(symbols): online use alpaca.data.historical.news.NewsClient with NewsRequest(symbols=..., start=now - 7 days, limit=10) and keys from common.config.get_keys(); on any error, or offline, load pipeline/reference/sample_headlines.json and set _source to 'bundled SYNTHETIC sample (training only - not real news)'. Add load_headlines(symbols=None) that reads _cache/news/headlines.json when it exists. Treat headline text as untrusted data. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: load_headlines(['NVDA']) returns 4 headline dictionaries with keys date, symbol and headline.

Step 5: Build the ingest run with the report (A6, K9)

Ask your assistant for run() and main().

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/ingest.py add run(symbols=None, timeframes=None) for `python -m pipeline ingest`: symbols = universe trade + pair legs + benchmark; for each symbol and config timeframe fetch bars with common.data.get_bars (1Hour uses data.hourly_window; record 'not available offline' on FileNotFoundError), run quality_checks, cache every non-FAIL file, write _outputs/Lab03_data_quality.csv and _cache/manifest.json (ingested_at, source, rows/first/last/status per file, headline count), save the headlines to _cache/news/headlines.json, print the daily quality table, bars per timeframe, headline count by symbol, SPY and AAPL annualised mean, volatility, skew, excess kurtosis and correlation, a DATA_FORMS table (bars, quotes, trades, corporate actions, fundamentals = structured; news, alternative data = unstructured) and three lines on Volume, Velocity and Variety. Save chart Lab03_market_data.png (universe growth of 100 on a log scale, SPY/AAPL return histograms, SPY 21-day volatility, SPY hourly closes). Exit code 1 if any file FAILs. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline ingest --help shows --symbols and --timeframes.

Step 6: Run the ingestion stage (A6, K9)

Run: python -m pipeline ingest (same as: python3 Lab03-alpaca-market-data/ingest_stage.py).

Check: 12 symbols x 4 timeframes; daily files 2,011 rows each (2018-01-02 to 2025-12-31); 1Day 24,132 bars, 1Week 5,016 (418 per symbol), 1Month 1,152 (96), 1Hour 440 (SPY only offline - 11 symbols 'not available offline'); status CHECK for big moves (NVDA 40, AAPL 12, JPM 12, BAC 11, SPY 5) and OK for GLD and TLT; 12 SYNTHETIC headlines; SPY mean 15.14%, vol 19.46%, excess kurtosis 13.12; AAPL 28.66% / 30.79%; correlation 0.77; 'RESULT: 37 files cached, 0 failed quality checks'.

Step 7: Inspect the cache and the manifest (A6)

Open _cache/manifest.json and _outputs/Lab03_data_quality.csv. Rerun the Step 2 check command - it now reads from the cache. Look at _outputs/Lab03_market_data.png.

Check: manifest.json lists 37 files with rows, first, last and status; the SPY 2025-12-31 close is still 676.64 (the cache is identical to the source).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 8: Design a big-data extension (A6)

Decide which additional data form would most improve the desk's pipeline and what governance it needs.

AI prompt - paste into your AI coding assistant
Using the DATA_FORMS table in labs_fullcode/pipeline/ingest.py, propose how to add Alpaca news for the full trade universe and 1-minute bars for 500 US stocks as scheduled ingest jobs: storage format, refresh frequency, deduplication key, estimated rows per year, and three governance controls (data licence, point-in-time universe, prompt-injection risk when headlines reach an AI agent). Answer in a short table; do not write code yet.

Check: Your table estimates about 49 million one-minute rows a year for 500 stocks and names at least three controls.

Expected Output (Full Code)

python -m pipeline ingest (offline): 37 files cached in _cache/bars, 0 failed quality checks; daily 2,011 bars per symbol, weekly 418, monthly 96, SPY hourly 440 (Q4 2025); big-move flags NVDA 40, AAPL 12, SPY 5; 12 synthetic headlines cached; SPY annualised mean 15.14% and volatility 19.46% (excess kurtosis 13.12), AAPL 28.66% / 30.79%, correlation 0.77; files _outputs/Lab03_data_quality.csv, _cache/manifest.json and chart _outputs/Lab03_market_data.png.

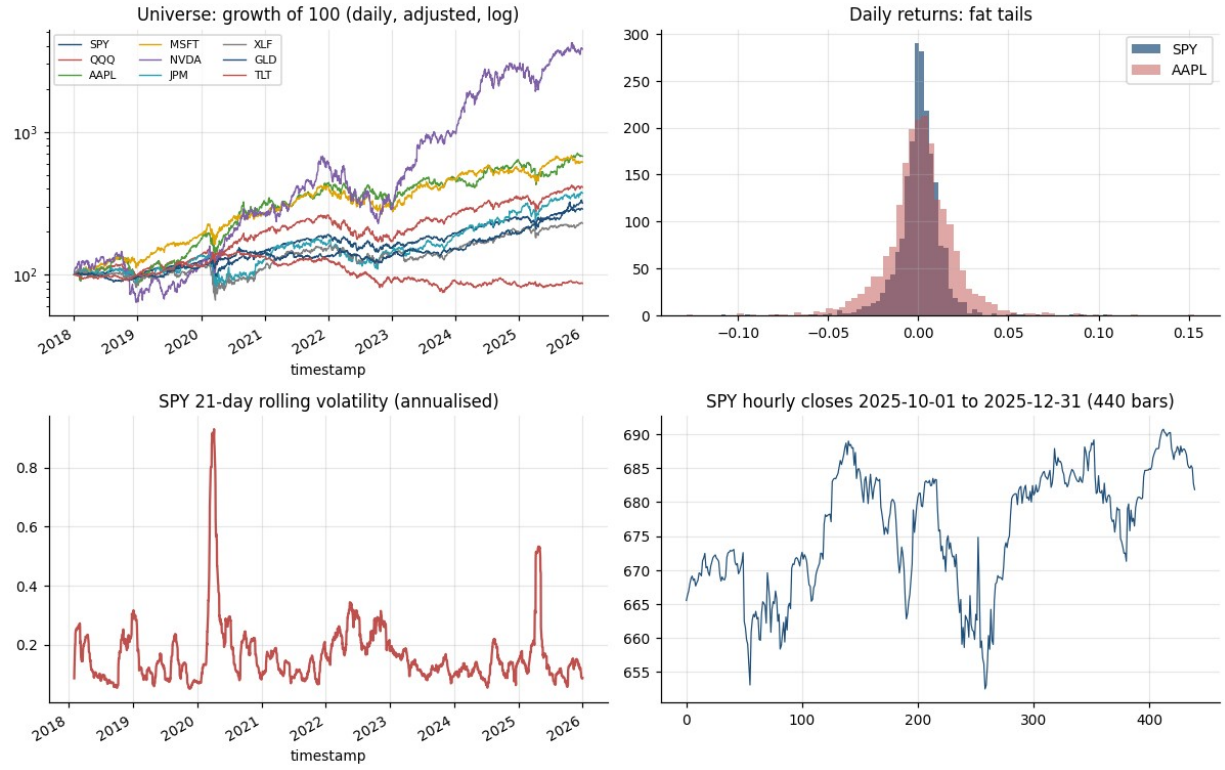


Figure: Lab 3 output (labs_fullcode/_outputs/Lab03_market_data.png), offline mode on the bundled data.

Debrief

Ingestion is the first automated stage: it fixes one version of the truth for every later stage and blocks bad data before a model can learn from it. Bars, quotes, trades, news and alternative data are different forms of financial data (K9) with different volume, velocity and variety - which is what turns a trading data set into a big-data application (A6).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Troubleshooting

Symptom	What to do
Hourly bars 'not available offline' for most symbols	Expected: only SPY hourly data are bundled; online Alpaca returns hourly bars for every symbol.
Alpaca returns fewer hourly bars than expected	The free IEX feed covers only IEX trades; use daily bars or a SIP subscription for full volume.
Status CHECK on many symbols	CHECK means 'look at the big moves' (earnings, splits, 2020 crash); only FAIL blocks a file.
Later stages ignore new data	load_bars reads the cache offline; rerun python -m pipeline ingest after adding data.

1.10 Lab 4: Hypothesis Stage: from a Written Card to an Automated Statistical Test

Maps to: A1, A6, K1 | Duration: 15 min | Slide 73

Scenario

A portfolio manager believes SPY tends to bounce after several losing days. The desk's rule is that no idea enters the signal and backtest stages until it is written as a hypothesis card - before anyone looks at results - and passes an automated statistical test with a decision rule that includes trading costs.

Objective

Write a falsifiable hypothesis card in YAML (H1, H0, signal known at the close, outcome, test, decision rule, big-data use case) and build `python -m pipeline hypothesis` to test it with a Welch t-test, a bootstrap confidence interval and a robustness grid.

Files

labs_fullcode/Lab04-trading-hypothesis/ - Lab04-trading-hypothesis/hypothesis_stage.py, pipeline/hypotheses/spy_dip_after_streak.yaml, pipeline/hypothesis.py, Lab04-trading-hypothesis/solution/hypothesis.py, Lab04-trading-hypothesis/solution/spy_dip_after_streak.yaml

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 04	python3 Lab04-trading-hypothesis/hypothesis_stage.py (same as: python -m pipeline hypothesis)
The complete code	pipeline/hypothesis.py (reference copy: Lab04-trading-hypothesis/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab04-trading-hypothesis/Lab04_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

Step 1: Read the hypothesis card (A1, K1)

Open pipeline/hypotheses/spy_dip_after_streak.yaml. Identify H1, H0, the signal (3 consecutive down closes, known at the close), the outcome (next-day return), the test and the decision rule ($p < 0.05$ AND an edge of at least 3 bps, the estimated round-trip cost).

Check: You can say why the outcome must be a FUTURE return and why the decision rule includes a cost hurdle.

Step 2: Write a card for your own idea (A1, K1)

Copy the card to pipeline/hypotheses/my_card.yaml and adapt it to an idea of your own (for example QQQ after closing below its 200-day average, or AAPL after a losing streak). Write it BEFORE running anything.

AI prompt - paste into your AI coding assistant
Help me write a hypothesis card in YAML for this trading idea: <describe your idea in one sentence>. Use exactly the fields of labs_fullcode/pipeline/hypotheses/spy_dip_after_streak.yaml (id, name, instrument, timeframe, sample, rationale, h1, h0, signal, outcome, test, decision_rule, robustness, biases_to_watch, big_data_use_case). The signal type must be one of losing_streak (n), rsi_below (window, level) or close_below_sma (window) and must be known at the close of day t; the outcome must be a forward return over horizon_days; H0 must be 'no difference'. Do not analyse any data.

Check: my_card.yaml has an economic rationale, H1, H0, a signal known at the close and a decision rule.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 3: Build the test engine (A1)

Ask your assistant for the core of pipeline/hypothesis.py.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/hypothesis.py. losing_streak(close) counts consecutive down closes ending at each day. condition(bars, signal) builds the boolean signal for the types losing_streak, rsi_below (RSI from common.indicators) and close_below_sma, using data up to the close of day t only. test_card(bars, card, signal=None): outcome fwd = close.shift(-horizon) / close - 1 (the outcome, never a feature); compare signal days with all other days: counts, means, hit rates, scipy.stats.ttest_ind(equal_var=False), a bootstrap 95% CI of the signal-day mean with numpy default_rng(card seed) and card bootstrap draws, edge_bps = (mean signal - mean other) x 10,000, supported = p < alpha AND t > 0 AND edge_bps >= min_edge_bps. Load bars with pipeline.ingest.load_bars. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: test_card returns a dict with signal_days, means, ci95, welch_t, p_value, edge_bps and supported.

Step 4: Add the robustness grid, chart and result file (A1, A6)

Ask your assistant for run() and main().

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/hypothesis.py add run(card_path=None) for `python -m pipeline hypothesis [--card PATH]` (default pipeline/hypotheses/spy_dip_after_streak.yaml): print H1, H0, sample, signal days, means, hit rates, CI, Welch t-test, edge and a DECISION line ('SUPPORTED - reject H0; promote the idea to the signal and backtest stages' or 'NOT SUPPORTED - do not reject H0'); a robustness grid over card.robustness.streaks for the instrument and each robustness symbol with a Bonferroni threshold alpha / number of variants; a table of forward return by streak length (0..4, 5+) with standard errors; chart Lab04_hypothesis.png (bars with 95% error bars, unconditional mean line); and write the card plus results to _outputs/Lab04_hypothesis_result.yaml. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline hypothesis --help shows --card.

Step 5: Run the test on the desk's card (A1)

Run: python -m pipeline hypothesis (same as: python3 Lab04-trading-hypothesis/hypothesis_stage.py).

Check: SPY 2018-01-02 to 2025-12-30 (2,010 days): 178 signal days (8.9%); mean next-day return +0.2828% vs +0.0384% on other days (unconditional +0.0601%); hit rate 56.7% vs 55.4%; bootstrap CI [+0.0531%, +0.5136%]; Welch t = 2.03, p = 0.044; edge +24.4 bps; 'DECISION: SUPPORTED'.

Step 6: Read the robustness grid (A1, K1)

Read the robustness table and the Bonferroni line. Which variants would survive if the desk counted every test it ran?

Check: 3 of 6 variants supported: SPY n=3 (p 0.044), QQQ n=2 (p 0.031) and QQQ n=3 (p 0.006); SPY n=2 (p 0.296), SPY n=4 (p 0.053) and QQQ n=4 are not. Bonferroni threshold p < 0.0083 - only QQQ n=3 passes it.

Step 7: Study the shape of the effect (A1)

Read the table 'Forward return by losing-streak length' and open _outputs/Lab04_hypothesis.png.

Check: Streaks of 4 (49 days, +0.439%) and 5+ (27 days, +0.533%) look strongest but have the widest error bars - small samples.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 8: Test your own card and its big-data use case (A1, A6)

Run: python -m pipeline hypothesis --card pipeline/hypotheses/my_card.yaml. Then complete the big_data_use_case section of your card (data, source, tooling, governance, success measure).

AI prompt - paste into your AI coding assistant
Review my hypothesis result in labs_fullcode/_outputs/Lab04_hypothesis_result.yaml. In plain English, say whether the evidence supports H1, how many variants I tested, whether the result survives a Bonferroni correction, and which bigger or alternative data set (volume, velocity or variety) would test the idea better. Do not change any code.

Check: Your card prints a DECISION line and _outputs/Lab04_hypothesis_result.yaml is rewritten for your card.

Expected Output (Full Code)

python -m pipeline hypothesis: SPY streak >= 3 - 178 signal days (8.9%), mean next-day return +0.2828% vs +0.0384% (all days +0.0601%), hit rate 56.7% vs 55.4%, bootstrap CI [+0.0531%, +0.5136%], Welch t = 2.03, p = 0.044, edge +24.4 bps -> SUPPORTED; robustness 3 of 6 variants supported, Bonferroni p < 0.0083; files _outputs/Lab04_hypothesis_result.yaml and chart _outputs/Lab04_hypothesis.png.

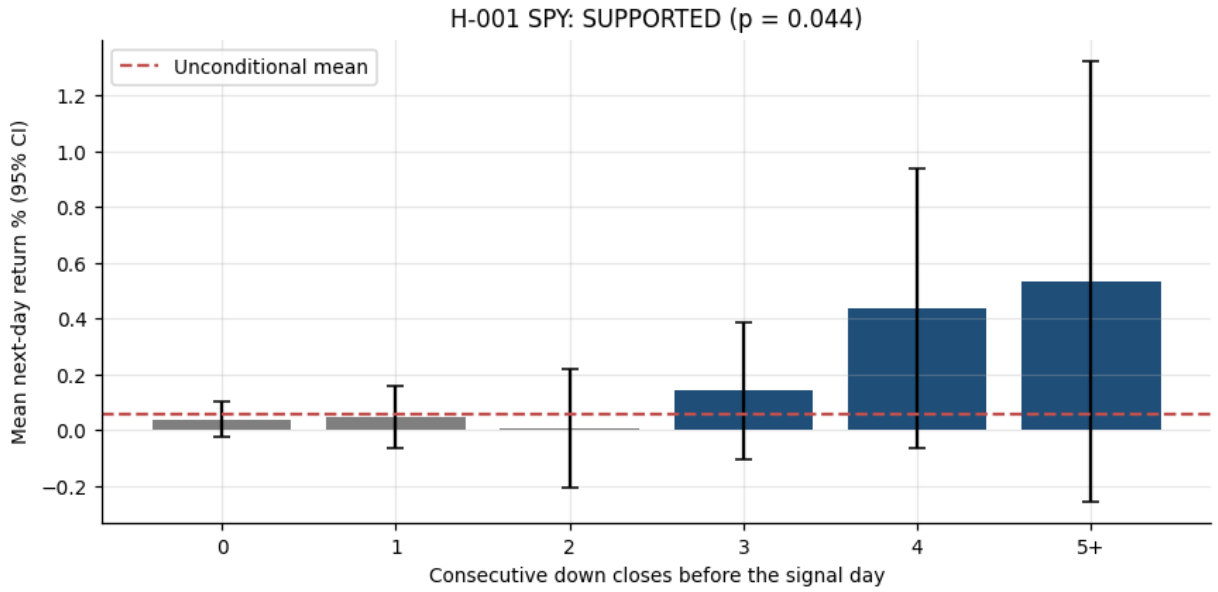


Figure: Lab 4 output (labs_fullcode/_outputs/Lab04_hypothesis.png), offline mode on the bundled data.

Debrief

Writing the hypothesis before testing, with a null, a cost-aware decision rule and a count of the variants tried, is the foundation of data-driven trading (A1). AI and machine-learning methods (K1) only add value when they test well-posed questions, and larger data sets (A6) are how a promising effect in one ETF is confirmed or rejected.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Troubleshooting

Symptom	What to do
p-value changes when you change the dates	Expected - small samples are sensitive. Record it as evidence, not as a problem to hide.
ValueError: unknown signal type	Use losing_streak, rsi_below or close_below_sma, or ask your assistant to add a new type to condition().
Very few signal days	With fewer than about 30 observations the test has little power; loosen the condition.
Unsure what H0 should be	H0 is 'no difference': the mean forward return on signal days equals the mean on other days.

1.11 Topic Summary

- A1 - Develop hypotheses for implementing data analytics and computational modelling activities
- A6 - Identify possible big data applications
- K1 - Data analytics and modelling business use cases
- K9 - Forms of data within financial services

Assessed by: Practical Performance Task 1 | Written Assessment Q1, Q9

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Topic 2: Technical Indicators, Trading Strategies and Metrics with TradingView and Alpaca

Day	Abilities	Knowledge	Labs	Slides
Day 1	A2, A3	K3, K5, K7	5 - 9	76 - 140

Learning Outcome

Identify appropriate statistical algorithms and data models to test trading hypotheses, and use appropriate analytics platforms, analytical tools and statistical methods given specific analytics and reporting requirements.

2.1 Technical Indicators: Trend, Momentum, Volatility and Volume

Maps to: K3 | Slides 77-88

- Simple Moving Average (SMA): the average close over n bars. Exponential Moving Average (EMA): a weighted average with smoothing factor 2/(n+1), reacting faster to recent prices. Both are trend indicators.
- Relative Strength Index (RSI): 100 - 100/(1 + RS), where RS is the ratio of average gains to average losses over 14 bars (Wilder smoothing). Readings above 70 are conventionally 'overbought', below 30 'oversold'.
- The number in brackets is the look-back period. RSI(2) uses the last 2 bars and swings to extremes after a 1-2 day move (SPY RSI(2) was below 10 on 9.6% of days in 2018-2025); RSI(14) moves slowly (below 30 on 1.7% of days). On 10 October 2025 SPY fell 2.7%: RSI(2) dropped to 5.8 while RSI(14) read a neutral 42.5. SMA(50), EMA(20), Bollinger(20, 2) and ATR(14) follow the same convention.
- MACD: the 12-period EMA minus the 26-period EMA, with a 9-period EMA 'signal line'. A MACD cross above its signal line is a momentum buy signal.
- Bollinger Bands: a 20-period SMA plus and minus 2 standard deviations. The bands widen in volatile markets and narrow in quiet ones; closes outside the bands feed both mean-reversion and breakout strategies.
- Average True Range (ATR): the average of the true range (the largest of high-low, |high-previous close| and |low-previous close|). Used to size positions and to set volatility-based stops.
- Volume indicators: Volume-Weighted Average Price (VWAP), the benchmark price for execution quality, and On-Balance Volume (OBV), which accumulates volume on up days and subtracts it on down days.
- Every indicator is a statistical transformation of past prices; none predicts the future on its own. Indicators lag, and many are highly correlated with each other - adding more rarely adds information.

2.2 Classification of Algorithmic Trading and Quant Algorithms

Maps to: K3, A2 | Slides 89, 95-117

- By purpose, trading algorithms fall into seven families: execution, market making, arbitrage, directional (alpha), statistical arbitrage and pairs (hedged), factor and portfolio construction, and hedging and risk control.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

- Execution algorithms work a large order at low cost and market impact: TWAP slices evenly over time; VWAP follows the day's volume curve; percentage-of-volume (POV) trades a fixed share of live volume; implementation shortfall minimises cost versus the arrival price; iceberg orders show only part of the size; smart order routing chooses the venue.
- The SEC's 2010 Concept Release on Equity Market Structure groups high-frequency strategies into four types: passive market making (resting bids and offers that provide liquidity), arbitrage (pricing inefficiencies between related products, such as an ETF and its basket), structural (exploiting structural vulnerabilities, such as stale quotes), and directional (short-term positions anticipating an intraday move).
- Directional (alpha) strategies: trend following (moving-average crossover), momentum (hold recent winners), breakout (new n-day high), mean reversion (Bollinger Bands, RSI(2)) and event or news-driven trading.
- Long and short positions: a long buys first and profits when the price rises (the most it can lose is the amount paid); a short borrows shares and sells them first, buys them back later and profits when the price falls (losses are unlimited, and it needs a margin account - Alpaca requires at least $2,000 of equity). Selling shares you own only closes a long.
- Mean reversion buys when the price is stretched below its normal range (a close below the lower Bollinger Band, RSI(14) below 30, RSI(2) below 10, or a losing streak) and exits at the average. Compared with trend following it has a high win rate but rare large losses when a dip becomes a crash, so it needs a stop-loss or time limit, a trend filter (price above its 200-day SMA) and stable markets.
- Statistical arbitrage and pairs trading are hedged: long one asset and short a related one, so most market moves cancel and the profit comes from the spread reverting (Topic 3.6).
- By holding period: high-frequency (microseconds to seconds), intraday (minutes to hours), swing (days to weeks) and position (weeks to months). By model: rule-based, statistical, machine learning, reinforcement learning, and LLM / agentic.
- Matching the algorithm to the hypothesis - and to the right statistical test - is the A2 skill: a difference in means needs a t-test or bootstrap, a direction forecast needs a classifier, a regime question needs clustering, and a spread needs a cointegration test.

2.3 Analytics Platforms and Tools: TradingView versus Alpaca, and Python

Maps to: K5, A3 | Slides 118-120, 122

- TradingView is a charting and strategy-design platform: global markets including SGX, built-in indicators, the Pine Script language, the Strategy Tester for backtests, alerts and webhooks (webhooks need a paid plan), and a built-in Paper Trading account opened from the chart's Trading Panel with your own starting capital and commission.
- Alpaca is an API-first US broker: commission-free US stocks and ETFs, plus options and crypto; a free paper-trading account that mirrors the live API; the Trading API (account, orders, positions, clock) and the Market Data API (bars, quotes, trades, news; the free plan uses the IEX feed); and the official alpaca-py Python SDK.
- The key difference: TradingView is where a strategy is designed, visualised and first tested; Alpaca is where a program places and manages orders automatically. TradingView has no machine learning and a simplified backtester; Alpaca has no charting or strategy tester - you build those in Python.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

- Trading hours for automation from Singapore: Alpaca offers 24/5 trading of US stocks (8 pm ET Sunday to 8 pm ET Friday) - outside the regular session only limit orders with extended hours enabled are accepted, at lower liquidity.
- They work together: a TradingView alert can send a webhook to your server, which places the order on Alpaca; and an Alpaca account can be connected in TradingView's trading panel to trade from the chart.
- Python - pandas, NumPy, scikit-learn, XGBoost, SciPy, matplotlib and alpaca-py - joins them: data handling, machine learning, statistical tests, cost-aware backtests and the automation loop.
- API keys are credentials: keep them in environment variables or a .env file, never in code you share, and use separate paper and live keys.

2.4 Coding Languages for Algorithms and Signals, and Order Placement

Maps to: K7 | Slides 90-91, 121

- Pine Script v6 (TradingView's language): every script starts with //@version=6 and declares indicator() or strategy(). Built-in functions such as ta.ema, ta.rsi, ta.crossover and strategy.entry express a signal in a few lines.
- Python: the dominant language for quantitative research - readable, with mature data and machine learning libraries. A signal is typically a pandas Series of +1 / 0 / -1 aligned to the bar index.
- Other languages: C++ and Java for low-latency execution systems, R for statistics, SQL for data stores, and MQL for MetaTrader. LLMs can now draft code in all of them, but the developer remains accountable for testing it.
- Order types on Alpaca: market (fill now at the best price), limit (fill only at the limit price or better), stop (becomes a market order when the stop is hit), stop-limit, and trailing stop.
- Time in force: day, gtc (good till cancelled), opg (market on open), cls (market on close), ioc (immediate or cancel) and fok (fill or kill). Order classes: simple, bracket (entry with take-profit and stop-loss), OCO and OTO.
- The order lifecycle: new, accepted, partially filled, filled, cancelled, expired or rejected. A program must read the order status rather than assume a fill.
- Paper trading simulates fills against real market quotes but cannot model your own market impact or queue position - a limitation to remember when comparing paper and live results.

2.5 Performance and Risk Metric Definitions

Maps to: K3, A3 | Slides 92-94

- Return metrics: total return; compound annual growth rate CAGR = (ending value / starting value)^(1/years) - 1; and the benchmark-relative excess return.
- Risk metrics: annualised volatility = standard deviation of daily returns x sqrt(252); maximum drawdown = the largest peak-to-trough fall in the equity curve.
- Risk-adjusted metrics: Sharpe ratio = (mean return - risk-free rate) / standard deviation, annualised by sqrt(252); Sortino ratio uses only downside deviation; Calmar ratio = CAGR / |maximum drawdown|.
- Trade metrics: number of trades, win rate (% of profitable trades), profit factor (gross profit / gross loss), average win and loss, expectancy = win rate x average win - loss rate x average loss.
- Activity metrics: exposure (% of time in the market) and turnover (traded value / average capital). High turnover multiplies the drag of trading costs.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

- Market-relative metrics: beta (sensitivity to the benchmark) and alpha (return not explained by beta), estimated by regressing strategy returns on benchmark returns.
- No single metric is enough: report return, risk and cost together, and always against a buy-and-hold benchmark for the same period.

Reference: Classification of Trading Algorithms

Family	Purpose	Examples
Execution algorithms	Work a large order at low cost and market impact	VWAP, TWAP, POV, implementation shortfall, iceberg, smart order routing
Market making	Quote bids and offers, earn the spread, manage inventory	Passive market making with resting limit orders
Arbitrage	Capture mispricing between related products or markets	ETF versus its basket, index futures versus stocks
Directional / alpha	Take a view on the direction of price	Trend following, momentum, breakout, mean reversion, news
Statistical arbitrage and pairs	Hedged: long one asset, short a related one; trade the spread	Pairs trading, cointegration, market-neutral baskets
Factor and portfolio	Build and rebalance portfolios from return drivers and risk	Value, momentum, size factors; mean-variance optimisation
Hedging and risk	Offset unwanted exposure and enforce limits	Delta hedging, beta hedging, pairs hedge, kill switch

SEC HFT strategy type (2010)	Description
Passive market making	Submits non-marketable resting bids and offers that provide liquidity at specified prices
Arbitrage	Captures pricing inefficiencies between related products or markets, e.g. an ETF and its basket
Structural	Exploits structural vulnerabilities in the market or its participants, e.g. stale quotes via faster data
Directional	Takes a short-term position anticipating an intraday price move in a particular direction

Holding period	Horizon	Typical algorithms
High-frequency	Microseconds to seconds	Market making, ETF and latency arbitrage
Intraday	Minutes to hours	VWAP / TWAP execution, intraday momentum
Swing	Days to weeks	Mean reversion, pairs trading
Position	Weeks to months	Trend following, factor portfolios

Model type	Description
Rule-based	Fixed if-then rules on indicators, e.g. a moving-average crossover
Statistical	Econometric tests and models: cointegration, regression, t-tests
Machine learning	Models learn from labelled data: logistic regression, random forest, XGBoost
Reinforcement learning	Learns a policy from rewards, e.g. order-execution research
LLM and agentic	Language models read text and agents plan, code and act under limits

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Strategy	Idea	Example rule
Trend following	Rides sustained moves in the direction of the trend; cuts losers quickly	Buy when the 50-day moving average crosses above the 200-day
Mean reversion	Bets that prices stretched far from their average will snap back	Buy when price closes below the lower Bollinger Band (20-day, 2 std dev)
Momentum and breakout	Buys recent winners or prices breaking out of a range, expecting them to keep going	Buy on a close above the 20-day high (Donchian channel breakout)
Statistical / pairs arbitrage	Trades the spread between two related assets when it drifts from its normal level	Short the rich leg, buy the cheap leg when the spread z-score exceeds 2
Execution algorithms	Split a large order to minimise cost and market impact: VWAP, TWAP, POV, implementation shortfall	POV: trade 10% of market volume until the order is filled
Market making	Continuously quotes buy and sell prices, earning the spread while managing inventory	Quote bid and ask around mid-price; skew quotes as inventory builds
Index rebalancing / event-driven	Trades predictable flows around index changes, earnings and corporate events	Position ahead of announced index inclusions on the rebalance date
Sentiment / NLP	Turns news, filings and social media text into trading signals using NLP or LLMs	Go long when the average FinBERT headline sentiment is strongly positive

Comparison: TradingView versus Alpaca

	TradingView	Alpaca
What it is	Charting and strategy-design platform	API-first US broker for automated trading
Markets	Global charts incl. SGX, US, FX, crypto	US stocks and ETFs, options, crypto
Strategy design	Pine Script v6 indicators and strategies	Write it yourself in Python (alpaca-py)
Backtesting	Strategy Tester (simplified costs and fills)	None built in - you build a cost-aware backtest
Paper trading	Built-in Paper Trading account from the chart	Paper account mirroring the live API
Automation	Alerts; webhooks on paid plans	Trading API: orders, positions, clock; 24/5 trading
Data	Charts and Data Window	Market Data API: bars, quotes, trades, news (IEX free)
Machine learning	Not supported	Any Python library (scikit-learn, XGBoost)
Best used for	Designing, visualising and first-testing a rule	Executing and forward-testing it automatically

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Strategy Families: the Idea, When It Works, When It Will Not

Family	The idea	When it works	When it will not work
Trend Following (Lab 7: EMA 20/50 on SPY)	Prices that have started to move tend to keep moving. Buy when a fast average is above a slow one (or price is above its 200-day average); exit when the trend turns.	Long, persistent moves - bull or bear markets, macro trends lasting months. A few big winners pay for many small losses.	Sideways, choppy markets: price crosses the averages again and again (whipsaws), every crossing costs money, and signals arrive late at turns.
Mean Reversion (Lab 7: Bollinger 20 / k = 2.0 and 1.5 on KO)	Like a rubber band, short-term over-reactions snap back. Buy when price is stretched below its normal range (lower Bollinger Band, RSI(2) below 10) and sell when it returns to the average.	Range-bound, liquid markets and stable large stocks, where extremes tend to snap back within days.	A real trend or a news shock: 'cheap' keeps getting cheaper. A high win rate hides rare large losses - as in March 2020.
Momentum (Lab 7: 126-day time-series momentum on QQQ)	Winners keep winning for months. Buy what has risen over the last 6-12 months (time-series) or what has risen most among its peers (cross-sectional).	Persistent market or sector themes, spread across many assets and rebalanced regularly.	Sharp reversals - 'momentum crashes' such as the 2009 rebound, when last year's losers rally hardest - and choppy markets that churn the portfolio.
Breakout (Lab 7: Donchian 20 / 10 on QQQ)	A close above the recent range (for example the 20-day high, the Donchian channel) signals the start of a new move: buy the breakout, exit on a shorter low.	Volatility expanding after a quiet period and a new trend starting - indices and commodities in strong regimes.	False breakouts in a range: price pokes above the high and falls back. A low win rate, so exits must be disciplined and costs kept low.

Key Metrics Defined

Sharpe Ratio

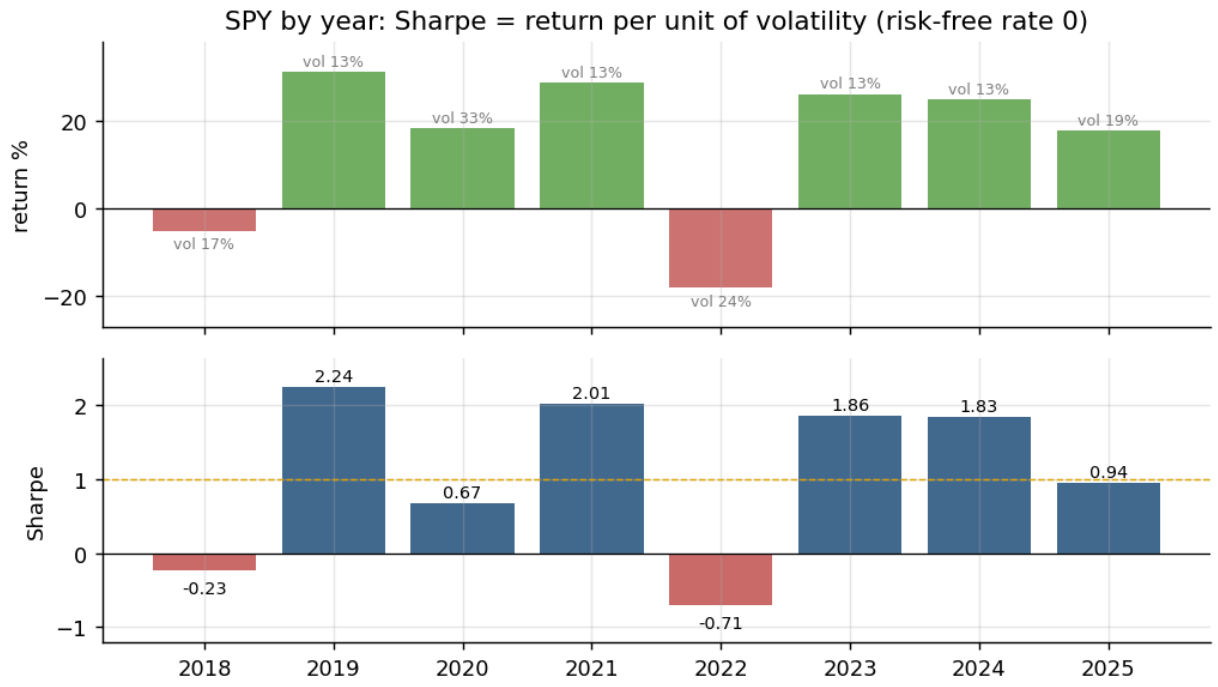


Figure: Metric: Sharpe Ratio - SPY 2018-2025.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Definition	Return earned per unit of risk taken: the average excess return divided by its volatility, annualised.
Formula	Sharpe = (mean daily return - daily risk-free rate) / std of daily returns x sqrt(252).
How to read it	Below 0.5 weak, about 1 good, above 2 rare for a daily strategy - check for errors. Always compare with buy-and-hold over the same period.
Insight	Volatility counts as risk whether it is up or down, and a short or costless backtest inflates it - 2020 returned 18% but at 33% volatility, a Sharpe of only 0.67.

Maximum Drawdown

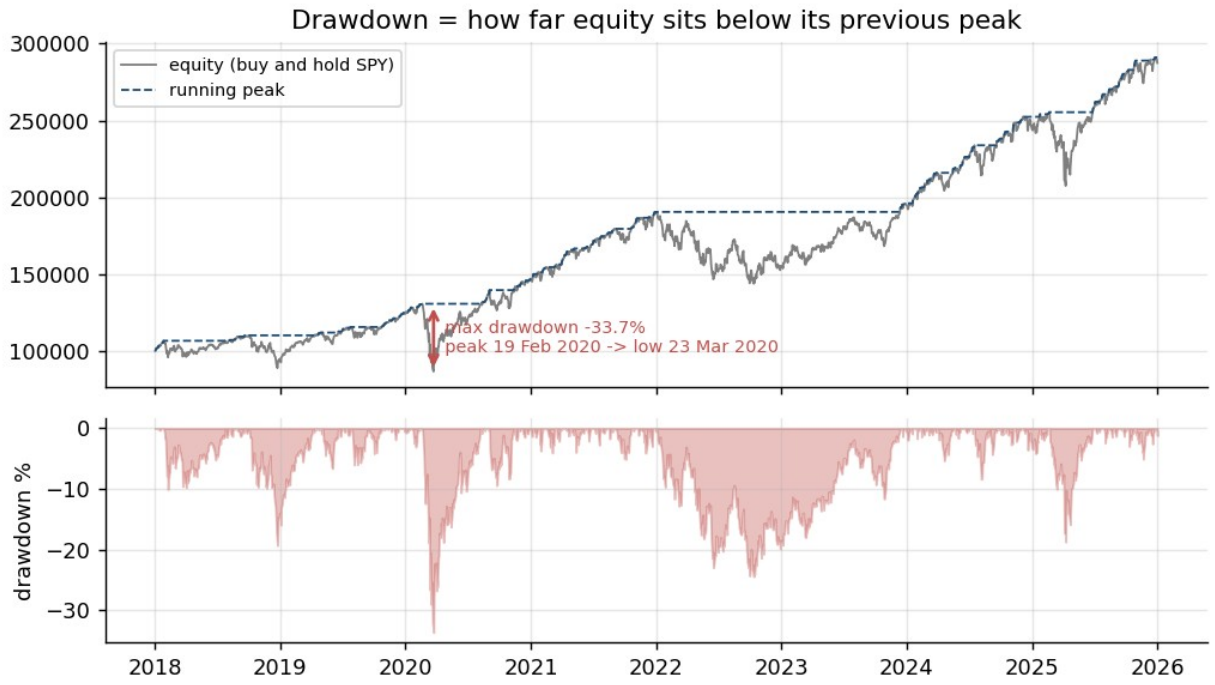


Figure: Metric: Maximum Drawdown - SPY 2018-2025.

Definition	The largest fall of the equity curve from a previous peak to a later low - the worst loss an investor would have lived through before a new high.
Formula	Drawdown(t) = equity(t) / running peak(t) - 1. Max drawdown = the most negative drawdown over the period.
How to read it	A -20% drawdown needs +25% to recover; -50% needs +100%. Compare with buy-and-hold and with the loss the investor can tolerate.
Insight	One path-dependent number: it does not say how long recovery took, and a short backtest may simply miss a bad period. SPY 2018-2025: -33.7% in five weeks of 2020.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Indicator by Indicator

Moving Averages (SMA and EMA)

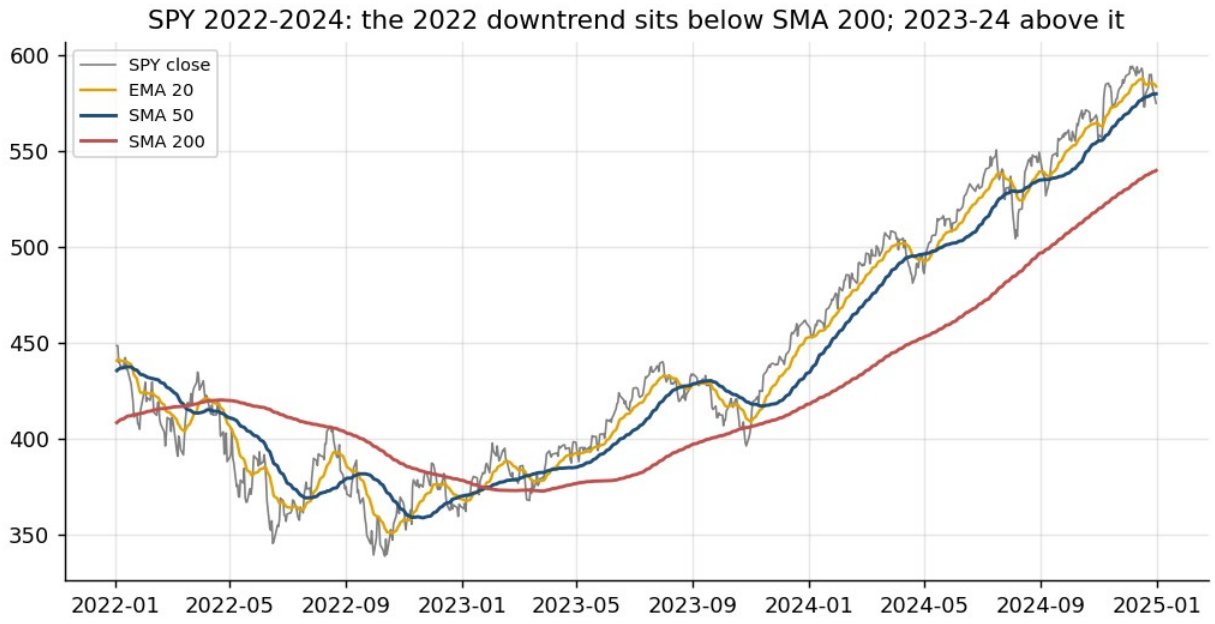


Figure: Indicator: Moving Averages (SMA and EMA) - on real market data.

Definition	The average closing price over the last n bars - a smoothed line that shows the trend. The EMA weights recent bars more, so it turns sooner.
Formula	SMA(n) = (C1 + C2 + ... + Cn) / n. EMA(t) = a x C(t) + (1 - a) x EMA(t-1), with a = 2 / (n + 1).
How to read it	Price above a rising average = uptrend. The 50 / 200-day pair is the classic trend filter; EMA 20 tracks the short-term trend.
Insight	Every average lags: the longer n, the smoother - and the later each signal. In sideways markets price whipsaws across it.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

MACD (Moving Average Convergence Divergence)

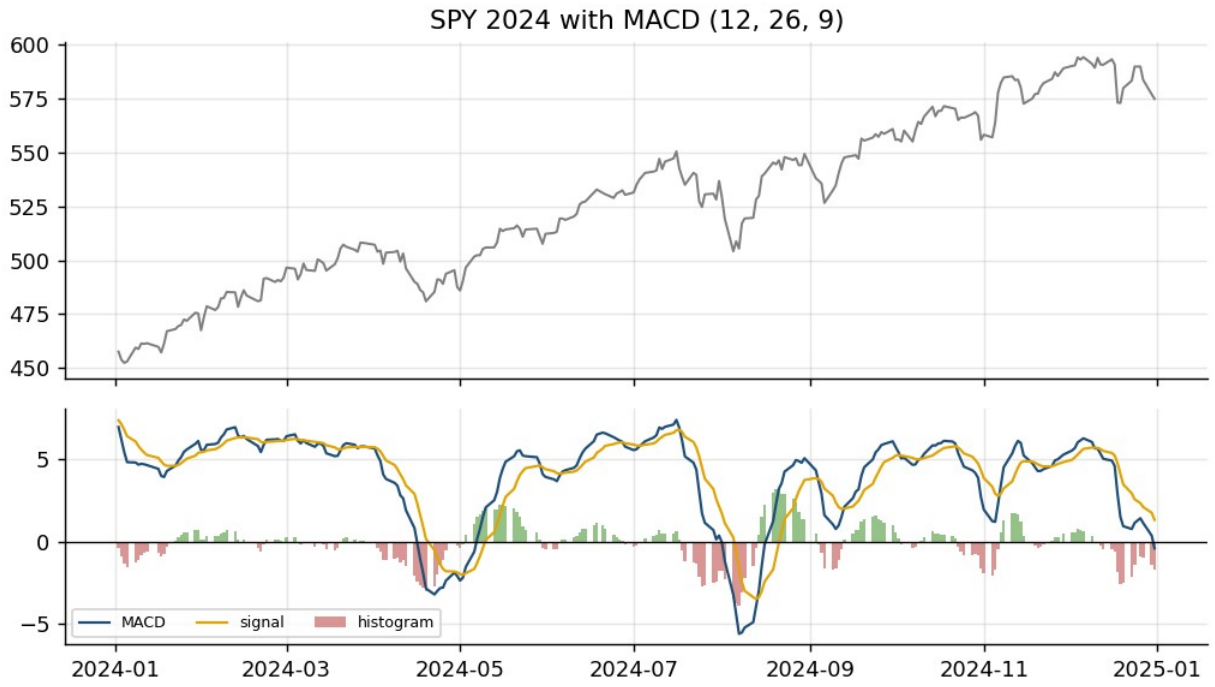


Figure: Indicator: MACD (Moving Average Convergence Divergence) - on real market data.

Definition	A momentum-of-trend gauge: the gap between a fast and a slow EMA, with a signal line and a histogram that show whether momentum is growing.
Formula	MACD = EMA12(C) - EMA26(C). Signal = EMA9(MACD). Histogram = MACD - Signal.
How to read it	MACD crossing above its signal line = momentum turning up; MACD above zero = the 12-day trend is above the 26-day trend.
Insight	A shrinking histogram warns that a move is losing strength before price turns - but it is built from EMAs, so it lags and whipsaws in ranges.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

RSI (Relative Strength Index)

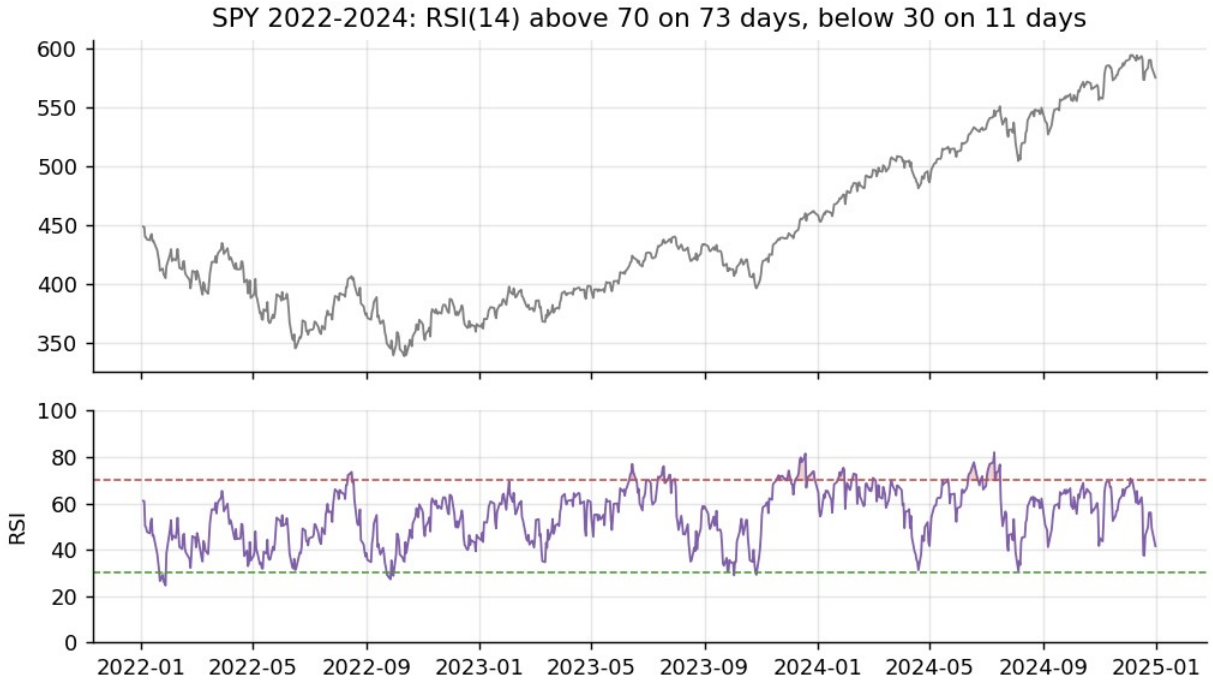


Figure: Indicator: RSI (Relative Strength Index) - on real market data.

Definition	An oscillator from 0 to 100 that compares the size of recent gains with recent losses - how stretched the latest move is.
Formula	RS = average gain / average loss over n bars (Wilder smoothing, n = 14). RSI = 100 - 100 / (1 + RS).
How to read it	Above 70 = overbought, below 30 = oversold. Short look-backs such as RSI(2) are used for mean-reversion entries.
Insight	In a strong trend RSI can stay above 70 for weeks: overbought is not a sell signal by itself. Test the thresholds - they are conventions.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Donchian Channel

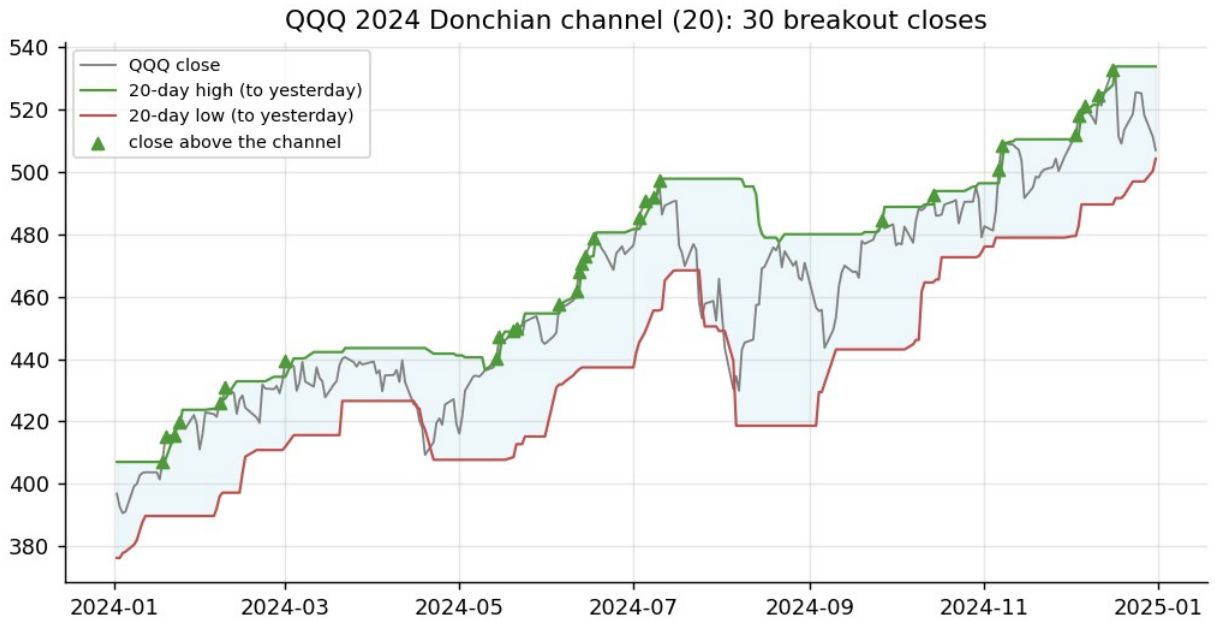


Figure: Indicator: Donchian Channel - on real market data.

Definition	A channel drawn through the highest high and the lowest low of the last n bars - the levels a breakout must clear.
Formula	Upper(n) = max(High, last n bars). Lower(n) = min(Low, last n bars). Classic n = 20 (the Turtle traders used 20 and 55).
How to read it	A close above the prior upper band = upside breakout; exit on a break of a shorter (e.g. 10-day) low.
Insight	Breakout systems win rarely but big. Compare today's close with the channel up to YESTERDAY - including today's high is look-ahead.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Bollinger Bands

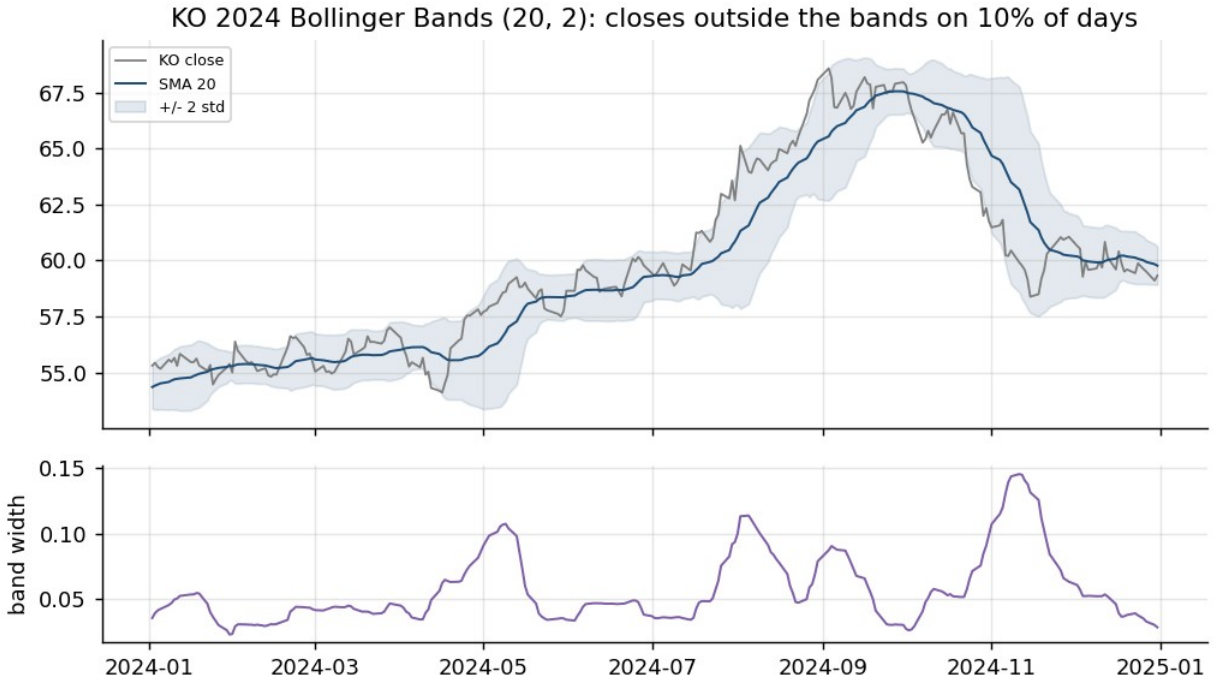


Figure: Indicator: Bollinger Bands - on real market data.

Definition	A moving average with bands set a number of standard deviations above and below it, so the bands widen and narrow with volatility.
Formula	Middle = SMA20(C). Upper / Lower = Middle +/- 2 x StDev20(C). %B = (C - Lower) / (Upper - Lower).
How to read it	Mean reversion: buy a close below the lower band, exit at the middle. A 'squeeze' (narrow bands) often comes before a big move.
Insight	Returns are not normal: closes fall outside the bands far more often than the 5% a normal curve implies. In a trend, 'oversold' keeps falling.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

ATR (Average True Range)

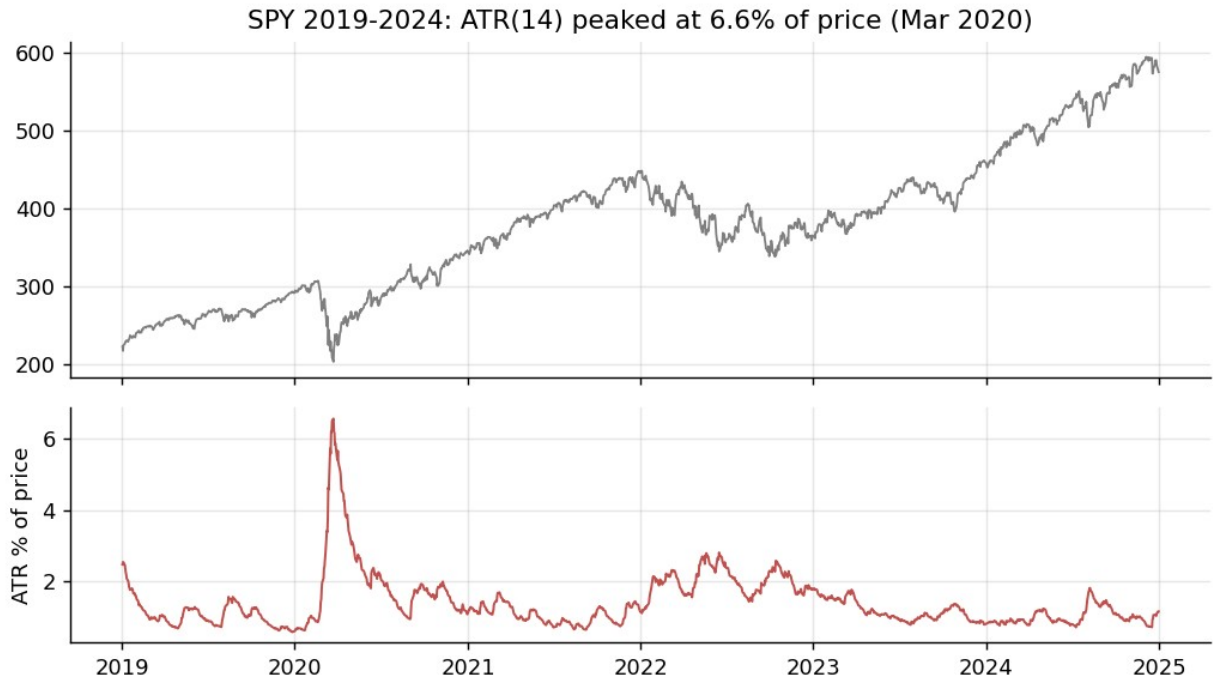


Figure: Indicator: ATR (Average True Range) - on real market data.

Definition	The average size of a day's price range, including overnight gaps - a volatility ruler in price units.
Formula	TR = max(H - L, |H - previous C|, |L - previous C|). ATR = average of TR over n bars (Wilder smoothing, n = 14).
How to read it	Set stops at 2-3 x ATR and size positions as risk budget / (k x ATR), so every trade carries similar risk.
Insight	ATR says how far, not which way. Compare instruments with ATR as a % of price, since ATR rises with the price level.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

VWAP (Volume-Weighted Average Price)

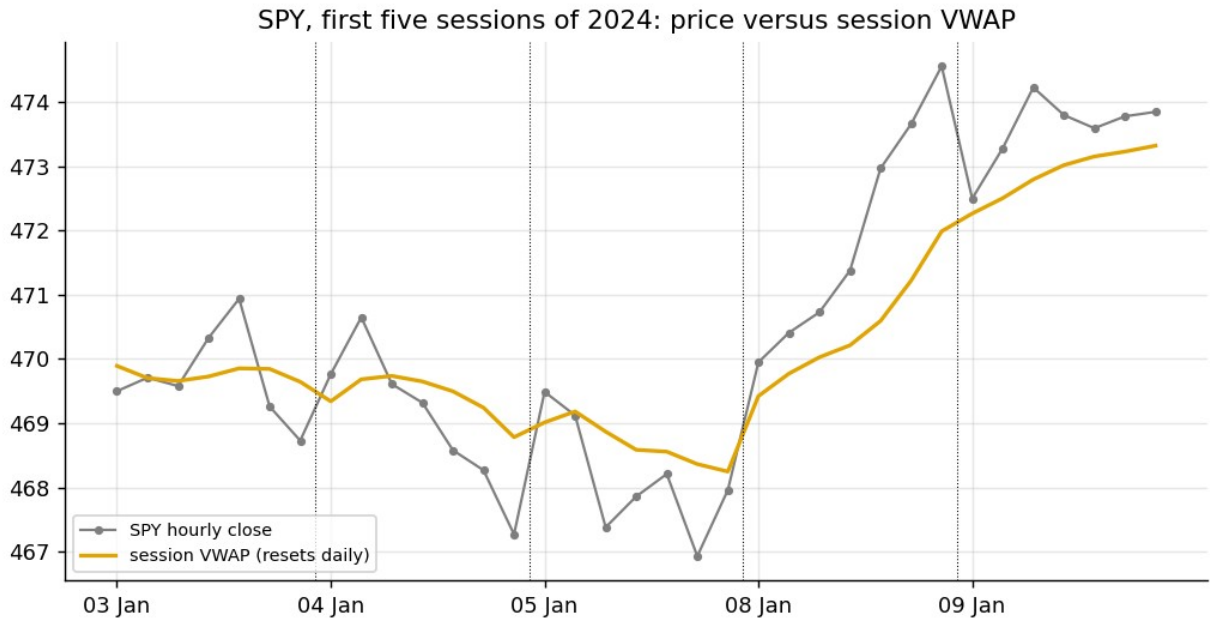


Figure: Indicator: VWAP (Volume-Weighted Average Price) - on real market data.

Definition	The average price paid during the session, weighted by the volume traded at each price. It resets at every session open.
Formula	Typical price P = (H + L + C) / 3. VWAP = sum(P x Volume) / sum(Volume), summed from the session open.
How to read it	The execution benchmark: buying below VWAP is a good fill. Intraday traders read price above VWAP as buyers in control.
Insight	Institutions judge execution algorithms against VWAP. It needs intraday data - on daily bars it is only an approximation.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

OBV (On-Balance Volume)

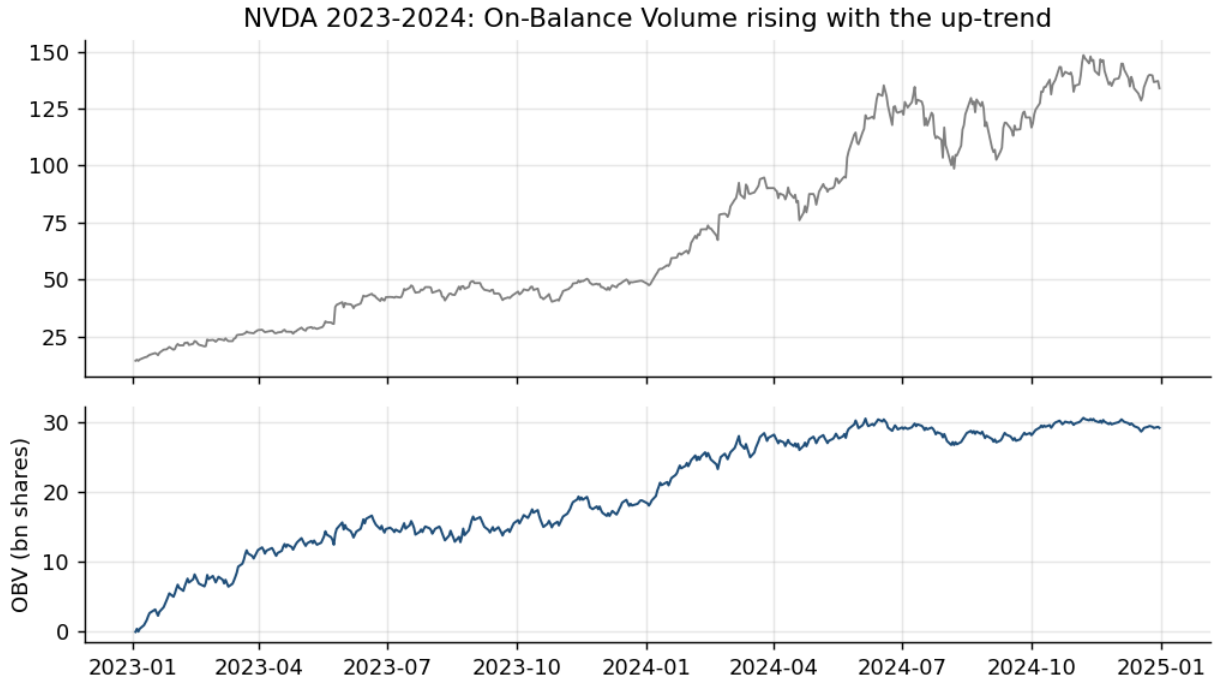


Figure: Indicator: OBV (On-Balance Volume) - on real market data.

Definition	A running total of volume that adds a day's volume when the close rises and subtracts it when the close falls.
Formula	OBV(t) = OBV(t-1) + Volume(t) if C(t) > C(t-1); - Volume(t) if C(t) < C(t-1); unchanged if equal.
How to read it	A rising OBV confirms an up-trend; OBV making new highs before price is read as accumulation. Read the slope, not the level.
Insight	Volume shows the conviction behind a move. An OBV that falls while price rises (divergence) is a warning, not a signal on its own.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

How It Works: Moving-Average Crossover (Trend Following)

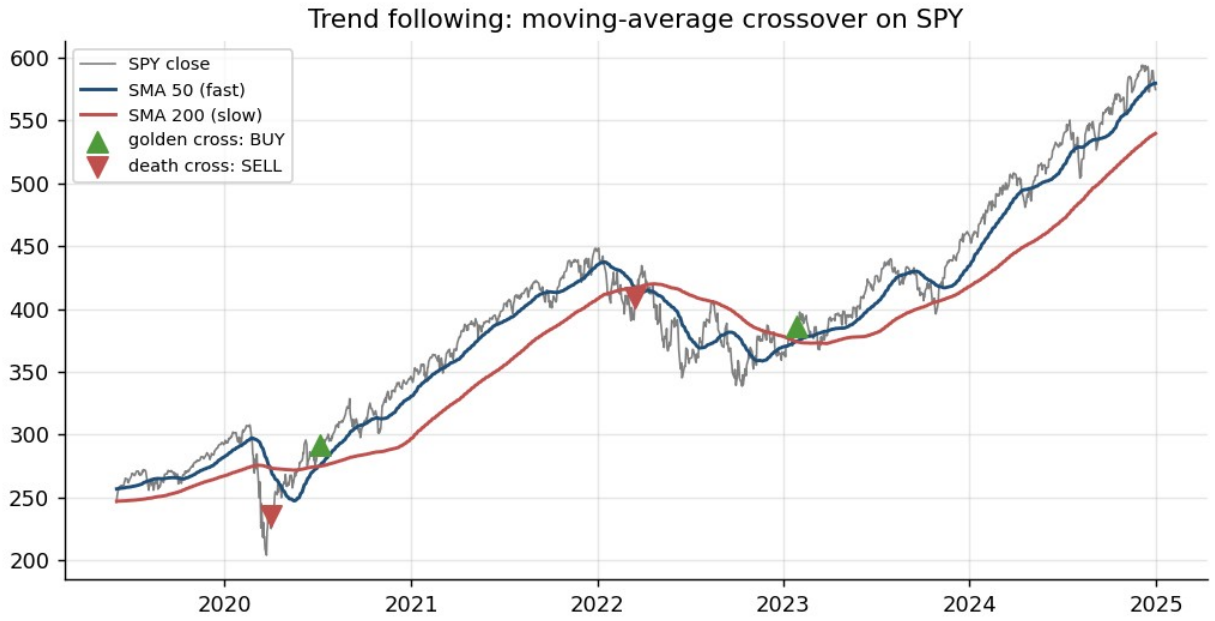


Figure: Algorithm: Moving-Average Crossover (Trend Following) - illustrated on real market data.

How it works	Two moving averages of the close: a fast one reacts quickly, a slow one shows the trend.
Rule	Fast crosses ABOVE slow (golden cross): buy. Fast crosses BELOW (death cross): sell or go flat.
Works best when	Markets trend for months - it rides the move and cuts losers when the trend turns.
Watch out for	Whipsaws in sideways markets; signals lag; trade on the next bar, not the crossing bar.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

How It Works: Bollinger Band Mean Reversion

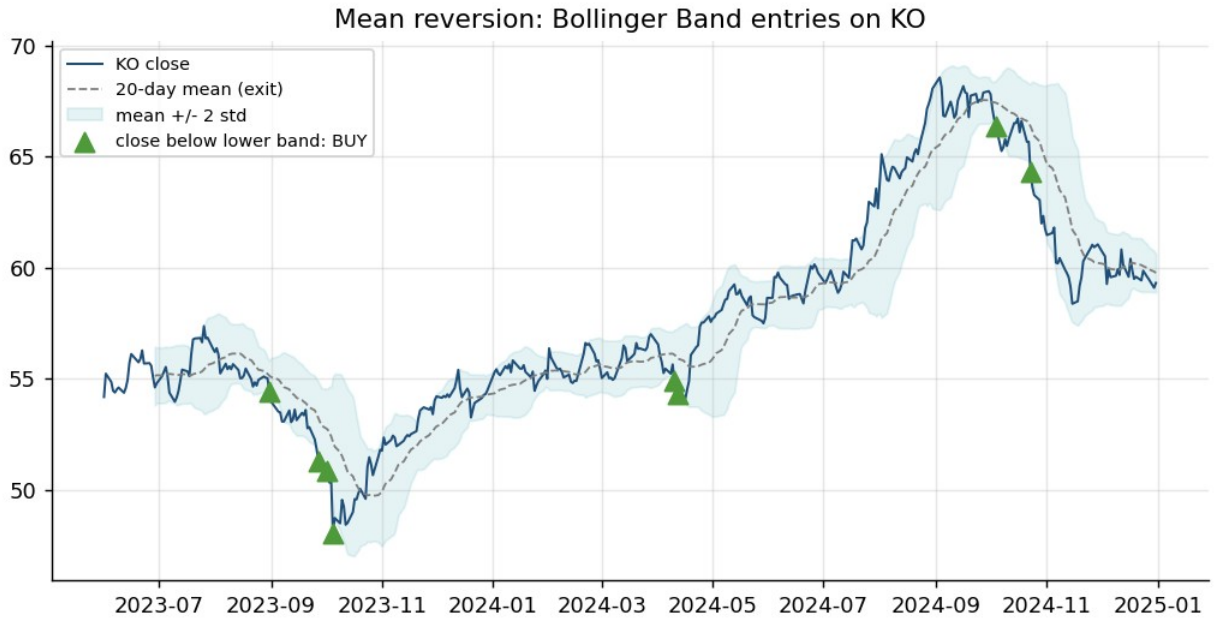


Figure: Algorithm: Bollinger Band Mean Reversion - illustrated on real market data.

How it works	Bands = 20-day mean +/- 2 standard deviations. A close outside the band is 'stretched'.
Rule	Close below the lower band: buy. Exit when the price returns to the middle band (the mean).
Works best when	Range-bound markets where over-reactions snap back, e.g. a defensive stock such as KO.
Watch out for	Strong trends keep 'stretching' - add a stop-loss or a trend filter; many small wins, rare big losses.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

How It Works: RSI and MACD Momentum Oscillators

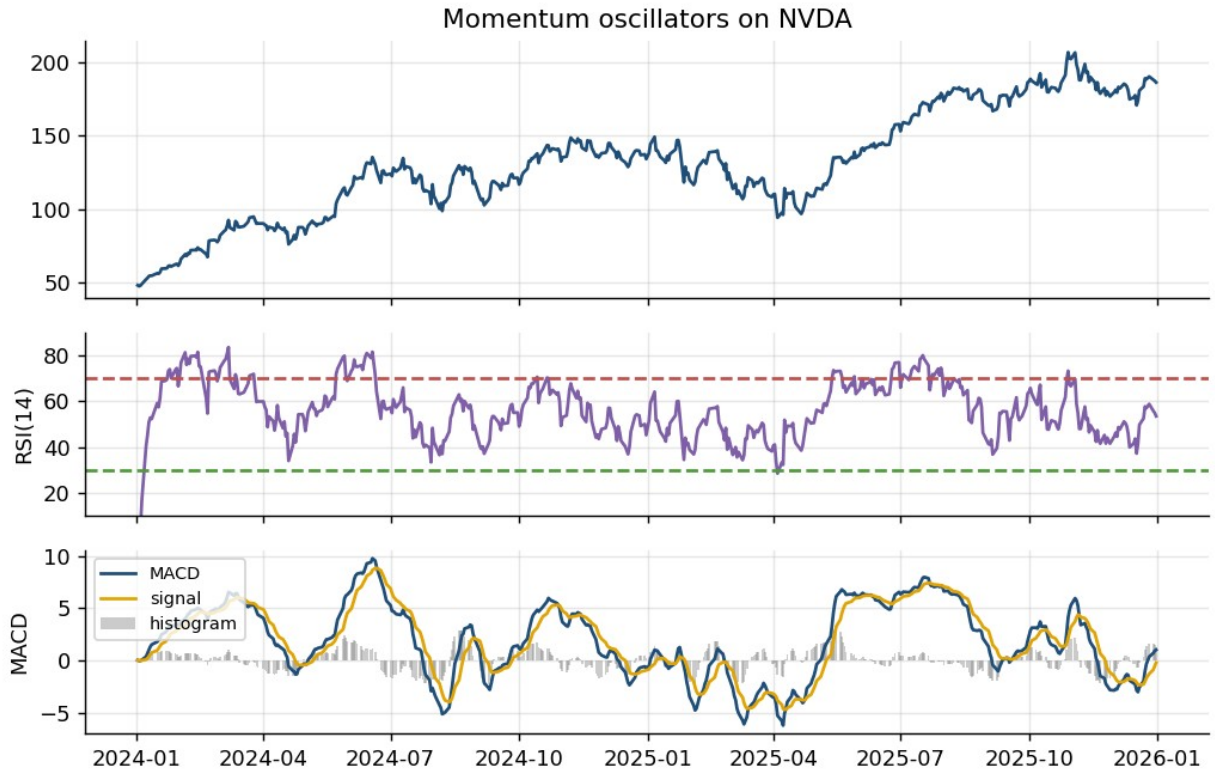


Figure: Algorithm: RSI and MACD Momentum Oscillators - illustrated on real market data.

How it works	RSI = 100 - 100/(1+RS), RS = average gain / average loss over 14 bars. MACD = EMA12 - EMA26.
Rule	RSI < 30 oversold / > 70 overbought. MACD crossing above its 9-EMA signal line = momentum buy.
Works best when	As filters or confirmation for a trend or mean-reversion rule, not as stand-alone predictors.
Watch out for	Oscillators can stay overbought in strong trends; RSI and MACD are highly correlated with price.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

How It Works: Cross-Sectional Momentum

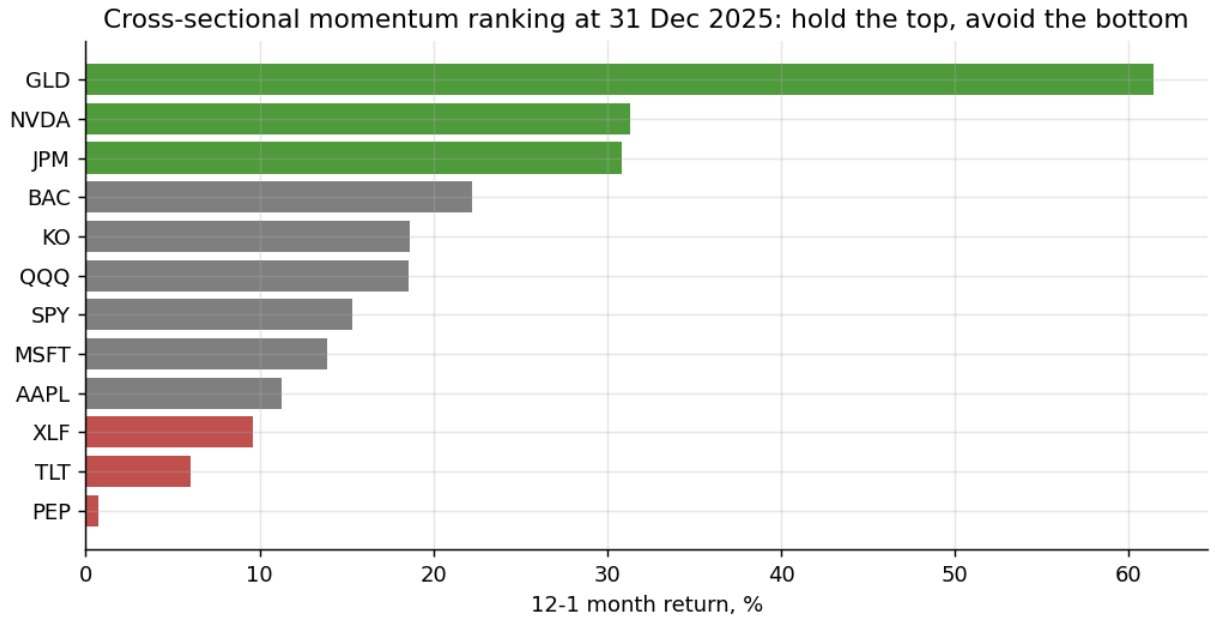


Figure: Algorithm: Cross-Sectional Momentum - illustrated on real market data.

How it works	Rank a universe by past return (e.g. 12 months, skipping the last month) each month.
Rule	Hold the top-ranked assets, avoid or short the bottom-ranked; rebalance monthly.
Works best when	Information spreads slowly and winners keep winning for several months.
Watch out for	Sharp 'momentum crashes' when markets reverse; turnover costs at every rebalance.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

How It Works: Donchian Channel Breakout

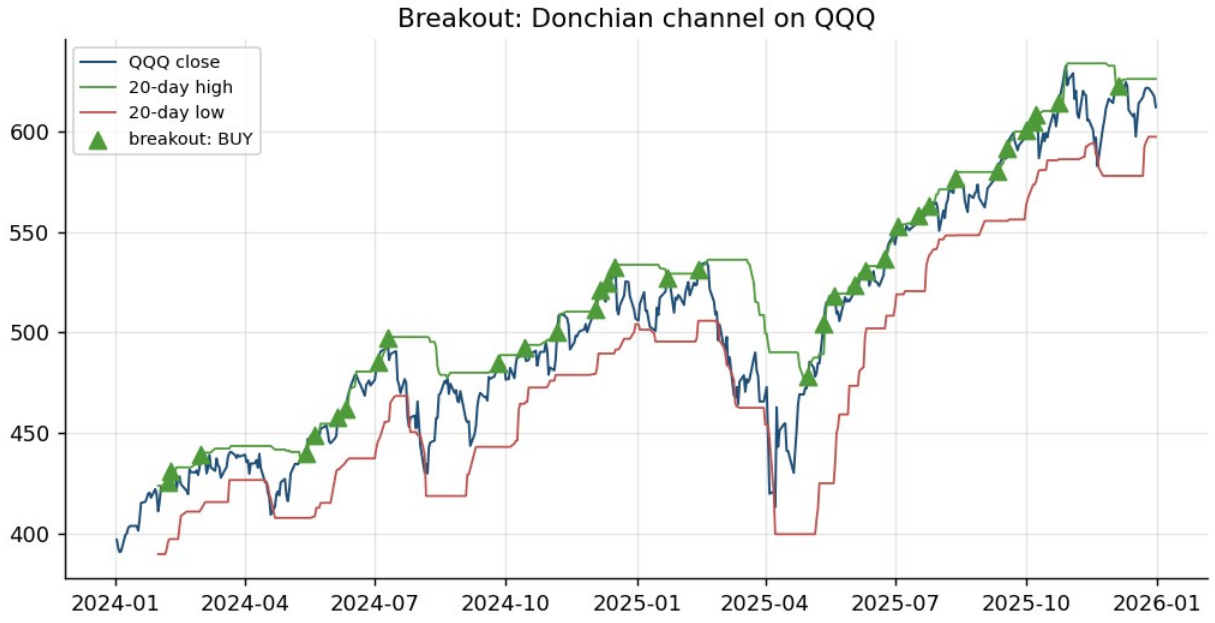


Figure: Algorithm: Donchian Channel Breakout - illustrated on real market data.

How it works	Track the highest high and lowest low of the last n bars (n = 20 here).
Rule	Close above the 20-day high: buy the breakout. Exit below the 20-day low (or a trailing stop).
Works best when	New trends start with a break of a range - the rule used by the 1980s 'Turtle Traders'.
Watch out for	False breakouts in choppy markets; the channel must use PREVIOUS bars only (no look-ahead).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

How It Works: VWAP and TWAP Execution

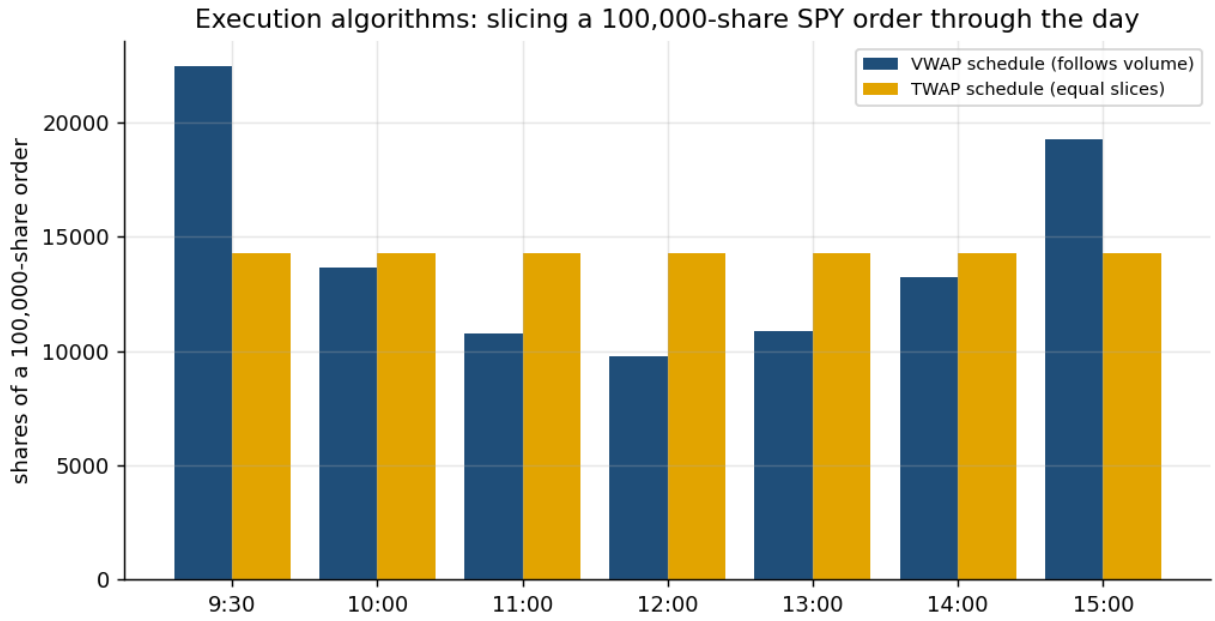


Figure: Algorithm: VWAP and TWAP Execution - illustrated on real market data.

How it works	Split a large parent order into child orders through the day to reduce market impact.
Rule	TWAP: equal slices per time interval. VWAP: slices proportional to the typical volume curve.
Works best when	Working client or fund orders too large to trade at once; benchmarked to the day's VWAP.
Watch out for	VWAP relies on a volume forecast; both are predictable to other traders - add randomisation.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

How It Works: Market Making

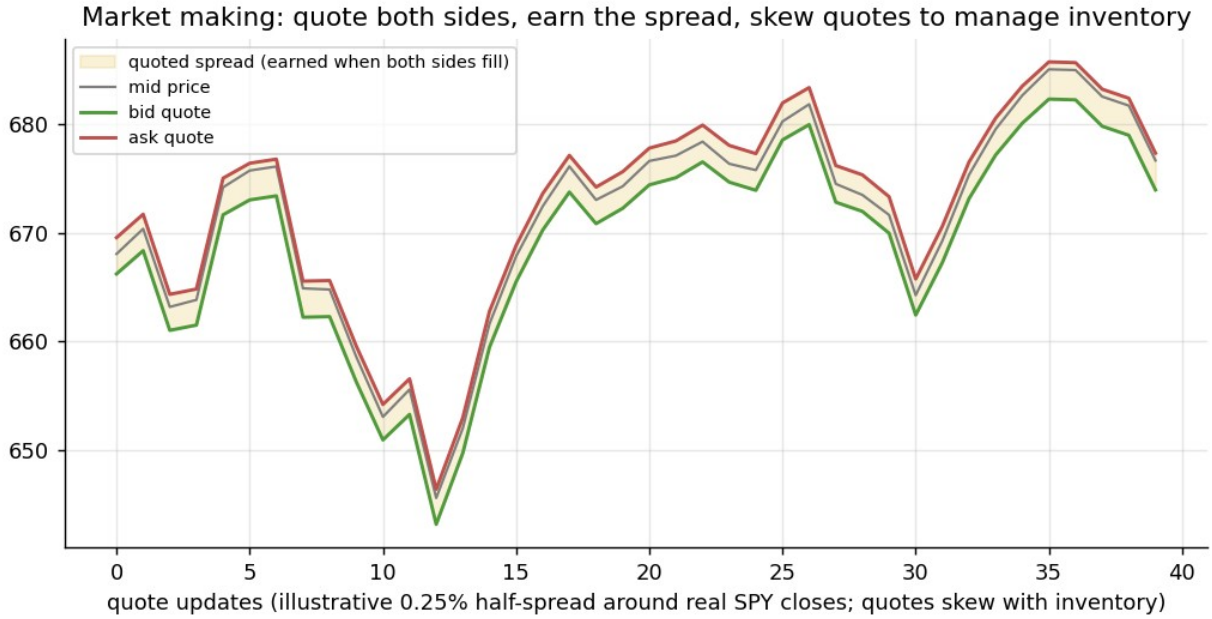


Figure: Algorithm: Market Making - illustrated on real market data.

How it works	Quote a bid below and an ask above the mid price at the same time.
Rule	Earn the spread when both sides fill; skew quotes down when long, up when short, to cut inventory.
Works best when	Liquid markets with steady two-way flow - the business of quant market makers.
Watch out for	Adverse selection by better-informed traders; inventory risk in fast moves; needs low latency.

2.6 Lab 5: Indicator Stage: Pine Script v6 Indicators and their Python Twins

Maps to: A3, K3, K7 | Duration: 25 min | Slide 123

Scenario

Lion City Capital's traders read trend (EMA 20/50), volatility (Bollinger Bands) and momentum (RSI) on TradingView, while the pipeline's signals are computed in Python. If the two disagree, a trader and the automated strategy will act on different numbers, so the desk builds Pine Script indicators and exact Python twins and cross-checks them value by value.

Objective

Add the desk's Pine Script v6 indicators to a TradingView chart, build pipeline/indicators.py with twins of ta.ema, ta.sma, ta.rsi, ta.bb, ta.atr, ta.macd, ta.crossover and ta.crossunder, and cross-check the values against the TradingView Data Window.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Files

labs_fullcode/Lab05-pine-indicators/ - Lab05-pine-indicators/indicators_stage.py, Lab05-pine-indicators/lcc_trend_bands.pine, Lab05-pine-indicators/lcc_rsi.pine, Lab05-pine-indicators/tradingview_indicator_values.csv, pipeline/indicators.py, Lab05-pine-indicators/solution/indicators.py

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 05	python3 Lab05-pine-indicators/indicators_stage.py (same as: python -m pipeline indicators)
The complete code	pipeline/indicators.py (reference copy: Lab05-pine-indicators/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab05-pine-indicators/Lab05_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

Step 1: Add the trend and bands indicator (A3, K3)

On TradingView open NASDAQ:AAPL, 1D. Open the Pine Editor, create a new indicator, replace the template with Lab05-pine-indicators/lcc_trend_bands.pine (it starts with //@version=6), Save as 'LCC Trend' and Add to chart.

AI prompt - paste into your AI coding assistant
Explain labs_fullcode/Lab05-pine-indicators/lcc_trend_bands.pine line by line for a finance professional: the inputs, ta.ema, ta.bb, ta.crossover / ta.crossunder, plotshape, barcolor and alertcondition. Which values will the Data Window show for each bar?

Check: EMA 20 (blue), EMA 50 (orange), shaded Bollinger Bands and green/red crossover triangles on the price pane.

Step 2: Add the RSI pane (A3, K3)

Create another indicator from Lab05-pine-indicators/lcc_rsi.pine, save it as 'LCC RSI' and add it.

Check: RSI(14) in its own pane with 70/30 lines and shaded overbought/oversold zones.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 3: Record the TradingView values (A3)

Move the cursor to 31 December 2025, open the Data Window and copy close, EMA fast, EMA slow, BB upper, middle, lower and RSI into Lab05-pine-indicators/tradingview_indicator_values.csv (overwrite the example row).

Check: Default (unadjusted) chart: close 271.86, EMA 20 about 273.98, EMA 50 about 269.60, BB 282.35 / 275.47 / 268.60, RSI about 45.64.

Step 4: Build the Pine twins (K3)

Ask your assistant for the first half of pipeline/indicators.py.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/indicators.py with Python twins of the Pine Script v6 built-ins, implemented by calling labs_fullcode/common/indicators.py: ta_sma(src, length), ta_ema(src, length) (alpha 2/(n+1), adjust=False), ta_rsi(src, length=14) (Wilder smoothing), ta_bb(src, length=20, mult=2.0) returning (middle, upper, lower) with the population standard deviation, ta_atr(bars, length=14), ta_macd(src, 12, 26, 9) returning (macd, signal, hist), ta_crossover(a, b) = a > b and a[1] <= b[1], ta_crossunder(a, b). Then lcc_trend_bands(bars, fast=20, slow=50, bb_len=20, bb_mult=2.0) returning close, ema_fast, ema_slow, bb_middle, bb_upper, bb_lower, bull_cross, bear_cross exactly like lcc_trend_bands.pine, lcc_rsi(bars, 14, 70, 30) returning rsi, alert_oversold and alert_overbought like lcc_rsi.pine, and indicator_table(bars) joining both. Every value at bar t may use data up to bar t only. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: indicator_table(load_bars('AAPL','2018-01-01','2025-12-31')).iloc[-1] shows ema_fast 273.23 and rsi 45.69.

Step 5: Build the cross-check run (A3, K3)

Ask your assistant for run() and main().

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/indicators.py add run(symbol='AAPL', date='2025-12-31', tv_file=None) for `python -m pipeline indicators --symbol --date --tv`: compute indicator_table on the ADJUSTED research bars (from config data.start) and on the RAW reference bars from pipeline.validate.reference_bars (TradingView's default view); print a snapshot table (close, ema_fast, ema_slow, bb_upper, bb_middle, bb_lower, rsi) with columns Python ADJUSTED, Python RAW (TV default), TradingView (you) from Lab05-pine-indicators/tradingview_indicator_values.csv and 'diff vs RAW' in %; the trend, band and RSI reading; the number of bullish/bearish crosses and RSI alert events since 2024-06-01; PASS if the largest gap is <= 0.5%. Save _outputs/Lab05_<SYMBOL>_indicators.csv and chart Lab05_pine_preview.png (price, EMAs, bands, cross markers, RSI pane). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline indicators --help shows --symbol, --date and --tv.

Step 6: Run the cross-check (A3, K3)

Run: python -m pipeline indicators --symbol AAPL --date 2025-12-31 (same as: python3 Lab05-pine-indicators/indicators_stage.py --symbol AAPL).

Check: ADJUSTED close 271.12, EMA 20/50 273.23 / 268.80, BB 281.58 / 274.72 / 267.87, RSI 45.69; RAW 271.86, 273.98 / 269.60, 282.35 / 275.47 / 268.60, RSI 45.64; largest gap to TradingView 0.006% -> PASS; trend UP; since 2024-06-01: 2 bullish and 2 bearish EMA crosses, 3 RSI-oversold and 15 RSI-overbought alert events.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 7: Change the inputs and count the extra trading (K3, K7)

In TradingView set LCC Trend to EMA 10/30 and watch the triangles multiply. Then measure it in Python.

AI prompt - paste into your AI coding assistant
In a Python shell started in labs, use pipeline.indicators.lcc_trend_bands on pipeline.ingest.load_bars('AAPL', '2018-01-01', '2025-12-31') with fast/slow 20/50 and then 10/30, count bull_cross and bear_cross since 2024-06-01 for each, and estimate the extra trading cost of the faster setting at 3 bps per round trip on USD 100,000. Print a small table.

Check: Since 2024-06-01: EMA 20/50 = 2 bullish / 2 bearish crosses; EMA 10/30 = 7 / 7 - about 3.5 times as many trades, so cost rises with signal frequency.

Step 8: Create the alert the automation stage will use (A3, K7)

Restore EMA 20/50. Click Alert, choose Condition: LCC Trend > 'EMA bullish cross', trigger 'Once per bar close', and save. This alert type is what Lab 17 connects to the pipeline webhook.

Check: The alert appears in the Alerts panel, active, on NASDAQ:AAPL.

Expected Output (Full Code)

python -m pipeline indicators --symbol AAPL --date 2025-12-31: adjusted close 271.12, EMA 20/50 273.23 / 268.80, Bollinger 281.58 / 274.72 / 267.87, RSI(14) 45.69; raw (TradingView default) 271.86, 273.98 / 269.60, 282.35 / 275.47 / 268.60, RSI 45.64; largest gap to TradingView 0.006% -> PASS; 2 bullish and 2 bearish EMA crosses since June 2024; files _outputs/Lab05_AAPL_indicators.csv and chart _outputs/Lab05_pine_preview.png.

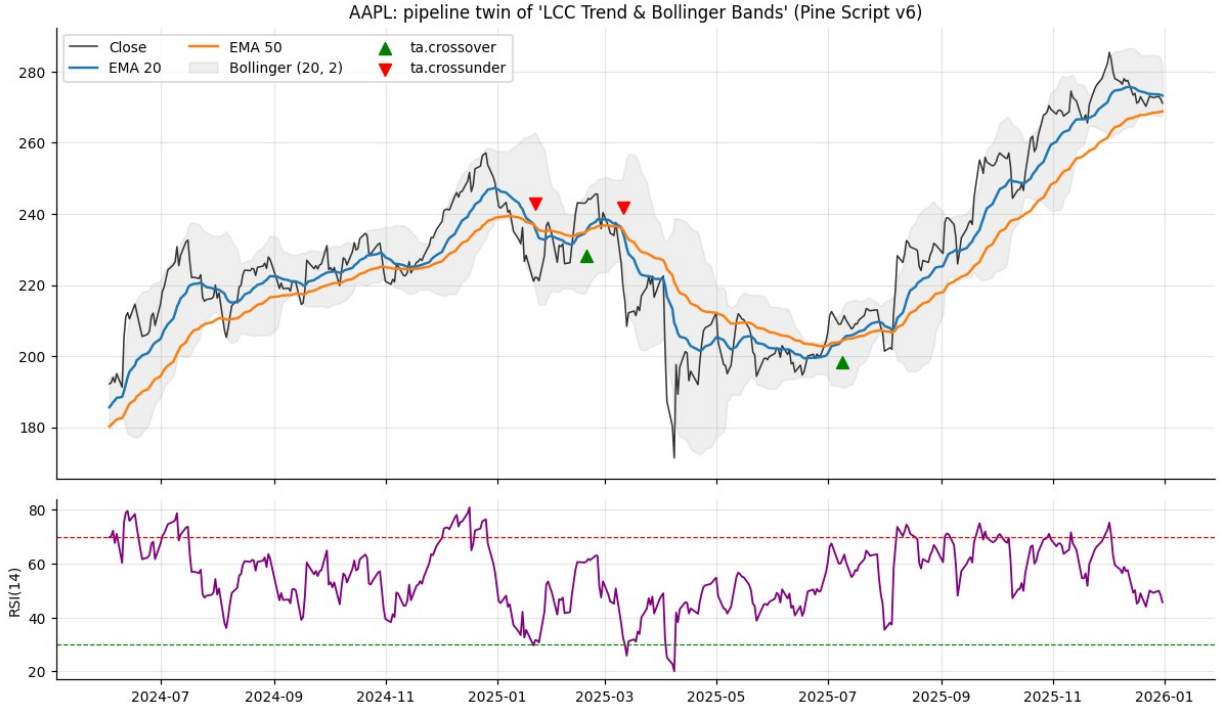


Figure: Lab 5 output (labs_fullcode/_outputs/Lab05_pine_preview.png), offline mode on the bundled data.

Debrief

Technical indicators (K3) turn prices into trend, volatility and momentum readings; defining them once in Pine Script and once in Python, and proving they match (A3), keeps traders and the automated pipeline on the same numbers. Faster settings fire more signals, which means more trades and more cost (K7).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Troubleshooting

Symptom	What to do
Compilation error in the Pine Editor	The first line must be //@version=6 and the whole file must be pasted.
Indicator draws in a separate pane	lcc_trend_bands.pine must keep overlay = true; only lcc_rsi.pine uses overlay = false.
Gap to TradingView above 0.5%	Check the chart is 1D, 'Adjust data for dividends' is unticked, and you read the 2025-12-31 bar; compare with the RAW column.
FileNotFoundError for the RAW column	Raw reference files exist for AAPL and D05.SI only; other symbols show the adjusted column.

2.7 Lab 6: Strategy Prototype: TradingView Strategy Tester, Paper Trading from the Chart and Pipeline Reconciliation

Maps to: A2, A3, K3, K7 | Duration: 30 min | Slide 126

Scenario

A portfolio manager prototypes a golden-cross rule for the desk's SPY sleeve in TradingView. Before it can enter the automated pipeline, the Strategy Tester result (with commission and slippage) must be reproduced by an independent pipeline backtest, and the trader must be able to put the idea into action on a paper account straight from the chart.

Objective

Configure a Pine Script v6 strategy with costs, read the Strategy Tester, build `python -m pipeline reconcile` to check the Pine settings and reproduce the results, and place and manage paper trades from the chart with TradingView Paper Trading (and optionally an Alpaca paper account).

Files

labs_fullcode/Lab06-pine-strategy-tester/ - Lab06-pine-strategy-tester/reconcile_stage.py, Lab06-pine-strategy-tester/lcc_ma_crossover_strategy.pine, Lab06-pine-strategy-tester/tv_strategy_results.yaml, pipeline/reconcile.py, Lab06-pine-strategy-tester/solution/reconcile.py

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 06	python3 Lab06-pine-strategy-tester/reconcile_stage.py (same as: python -m pipeline reconcile)
The complete code	pipeline/reconcile.py (reference copy: Lab06-pine-strategy-tester/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab06-pine-strategy-tester/Lab06_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

Step 1: Read the strategy code (A2, K3)

Open Lab06-pine-strategy-tester/lcc_ma_crossover_strategy.pine. Identify the strategy() settings (initial_capital 100000, 100% of equity, commission_type cash_per_order 1.0, slippage 2, pyramiding 0, process_orders_on_close false) and the rules (ta.crossover / ta.crossunder of SMA 50 and SMA 200).

AI prompt - paste into your AI coding assistant
Explain labs_fullcode/Lab06-pine-strategy-tester/lcc_ma_crossover_strategy.pine for a finance professional: each strategy() setting, when an order is filled (process_orders_on_close = false means the NEXT bar's open), and how commission and slippage reduce the result. Do not change the file.

Check: You can state that a signal at Monday's close is filled at Tuesday's open, with USD 1 commission per order.

Step 2: Run it in the Strategy Tester and record the results (A3, K7)

Open AMEX:SPY on 1D, paste the script into a new Pine Editor tab, save and Add to chart. Open the Strategy Tester panel and copy net profit, closed trades, percent profitable, profit factor, max drawdown, Sharpe, Sortino and commission paid into Lab06-pine-strategy-tester/tv_strategy_results.yaml (replace the nulls).

Check: The chart shows SMA 50 / SMA 200, green shading while in a position, and the Performance Summary tab.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 3: Build the reconciliation stage (A2, A3, K3)

Ask your assistant for pipeline/reconcile.py.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/reconcile.py. parse_pine(path) reads the //@version, the keyword settings inside strategy(...) and the input.int defaults (fastLen, slowLen) from Lab06-pine-strategy-tester/lcc_ma_crossover_strategy.pine. settings_checks(pine, cfg) returns PASS/FAIL rows: version 6; process_orders_on_close = false (next-bar-open fills = config strategy.execution next_open); commission_type cash_per_order with the value in config costs.tradingview; slippage > 0; default_qty_value 100 (% of equity); pyramiding 0. run(symbol='SPY', fast=None, slow=None) backtests the same rule with common.backtest.backtest(bars, cross_signal(ta_sma(fast), ta_sma(slow)), costs=pipeline.config.tradingview_cost_model(cfg), execution='next_open') and buy_and_hold, prints the Strategy Tester equivalents (net profit USD and %, closed and open trades, percent profitable, profit factor, max drawdown, Sharpe, Sortino, commission, buy and hold) next to the values in Lab06-pine-strategy-tester/tv_strategy_results.yaml with a difference column, the trade list, and four likely causes of gaps (adjusted vs unadjusted prices, 2018 history start, whole vs fractional shares, ticks vs bps slippage). Write _outputs/Lab06_reconciliation.yaml (plain Python numbers) and chart Lab06_crosscheck.png with common.plotting.plot_equity_and_drawdown. Exit code 1 if a settings check fails. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline reconcile --help shows --symbol, --fast and --slow.

Step 4: Reconcile TradingView with the pipeline (A2, A3)

Run: python -m pipeline reconcile --symbol SPY --fast 50 --slow 200 (same as: python3 Lab06-pine-strategy-tester/reconcile_stage.py).

Check: 6 settings checks PASS. Pipeline: net profit USD 79,878 (+79.88%), 4 closed trades + 1 open, 50.0% profitable, profit factor 5.03, max drawdown -33.72%, Sharpe 0.55, Sortino 0.76, commission USD 9.00, buy and hold +187.26%; first entry 2018-10-17; your TradingView figures appear in the TradingView column with differences.

Step 5: Explain the differences (A3)

Compare the two trade lists. Use the causes printed by the stage to explain each gap in one sentence.

AI prompt - paste into your AI coding assistant
Here is my reconciliation output: <paste the table from python -m pipeline reconcile>. For each metric where TradingView and the pipeline differ by more than 5%, say which of these causes is most likely and how I could confirm it: adjusted vs unadjusted prices, the 2018 data start and SMA 200 warm-up, whole vs fractional shares, slippage in ticks vs bps.

Check: Each gap is linked to a named cause; typically TradingView's longer history changes the first trade.

Step 6: Stress the costs and the speed of the rule (K7)

In TradingView change commission to 0.1% of order value, slippage to 5 ticks and the inputs to 20/50. Then run: python -m pipeline reconcile --fast 20 --slow 50.

Check: Pipeline SMA 20/50: net profit USD 88,228 (+88.23%), 16 closed trades, max drawdown -27.84%, Sharpe 0.72, commission USD 33 - more trades pay more cost; record how TradingView's net profit falls with 0.1% commission.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 7: Put the strategy into action with TradingView Paper Trading (A3)

Open the Trading Panel at the bottom of the chart and choose 'Paper Trading by TradingView', then Connect. In the account settings set the starting balance to 100,000 USD and a commission (for example USD 1 per order). From the chart (Buy button, or right-click > Trade) place: (1) a market buy of 10 SPY, (2) a limit buy 2% below the last price, (3) a buy with take-profit and stop-loss (bracket) in the order ticket.

AI prompt - paste into your AI coding assistant
Act as my execution checklist for TradingView Paper Trading on AMEX:SPY with USD 100,000. Using the last pipeline close of 676.64, give the order-ticket fields and prices for (1) a market buy of 10 shares, (2) a limit buy 2% below the last price, (3) a bracket buy with take-profit +5% and stop-loss -3%; then say which status each order should show in the Orders and Positions tabs.

Check: Market buy filled in Positions; limit buy (about 663.11) Working in Orders; bracket with take-profit about 710.47 and stop-loss about 656.34 shown as lines on the chart; commission deducted in the account summary.

Step 8: Manage the paper position from the chart (A3, K7)

Drag the stop-loss line to move it, close half the position, then close all. Read the History tab and the account balance. Note: a Pine strategy does not send orders to the Trading Panel by itself - automated orders go through alerts and webhooks (Lab 17) or the pipeline's execution stage (Lab 8).

Check: History lists every fill with its commission; the position is flat and the balance reflects P&L and costs.

Step 9: Optional - connect the Alpaca paper account in TradingView (A3)

In the Trading Panel choose Alpaca, log in, select the PAPER account and place one small market or limit order from the chart. Then run python -m pipeline check (online) to see the same position from the pipeline.

Check: The order appears in the Alpaca paper dashboard and as an open position in section [4] of the check.

Step 10: Decide whether to promote the rule (A2)

Apply the strategy to SGX:D05 and compare. Decide whether SMA 50/200 should replace the pipeline's configured ema_cross 10/30 (do not change config.yaml yet - Labs 9 and 10 measure it properly).

Check: A one-paragraph decision that cites net profit vs buy and hold (+79.88% vs +187.26%) and the drawdown.

Expected Output (Full Code)

python -m pipeline reconcile --symbol SPY --fast 50 --slow 200: 6 settings checks PASS; net profit USD 79,878 (+79.88%), 4 closed trades + 1 open, 50.0% profitable, profit factor 5.03, max drawdown -33.72%, Sharpe 0.55, Sortino 0.76, commission USD 9.00 versus buy and hold +187.26%; SMA 20/50: +88.23%, 16 trades, commission USD 33; TradingView Paper Trading shows a filled market order, a working limit order and a bracket; files _outputs/Lab06_reconciliation.yaml and chart _outputs/Lab06_crosscheck.png.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

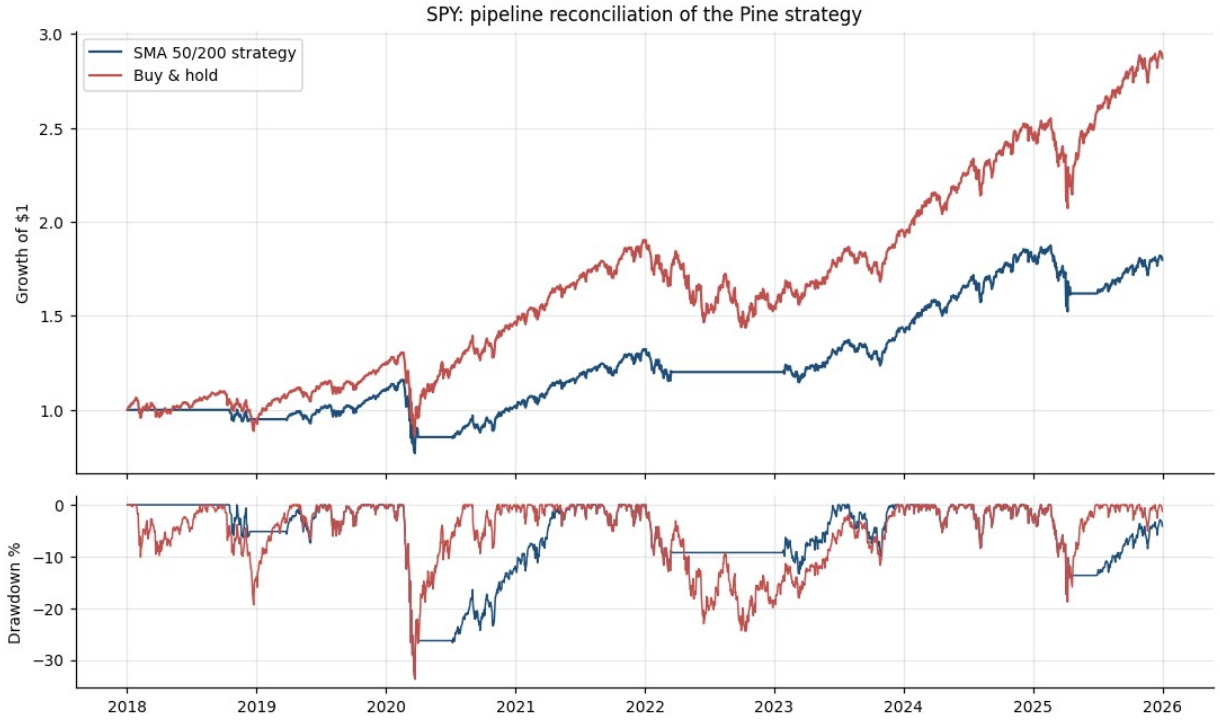


Figure: Lab 6 output (labs_fullcode/_outputs/Lab06_crosscheck.png), offline mode on the bundled data.

Debrief

Choosing a rule and its parameters is a modelling decision (A2); the Strategy Tester and TradingView Paper Trading are analytics and execution platforms (A3) that report standard metrics (K3). An independent reconciliation is the gate between a chart prototype and the automated pipeline, and costs typed into strategy() (K7) barely touch a slow rule but compound for a fast one.

Troubleshooting

Symptom	What to do
Strategy Tester shows 'No data'	Check the backtest dates in the inputs and that the chart is 1D.
Different first trade	Expected: the bundled data starts in 2018, so the pipeline's SMA 200 exists only from October 2018.
A settings check FAILs	Someone changed the Pine file or config.yaml costs.tradingview; make them agree and rerun.
Paper Trading not listed	Open the Trading Panel (bottom toolbar) and scroll the broker list; it is called 'Paper Trading by TradingView'.

2.8 Lab 7: Signal Stage: a Library of Trend, Mean-Reversion, Momentum and Breakout Signals

Maps to: A2, K3, K7 | Duration: 25 min | Slide 130

Scenario

Lion City Capital wants every strategy - from a TradingView prototype to an ML model - to produce the same thing: a target position per symbol, decided at the close and traded on the next bar. The quant

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

team builds one signal library that the evaluation, search, execution, automation and diagnostic stages all call.

Objective

Build pipeline/signals.py with trend, mean-reversion, momentum and breakout signals that return target positions with next-bar alignment, a registry, an automated look-ahead test and a universe target file for the execution stage.

Files

labs_fullcode/Lab07-technical-indicators/ - Lab07-technical-indicators/signals_stage.py, pipeline/signals.py, Lab07-technical-indicators/solution/signals.py

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 07	python3 Lab07-technical-indicators/signals_stage.py (same as: python -m pipeline signals)
The complete code	pipeline/signals.py (reference copy: Lab07-technical-indicators/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab07-technical-indicators/Lab07_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

Step 1: Agree the signal contract (A2)

Read the docstring of the reference Lab07-technical-indicators/solution/signals.py. Every signal returns a float Series of target positions (1 = long, 0 = flat) decided with information up to the CLOSE of bar t; the position is HELD from bar t+1. Warm-up bars (NaN indicators) are flat.

Check: You can explain why a signal computed at Friday's close can only earn Monday's return.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 2: Build the trend signals (A2, K3)

Ask your assistant for the trend part of pipeline/signals.py.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/signals.py. Helper _flat_until_ready(cond, *inputs) returns 1.0 where cond is true and 0.0 otherwise or while any input is NaN. Trend signals using pipeline.indicators: ema_cross(bars, fast=10, slow=30) long while ta_ema(fast) > ta_ema(slow); sma_cross(bars, fast=50, slow=200); trend_filter(bars, window=200) long while close > ta_sma(window). Each returns a float Series aligned to bars.index, using data up to bar t only. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: ema_cross(load_bars('SPY','2018-01-01','2025-12-31')).iloc[-1] is 1.0.

Step 3: Build mean-reversion, momentum and breakout signals (A2, K3)

Ask your assistant for the remaining families.

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/signals.py add enter_exit(entry, exit_) - a stateful long/flat series that is 1 from an entry bar until the next exit bar (entry wins a tie, NaN treated as False); rsi_reversion(bars, n=2, entry=10, exit=70) = enter_exit(RSI < entry, RSI > exit); dip_buy(bars, n_down=3, hold=3) = long for `hold` bars after a bar completing >= n_down consecutive down closes; momentum(bars, lookback=126, threshold=0.0) long while the past lookback-bar return > threshold; donchian_breakout(bars, entry=55, exit=20) buy a close above the PRIOR entry-bar high (rolling max shifted 1), sell a close below the prior exit-bar low; bollinger_breakout(bars, window=20, k=2.0) buy a close above the upper band, sell below the middle band. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: Each function returns only 0.0 and 1.0 values for SPY.

Step 4: Add the registry, next-bar alignment and the look-ahead test (A2, K3)

Ask your assistant for the shared helpers.

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/signals.py add SIGNALS = {name: (function, family, default_params)} for the eight signals; target_position(name, bars, **params) (params override defaults, Series renamed to name); next_bar_position(target) = target.shift(1).fillna(0) (the position actually held); lookahead_test(name, bars, checks=25, seed=0, fn=None, **params): pick 25 random dates t in the last two-thirds of the sample and recompute the signal (a) on bars.loc[:t] only and (b) after multiplying every price AFTER t by 1.5 and by 0.5; count dates where any value up to t differs from the full-sample signal (any difference = look-ahead); describe_signal(target) with latest_target, entries, exposure, avg_hold_bars and last_change; latest_targets(symbols, name, params, start, end) returning a DataFrame of symbol, as_of, close, target, last_change. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: lookahead_test('ema_cross', bars) returns 0.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 5: Build the signals command (A2)

Ask your assistant for run() and main().

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/signals.py add run(symbol='SPY') for `python -m pipeline signals --symbol`: for every signal in SIGNALS print family, params, latest target, entries, exposure, average holding bars, last change date and the look-ahead result (PASS or FAIL (n)); print one example of next-bar alignment for ema_cross (the close where the target changed and the first bar it is held); compute latest_targets for config universe.trade with config strategy name/params, print it and save _outputs/Lab07_targets.csv (input to the execution stage); save chart Lab07_signals.png (SPY 2024-2025 with green shading where ema_cross, rsi_reversion, momentum and donchian_breakout are long). Exit code 1 if any look-ahead test fails. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline signals --help shows --symbol.

Step 6: Run the signal stage (A2, K3)

Run: python -m pipeline signals --symbol SPY (same as: python3 Lab07-technical-indicators/signals_stage.py).

Check: ema_cross 26 entries, 71% exposure, 54.7 bars average hold, last change 2025-11-28; sma_cross 5 entries; trend_filter 24; rsi_reversion 88 entries, 20% exposure, 4.6 bars; dip_buy 102 entries, 19%, 3.7 bars; momentum 33; donchian_breakout 20; bollinger_breakout 37 (latest target 0); all look-ahead PASS. Alignment: target changes to 1 at the close of 2025-11-28 and is held from 2025-12-01. Universe: 7 of 9 symbols long (MSFT and TLT flat); _outputs/Lab07_targets.csv and Lab07_signals.png.

Step 7: Catch a look-ahead bug on purpose (K3)

Temporarily change ema_cross to use bars['close'].shift(-1) (tomorrow's close) and rerun the stage. Then undo the change and rerun.

Check: With the bug ema_cross shows 'FAIL (25)' (the future-shock test changes its history every time) and the command exits with code 1; after undoing it all signals PASS again.

Step 8: Relate signal frequency to trading cost (K7)

Compare entries per signal: dip_buy (102) and rsi_reversion (88) against sma_cross (5).

AI prompt - paste into your AI coding assistant
Using the signal table from python -m pipeline signals, estimate for each signal the round trips per year over 2018-2025 and the annual cost drag at 3 bps per round trip on USD 100,000, assuming the full equity is traded. Rank the signals from most to least cost-sensitive and say which ones need a strong edge to survive costs.

Check: dip_buy and rsi_reversion trade about 11-13 round trips a year versus under one for sma_cross, so they are the most cost-sensitive.

Expected Output (Full Code)

python -m pipeline signals --symbol SPY: eight signals in four families, all pass the look-ahead truncation test; ema_cross 26 entries and 71% exposure (latest flip at the close of 2025-11-28, held from 2025-12-01); dip_buy 102 entries; universe targets 7 of 9 long (MSFT, TLT flat) saved to _outputs/Lab07_targets.csv; chart _outputs/Lab07_signals.png.

Record your results in Lab07_Results.xlsx in this lab folder: the settings are filled in - type the metrics into the yellow cells. Its Definitions sheet explains every column.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

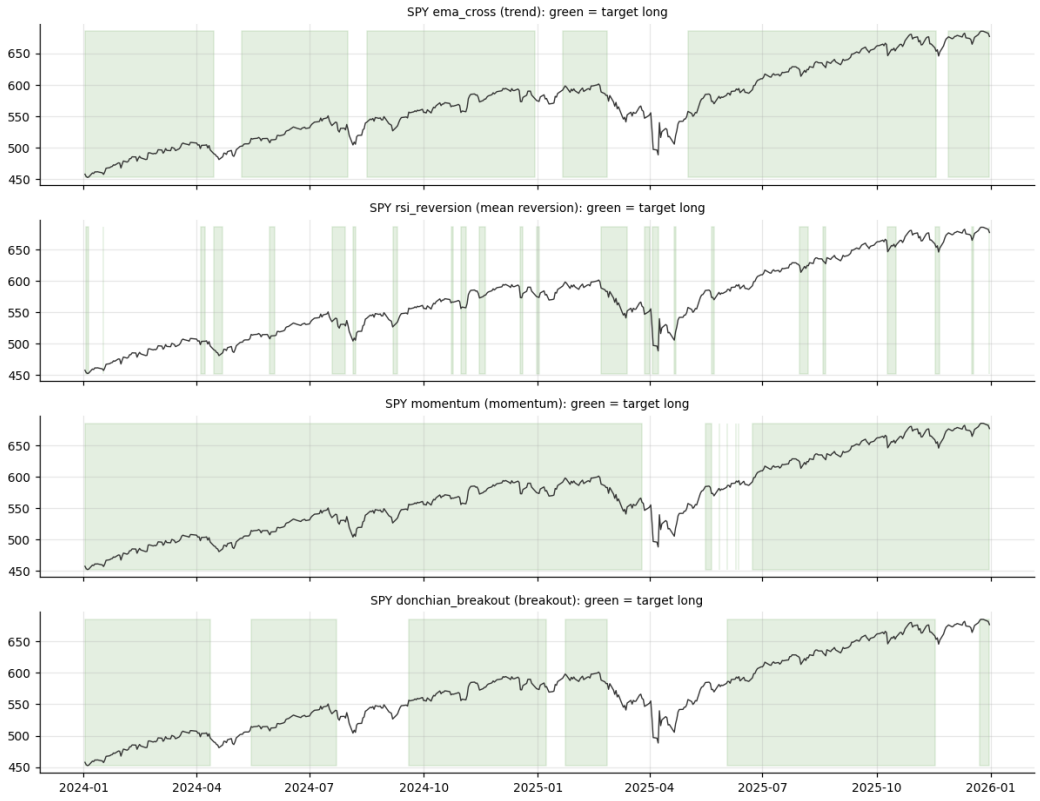


Figure: Lab 7 output (labs_fullcode/_outputs/Lab07_signals.png), offline mode on the bundled data.

Debrief

A single signal contract - target positions decided at the close, held from the next bar - is what lets one library feed backtests, live execution and diagnostics (A2). Indicators (K3) become signals through simple, testable rules, and the number of entries each rule produces is the first estimate of its trading cost (K7).

Troubleshooting

Symptom	What to do
Signals contain NaN	Wrap conditions with _flat_until_ready so warm-up bars are 0, not NaN.
A look-ahead test fails	Look for shift(-1), center=True or any use of data after bar t in that signal.
Targets file is empty	Run python -m pipeline ingest first or check universe.trade in config.yaml.
TypeError on a column called hist	Use df['hist'], not df.hist - hist is also a pandas method.

2.9 Lab 8: Execution Stage: an Order Manager that Trades the Paper Account in Any Session

Maps to: A3, K5 | Duration: 30 min | Slide 133

Scenario

The pipeline now produces target positions. The desk needs an order manager that turns 'target' versus 'actual' into paper orders safely: the right order type for the current US session (the Singapore class runs

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

while New York is in its overnight and pre-market sessions), prices off the live quote, no duplicate orders if a job runs twice, and a printed order lifecycle the trader can match in the Alpaca dashboard.

Objective

Build pipeline/execution.py: a session-aware, idempotent order manager that sizes orders from target weights, sends market, extended-hours limit, GTC-queued or bracket PAPER orders, polls their lifecycle, cancels protective legs before exits, and falls back safely when an order is rejected.

Files

labs_fullcode/Lab08-paper-orders/ - Lab08-paper-orders/execution_stage.py, pipeline/execution.py, pipeline/clock.py, common/broker.py, Lab08-paper-orders/solution/execution.py

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 08	python3 Lab08-paper-orders/execution_stage.py (same as: python -m pipeline execute)
The complete code	pipeline/execution.py (reference copy: Lab08-paper-orders/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab08-paper-orders/Lab08_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

Step 1: Learn the order rules for each session (K5)

Open pipeline/clock.py and read RULES. Alpaca 24/5 trading: overnight 20:00 Sunday to 04:00 Friday (New York), pre-market 04:00-09:30, regular 09:30-16:00, after-hours 16:00-20:00. Outside the regular session only LIMIT orders with time_in_force DAY or GTC and extended_hours=True are accepted.

AI prompt - paste into your AI coding assistant
Using labs_fullcode/pipeline/clock.py, tell me which US session it is now in New York, what that is in Singapore time, and which order types, time_in_force values and extended_hours flag Alpaca will accept right now. Then explain why a market order is not allowed overnight. Do not change any file.

Check: You can name the session for a 09:00-18:30 SGT class (OVERNIGHT until 16:00 SGT in US summer time, then PRE) and state 'limit only, extended_hours=True'.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 2: Build the planning logic: target vs actual (A3)

Ask your assistant for the OrderPlan dataclass and the planning methods.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/execution.py with a dataclass OrderPlan(symbol, side, qty, current_qty, pending_qty, target_qty, price, reason) and a class OrderManager(broker, cfg, session=None). status() returns pipeline.clock.market_status (the simulator's replay clock if set; the real Alpaca clock online; session=... overrides it offline) and tells the simulator whether the market is open. positions() -> {symbol: qty}; pending() -> signed shares on open orders (+buy, -sell) via GetOrdersRequest(status=OPEN); quote(symbol, session) -> bid/ask/last: online StockLatestQuoteRequest (feed OVERNIGHT first in the overnight session, then IEX), falling back to the latest trade; offline the simulator's last price +/- 0.5 bp. plan(targets, session, units='weight'|'shares'): target_qty = floor(weight x equity / price) (or the share count), delta = target - held - pending, buy if delta > 0, sell (long only, never more than held) if delta < 0, otherwise print 'at target' and send nothing. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: OrderManager(get_broker(state_file='exec_sim.json', reset=True), load_config()).plan({'SPY': 0.30}, 'regular') returns one buy plan for 44 shares.

Step 3: Build session-aware, idempotent order submission (A3, K5)

Ask your assistant for client ids, the order request builder and submit().

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/execution.py add client_id(plan, tag) = 'lcc-<tag>-<SYMBOL>-<side>-<qty>' (alphanumerics and dashes, max 48 characters; tag = decision date) so the same decision always gets the same id. request(plan, order_type, status, cid, quote, take_profit=0.05, stop_loss=0.03) builds an alpaca-py request: regular + market -> MarketOrderRequest DAY; regular + bracket (buys) -> MarketOrderRequest GTC with OrderClass.BRACKET, TakeProfitRequest and StopLossRequest; closed -> LimitOrderRequest GTC (queues for the next session); pre/after/overnight -> LimitOrderRequest DAY with extended_hours=True. Limit price = ask x (1 + live_paper.limit_buffer_bps) for buys, bid x (1 - buffer) for sells, rounded to 2 decimals. submit(plan, status, tag, order_type='auto'): first broker.get_order_by_client_id(cid) - if it exists print 'DUPLICATE ... not sent again' and return it; print the order (type, TIF, extended_hours, limit, quote and source, cid); send it with broker.submit_order. The broker comes from common.broker.get_broker (Alpaca TradingClient with paper=True, or PaperBrokerSim) - never create a live client. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: Submitting the same plan twice with the same tag prints DUPLICATE the second time.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 4: Add polling, protective-order handling and safe fallbacks (A3, K5)

Ask your assistant for poll(), cancel_protective(), the rejection fallback, the crypto demo and the command.

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/execution.py add: poll(order, seconds) that re-reads the order every 2 seconds and prints the lifecycle (for example 'new -> filled') and the final order line, or '(still working - it stays on the book)'; cancel_protective(symbol) that cancels open SELL orders and bracket exit legs (GetOrdersRequest nested=True) before an exit, because they reserve the shares; in submit(), if an extended-hours order is rejected, print why (asset or account not enabled for overnight trading, price band, session change) and fall back to a GTC limit order with client id + '-q' that queues for the next session; rebalance(targets, tag, order_type, poll_seconds, units) = status -> plan -> submit -> poll; crypto_demo(broker, cfg) - ONLINE ONLY, clearly labelled DEMO: buy about USD 25 of BTC/USD with a GTC limit at ask + buffer, poll, then sell it back - only to show the fill loop working while US stocks are closed. run(...) and main(argv) for `python -m pipeline execute [--targets SPY=0.30 AAPL=0.10] [--order-type auto|market|limit|bracket] [--session ...] [--live-paper] [--crypto-demo] [--flatten] [--tag] [--reset]`: print [1] the clock and allowed orders, [2] the account, [3] plan -> orders, [4] an idempotency re-run of the same decision plus a deliberate duplicate submission, [5] positions and open orders (with bracket legs), the Alpaca dashboard address when online, and a RESULT line. With --live-paper or online, cap each weight at live_paper.max_position_pct. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline execute --help lists the eight options.

Step 5: Run the order manager in the current session (A3, K5)

Run: python -m pipeline execute --reset (same as: python3 Lab08-paper-orders/execution_stage.py --reset). Offline the session comes from the real clock, so the result depends on when you run it.

Check: Weekend (CLOSED): BUY 44 SPY at LIMIT GTC 678.36 and BUY 36 AAPL at 271.81, both 'accepted' (queued); the re-run prints 'at target (44 shares; held 0, pending +44)' and 'DUPLICATE lcc-<date>-SPY-buy-44'. Weekday Singapore class (OVERNIGHT or PRE): LIMIT DAY extended_hours orders, filled at about 676.63 and 271.12.

Step 6: Compare the sessions with the teaching override (K5)

Run: python -m pipeline execute --reset --session regular, then --session overnight, then --session closed.

Check: regular: MARKET DAY, filled at 676.70 (SPY) and 271.15 (AAPL), equity USD 99,996.05; overnight: LIMIT DAY extended_hours limit 678.36 / 271.81, filled at 676.63 / 271.12, equity USD 100,000.00; closed: LIMIT GTC, status accepted, 2 open orders. Each run ends with 'no new orders' and one DUPLICATE line.

Step 7: Bracket orders and a clean exit (A3, K5)

Run: python -m pipeline execute --reset --session regular --order-type bracket. Then flatten: python -m pipeline execute --flatten --session regular.

Check: Bracket legs: SPY take-profit 710.47 / stop 656.34, AAPL 284.68 / 262.99 (status new). Flatten prints 'cancelled 2 protective exit order(s)' for each symbol, sells 44 SPY at 676.57 and 36 AAPL at 271.09, equity USD 99,992.09, 0 open orders.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 8: Trade LIVE on the Alpaca paper account during class (A3, K5)

With paper keys in .env run: python -m pipeline execute --live-paper --targets SPY=0.05. Watch the clock line, the quote, the extended-hours limit order and its lifecycle for up to 30 seconds. Then open the Alpaca paper dashboard (app.alpaca.markets > Paper > Orders and Positions) and find the same client order id.

Check: Clock shows OVERNIGHT or PRE; one LIMIT DAY extended_hours buy priced at ask + 25 bps; lifecycle 'new -> filled' (or still working); the order and position appear in the dashboard. If Alpaca rejects it, the stage explains why and queues a GTC limit order instead.

Step 9: Optional - watch a fill right now with the crypto demo (K5)

If US stocks are closed (weekend) or your extended-hours order is still working, run: python -m pipeline execute --live-paper --targets SPY=0.05 --crypto-demo. This is a DEMO of the fill loop only; the course and the pipeline remain stock-focused.

Check: A BTC/USD buy of about USD 25 and the matching sell, each GTC limit, filled within seconds and visible in the dashboard.

Step 10: Flatten the live paper account (A3)

Run: python -m pipeline execute --live-paper --flatten, then refresh the dashboard. Offline, run the same command without --live-paper.

Check: Protective orders are cancelled, sell orders are sent (extended-hours limits outside regular hours) and the positions list empties once they fill.

Expected Output (Full Code)

python -m pipeline execute --reset --session regular (offline): BUY 44 SPY and BUY 36 AAPL as MARKET DAY orders, filled at 676.70 and 271.15, equity USD 99,996.05; the idempotency re-run sends no new order and a duplicate submission is skipped; --session overnight sends LIMIT DAY extended_hours orders (filled 676.63 / 271.12); --session closed queues LIMIT GTC orders (accepted); bracket legs SPY 710.47 / 656.34; --flatten cancels the protective legs and exits (equity USD 99,992.09). With --live-paper the same flow runs on the Alpaca paper account in any session.

Debrief

Order types are tools for price, timing and risk, and the trading session decides which ones the broker accepts (K5). An order manager that reads the clock, prices limits off the quote, makes every order idempotent and prints its lifecycle is how an automated program knows exactly what happened at the broker (A3) - in New York's regular session or in its overnight session during a Singapore workday.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Troubleshooting

Symptom	What to do
Extended-hours order rejected online	The asset or account may not be enabled for overnight trading or the limit is outside the price band; the stage falls back to a GTC limit that queues for the next session.
Order stays 'new' or 'accepted'	Normal outside regular hours or when the market is closed; it remains working - check the Orders page in the dashboard.
insufficient qty available	A stop or bracket leg reserves the shares; cancel_protective() must run before any sell.
insufficient buying power	Run with --reset offline, or lower the target weights.

2.10 Lab 9: Evaluation Stage: Performance and Risk Metrics with a Benchmark Table

Maps to: A3, K3 | Duration: 25 min | Slide 136

Scenario

The investment committee wants to know which of the pipeline's signals deserves capital. Return alone cannot answer that, so the evaluation stage runs every signal through the same backtester, over the same window, with the same costs, and compares it with simply holding SPY.

Objective

Build `python -m pipeline evaluate` to compute total return, CAGR, volatility, Sharpe, Sortino, maximum drawdown, Calmar, win rate, profit factor, exposure, turnover, alpha and beta for every signal against a buy-and-hold benchmark, and interpret the ranking.

Files

labs_fullcode/Lab09-performance-metrics/ - Lab09-performance-metrics/evaluate_stage.py, pipeline/evaluate.py, common/metrics.py, Lab09-performance-metrics/solution/evaluate.py

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 09	python3 Lab09-performance-metrics/evaluate_stage.py (same as: python -m pipeline evaluate)
The complete code	pipeline/evaluate.py (reference copy: Lab09-performance-metrics/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab09-performance-metrics/Lab09_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

Step 1: Read the metric formulas (K3)

Open common/metrics.py; each function's docstring gives its formula (annualised with 252 trading days).

AI prompt - paste into your AI coding assistant
Explain each function in labs_fullcode/common/metrics.py with its formula and one sentence on what a portfolio manager learns from it; say which metrics need trade-level returns and which need a benchmark. Do not change the file.

Check: You can explain the difference between Sharpe (all volatility) and Sortino (downside only).

Step 2: Build the evaluation stage (A3, K3)

Ask your assistant for pipeline/evaluate.py.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/evaluate.py. evaluate_strategy(bars, name, costs, start, bench_ret, **params) backtests pipeline.signals.target_position(name, bars, **params) with common.backtest.backtest (next-bar open fills, costs = pipeline.config.cost_model(cfg)), slices returns and position from `start` and returns common.metrics.summary(...) with trade net returns and the benchmark. run(symbol='SPY'): start = first date the 200-day SMA exists (so every signal is warm), benchmark = buy_and_hold sliced from start; evaluate every signal in SIGNALS; print a table (rows = strategies + buy & hold, columns CAGR, Ann. volatility, Sharpe, Sortino, Max drawdown, Calmar, Trades, Win rate, Profit factor, Exposure, Turnover, Alpha, Beta) ranked by Sharpe and formatted with common.metrics.format_table; list which strategies beat buy and hold on Sharpe and how many have a smaller drawdown; print 'Sharpe by hand' for trend_filter (mean / stdev x sqrt(252)), the worst day and SPY's drawdown trough; save _outputs/Lab09_metrics.csv and chart Lab09_metrics.png (top 4 by Sharpe plus buy & hold, equity and drawdown). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline evaluate --help shows --symbol.

Step 3: Run the evaluation (A3)

Run: python -m pipeline evaluate --symbol SPY (same as: python3 Lab09-performance-metrics/evaluate_stage.py).

Check: Window 2018-10-16 to 2025-12-31, Alpaca-style costs. ema_cross: CAGR 11.09%, vol 11.87%, Sharpe 0.95, max drawdown -14.19%, Calmar 0.78, 26 trades, win rate 50.00%, profit factor 3.54, beta 0.35. SPY buy & hold: 15.28%, 19.93%, 0.81, -33.72%, 0.45. trend_filter: 9.70%, 12.42%, 0.81, -19.36%, 24 trades, win 41.67%, profit factor 3.79, exposure 78.53%, beta 0.38. Only ema_cross beats buy & hold on Sharpe; 7 of 8 have a smaller drawdown.

Step 4: Check one metric by hand (K3)

Read the 'Sharpe by hand' line and confirm it equals the table.

Check: mean 0.00040 / stdev 0.00782 x sqrt(252) = 0.81, the same as trend_filter's Sharpe in the table.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 5: Interpret trade and market metrics (A3, K3)

Explain why trend_filter can win only 41.67% of trades yet have a profit factor of 3.79, and why beta is 0.38 when exposure is 78.53%.

AI prompt - paste into your AI coding assistant
Using labs_fullcode/_outputs/Lab09_metrics.csv, write a 120-word note for Lion City Capital's investment committee comparing ema_cross, trend_filter and SPY buy & hold on return, risk and risk-adjusted return. Quote the numbers, explain win rate versus profit factor, and do not recommend live trading.

Check: Few large winners pay for many small losers; the strategy is out of the market in many stress periods, so its beta is far below its exposure.

Step 6: Locate the worst periods (K3)

Read the worst-day and drawdown-trough line and find those dates on _outputs/Lab09_metrics.png.

Check: Worst trend_filter day 2020-06-11 (-5.76%); worst SPY day 2020-03-16 (-10.94%); SPY drawdown trough 2020-03-23.

Step 7: Add a metric to the stage (K3)

Extend the evaluation table with one more risk metric.

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/evaluate.py add an 'Ulcer index' column = sqrt(mean(drawdown% squared)) computed from common.metrics.drawdown_series of each strategy's returns, without changing common/metrics.py. Show it in the printed table and the CSV, and explain in one sentence what it adds to max drawdown. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: The new column shows a much lower Ulcer index for ema_cross than for SPY buy & hold.

Expected Output (Full Code)

python -m pipeline evaluate --symbol SPY: window 2018-10-16 to 2025-12-31; ema_cross CAGR 11.09%, Sharpe 0.95, max drawdown -14.19% (best Sharpe); SPY buy & hold 15.28%, 0.81, -33.72%; trend_filter 9.70%, 0.81, -19.36%, 24 trades, win rate 41.67%, profit factor 3.79, beta 0.38; bollinger_breakout last (Sharpe 0.35); files _outputs/Lab09_metrics.csv and chart _outputs/Lab09_metrics.png.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

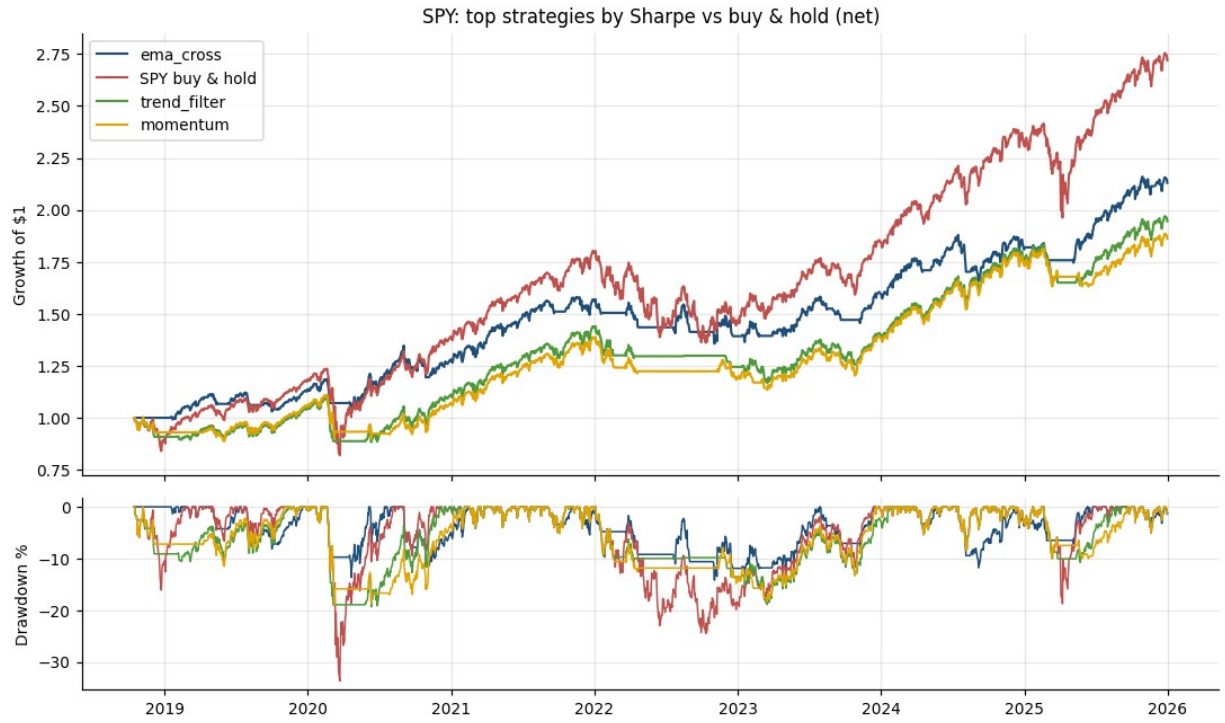


Figure: Lab 9 output (labs_fullcode/_outputs/Lab09_metrics.png), offline mode on the bundled data.

Debrief

Performance metrics (K3) answer different questions: CAGR measures growth, Sharpe and Sortino reward per unit of risk, drawdown and Calmar the pain an investor must survive, beta how much is just the market. Running every strategy through one evaluation stage (A3) is what makes the comparison fair.

Troubleshooting

Symptom	What to do
Sharpe looks too high	Check that daily (not cumulative) returns are used and annualised with sqrt(252).
Win rate below 50% yet profitable	Expected for trend following: few large winners pay for many small losers (see profit factor).
Strategies cover different periods	Slice every return series from the same start date (first valid 200-day SMA).

2.11 Topic Summary

- A2 - Identify appropriate statistical algorithms and data models to test hypotheses and/or theories
- A3 - Use appropriate analytics platforms, analytical tools, statistical methods and analytical approaches given specific analytics and reporting requirements
- K3 - Range and application of various statistical methods and algorithms
- K5 - Usage of analytics platforms and tools
- K7 - Coding languages for programming of algorithms and signals

Assessed by: Practical Performance Task 2 | Written Assessment Q3, Q5, Q7

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Topic 3: Machine Learning Models, Statistical Modelling and A/B Testing for Trading Signals

Day	Abilities	Knowledge	Labs	Slides
Day 2	A4, A5	K2, K4, K6	10 - 14	141 - 174

Learning Outcome

Conduct statistical modelling of market data with machine learning models to derive patterns and solutions, and perform A/B testing of trading strategies based on different hypotheses.

3.1 Types of Statistical Analyses, Data Models, Algorithms and Computational Methods

Maps to: K2, A2 | Slides 142-147

- Descriptive analysis summarises what happened (mean return, volatility, skew, correlation); inferential analysis tests whether an effect is real (t-test, bootstrap); predictive analysis forecasts (regression, classification); prescriptive analysis decides (optimisation).
- Supervised learning learns a mapping from features to a known target - regression for a continuous target (tomorrow's return), classification for a category (up or down).
- Unsupervised learning finds structure without a target - clustering (market regimes) and dimensionality reduction (PCA of a yield curve or a sector basket).
- Reinforcement learning learns a policy from rewards by interacting with an environment - used in research for execution and portfolio allocation, but data-hungry and fragile.
- Advanced computational methods: grid and random search over strategy parameters, Monte Carlo simulation and bootstrapping, walk-forward optimisation, and gradient-boosted ensembles.
- Strategy search is itself an algorithm: testing many parameter combinations on in-sample data and then confirming only the best few out of sample - always counting how many combinations were tried.

Three ways a machine learns | Slides 144 - 147

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Supervised Learning: Learning from Labelled Examples

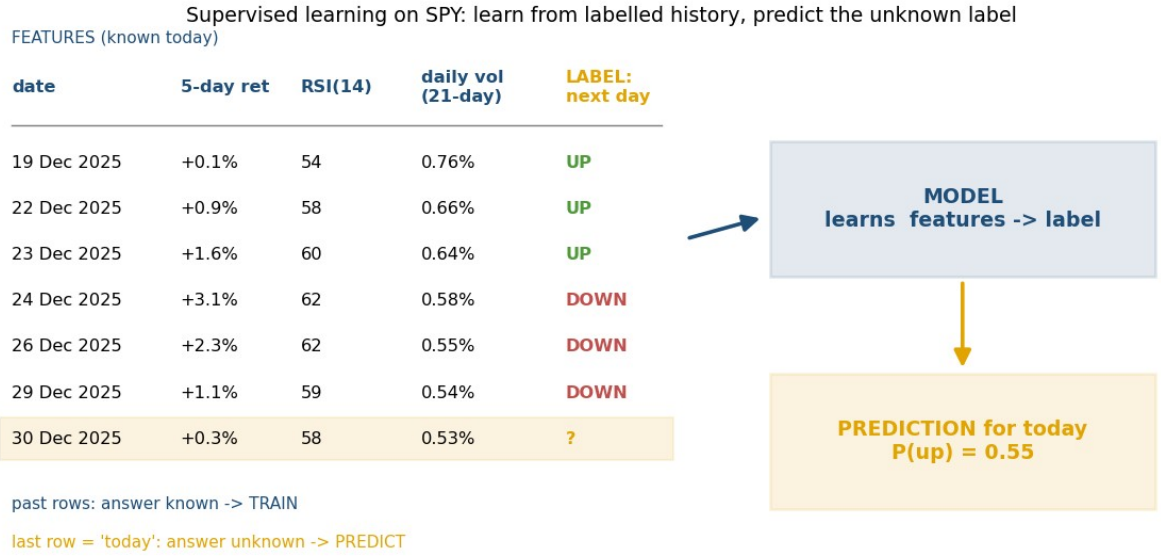


Figure: supervised learning - labelled SPY history, one prediction (Dec 2025).

Aspect	Supervised Learning
Definition	The model learns a mapping from inputs (features) to a known answer (the label), using history in which the answer is already known.
In trading	Features known at today's close - 5-day return, RSI, volatility. Label: did the price rise the NEXT day (classification) or by how much (regression)?
What it gives you	A prediction for a new day - P(up) = 0.55 or an expected return. A rule such as 'long if P(up) > 0.55' turns it into a position.
Watch out for	Features must use only data known at decision time. Accuracy near 50% is normal for daily direction - judge the strategy net of costs.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Unsupervised Learning: Finding Structure Without Labels

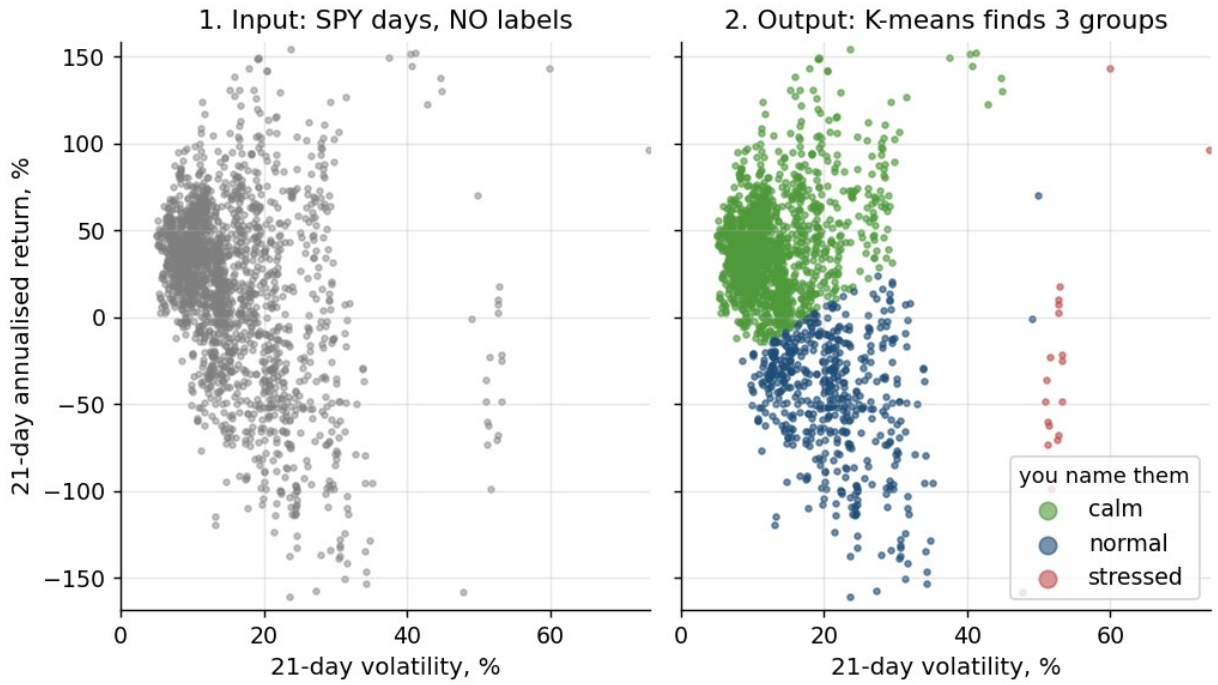


Figure: unsupervised learning - SPY days before and after K-means, 2018-2025.

Aspect	Unsupervised Learning
Definition	There is no answer key. The algorithm groups or compresses the data by similarity, and you decide what the groups mean.
In trading	Cluster days by return and volatility into regimes; PCA reduces many correlated assets to a few factors; cluster stocks for diversification.
What it gives you	A group per day or asset (calm / normal / stressed) or a few factors - inputs to a decision, such as cutting position size when stressed, not trades by themselves.
Watch out for	Clusters have no built-in meaning and k is your choice. Fit on past data only and re-check as the market changes.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Reinforcement Learning: Learning by Action and Reward

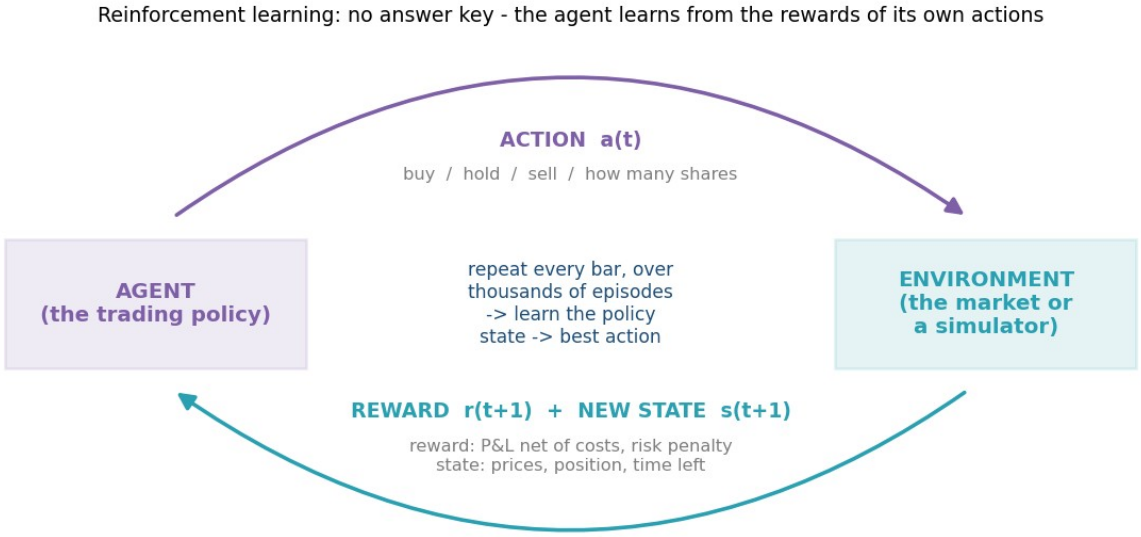


Figure: reinforcement learning - the agent-environment loop.

Aspect	Reinforcement Learning
Definition	An agent observes a state, takes an action and receives a reward; by trial and error it learns a policy that maximises total reward over time.
In trading	State: prices, position, time left. Action: buy, hold, sell or how many shares. Reward: P&L net of costs, minus a penalty for risk.
What it gives you	A policy - the best action for each state. Used for optimal execution of large orders, market making and dynamic hedging, mostly at banks and research labs.
Watch out for	Needs a realistic simulator and huge amounts of experience; it learns to exploit any flaw in the simulator. Unstable and hard to explain - rarely the first tool.

Supervised vs Unsupervised vs Reinforcement Learning

Aspect	Supervised	Unsupervised	Reinforcement
Learns from	Labelled history: features + the known answer	Unlabelled data: features only	Its own experience: states, actions, rewards
Question it answers	What will happen? - predict a target	What patterns exist? - group or compress	What should I do? - choose the next action
Feedback	Error against the correct answer	None - judged by how useful the groups are	A reward, often delayed (profit, cost)
Typical algorithms	Logistic regression, random forest, XGBoost, LSTM	K-means, hierarchical clustering, PCA	Q-learning, policy gradients (PPO)
Trading example	P(SPY up tomorrow) from RSI and recent returns	Calm / normal / stressed market regimes	An agent slicing a 100,000-share order to cut cost
Main risk	Overfitting noise; look-ahead leakage	Meaningless or unstable clusters	Exploiting simulator flaws; unstable training

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

3.2 Supervised Learning Models for Trading Signals

Maps to: K4, A4 | Slides 148, 152-154, 159

- Target and features: the target is what you predict (next-day direction); the features are what you know at prediction time (lagged returns, RSI, MACD, volatility, volume change). Any feature using future data is leakage.
- Linear regression predicts a continuous return as a weighted sum of features; logistic regression outputs the probability of an up day and is a strong, interpretable baseline.
- Naive Bayes applies Bayes' theorem with an independence assumption; fast and surprisingly useful for text such as news headlines.
- Decision trees split the data on feature thresholds; random forests average many decorrelated trees; XGBoost builds trees sequentially, each correcting the last - typically the strongest tabular models.
- Neural networks (multi-layer perceptrons, LSTMs, Transformers) can model non-linear and sequential patterns, but need far more data and are easy to overfit on daily prices.
- Evaluation: accuracy, precision, recall, F1 and ROC AUC for classifiers; RMSE and R-squared for regressors. In trading, a model is judged finally by its net-of-cost strategy performance, not by accuracy alone.
- Turning probabilities into positions: go long when P(up) exceeds a threshold such as 0.55, stay flat otherwise, and re-train on a rolling window.

3.3 Unsupervised Learning: Clustering and PCA for Market Regimes

Maps to: K4, A4 | Slides 149, 155-156

- K-means clustering partitions observations into k groups by minimising within-cluster variance. Applied to rolling return and volatility features it separates calm-bull, choppy and stressed market regimes.
- Hierarchical clustering builds a tree (dendrogram) of similarity - used to group correlated assets and to build more diversified portfolios.
- Principal Component Analysis (PCA) finds orthogonal combinations that explain the most variance. For equities the first component is usually 'the market'; for yield curves the first three are level, slope and curvature.
- Always standardise features before clustering or PCA, and choose k with the elbow method or silhouette score - then name the clusters by inspecting their average characteristics.
- Regime-aware trading: run trend following in trending regimes and mean reversion in range-bound ones, or cut position size in the high-volatility regime.
- Clusters found on the whole history use future information; for trading, fit the clustering on past data only and assign new observations as they arrive.

3.4 Statistical Modelling Techniques for Financial Time Series

Maps to: K6, A4 | Slides 150, 157

- Returns, not prices: model log or simple returns, which are closer to stationary. Prices trend and wander; regressions on raw prices produce spurious results.
- Stylised facts of returns: fat tails, volatility clustering, weak autocorrelation of returns but strong autocorrelation of absolute returns. Normal-distribution assumptions understate tail risk.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

- Time-ordered validation: never shuffle a time series. Use a chronological train / validation / test split, scikit-learn's TimeSeriesSplit, or walk-forward analysis that re-fits as time advances.
- Feature engineering: lagged returns, rolling statistics, indicator values, calendar effects and cross-asset features - each computed only from data available at the decision time.
- Regularisation (L1/L2 penalties, tree depth limits, early stopping) controls overfitting; fewer, well-motivated features usually beat many.
- Classical time-series models - autoregression (AR), ARIMA and GARCH for volatility - remain useful baselines against which a machine learning model must prove its value.

3.5 A/B Testing Trading Strategies

Maps to: A5, K2 | Slides 151, 158

- An A/B test compares variant A (for example a 20/50 crossover) with variant B (a 10/30 crossover, or the same rule with an RSI filter) on the same data and period, to decide whether B is genuinely better.
- State the hypotheses first: H0 - the mean daily returns (or Sharpe ratios) are equal; H1 - variant B's is higher. Choose the significance level (commonly 5%) before running the test.
- Tests: Welch's t-test on daily returns; a paired test on the daily return difference; and a bootstrap confidence interval for the Sharpe difference, which does not assume normality.
- Interpretation: a p-value below 0.05 rejects H0; a confidence interval for the difference that excludes zero tells the same story with a size attached. Statistical significance is not economic significance - check the effect after costs.
- Multiple testing: trying 100 variants will produce about 5 'significant' results by chance. Apply a Bonferroni correction or hold out a final test period.
- Live A/B testing: run both variants side by side in separate paper accounts during forward testing to compare them on unseen data.

3.6 Pairs Trading: Statistical Arbitrage as a Hedging Algorithm

Maps to: K6, A4 | Slides 160-162

- Pairs trading holds one stock long and an economically linked stock short, in hedge-ratio proportion. Market moves that hit both legs largely cancel, so the position is a hedge: its profit or loss comes from the spread between the two, not from the direction of the market.
- Origins: pioneered by Gerry Bamberger and developed by Nunzio Tartaglia's quantitative group at Morgan Stanley in the 1980s; still used by statistical-arbitrage funds.
- Cointegration, not correlation: two prices are cointegrated when a weighted combination of them is stationary. Highly correlated stocks can still drift apart for ever; only a cointegrated spread is expected to revert.
- Engle-Granger method: (1) regress log price A on log price B - the slope is the hedge ratio and the residual is the spread; (2) run an ADF unit-root test on the residual. A test statistic below the Engle-Granger critical value (about -3.34 at 5%) indicates cointegration.
- Trading rule: standardise the spread as a z-score over a rolling window; go short the spread (short A, long B) when $z > +2$, long the spread when $z < -2$, and exit as z returns towards 0 (for example $|z| < 0.5$). Trade both legs on the next bar and charge costs on both.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

- Evidence: Gatev, Goetzmann and Rouwenhorst (2006) found a simple pairs rule on US stocks earned average annualised excess returns of up to 11% from 1962 to 2002; later studies find smaller and less consistent profits.
- Risks: the relationship can break (re-test cointegration on a rolling basis), the hedge ratio drifts, costs are paid twice, short positions incur borrow costs, and screening many pairs creates false discoveries.

Reference: Pairs Trading Step by Step

1. Select an economically linked pair
2. Estimate the hedge ratio (OLS)
3. Test cointegration (ADF on residual)
4. Compute the spread z-score
5. Enter / exit both legs
6. Monitor and re-test

Concept	Explanation
Why it is a hedge	Long the cheap stock, short the rich one in hedge-ratio proportion: market moves largely cancel
Choose the pair	Economically linked stocks (e.g. KO and PEP) that are cointegrated, not merely correlated
Hedge ratio and spread	Regress log price A on log price B: slope = hedge ratio, residual = the spread (Engle-Granger)
Test the spread	ADF test on the residual: a stationary spread tends to revert to its mean
Trade the z-score	Enter when |z| > 2, exit as z returns towards 0; trade both legs on the next bar
Risks	Cointegration breaks, hedge ratio drifts, double costs, short-borrow, multiple testing

Gatev, Goetzmann and Rouwenhorst (2006): a simple pairs rule on US stocks earned average annualised excess returns of up to 11% (1962-2002); later studies find smaller and less consistent profits.

How It Works: Logistic Regression for Direction

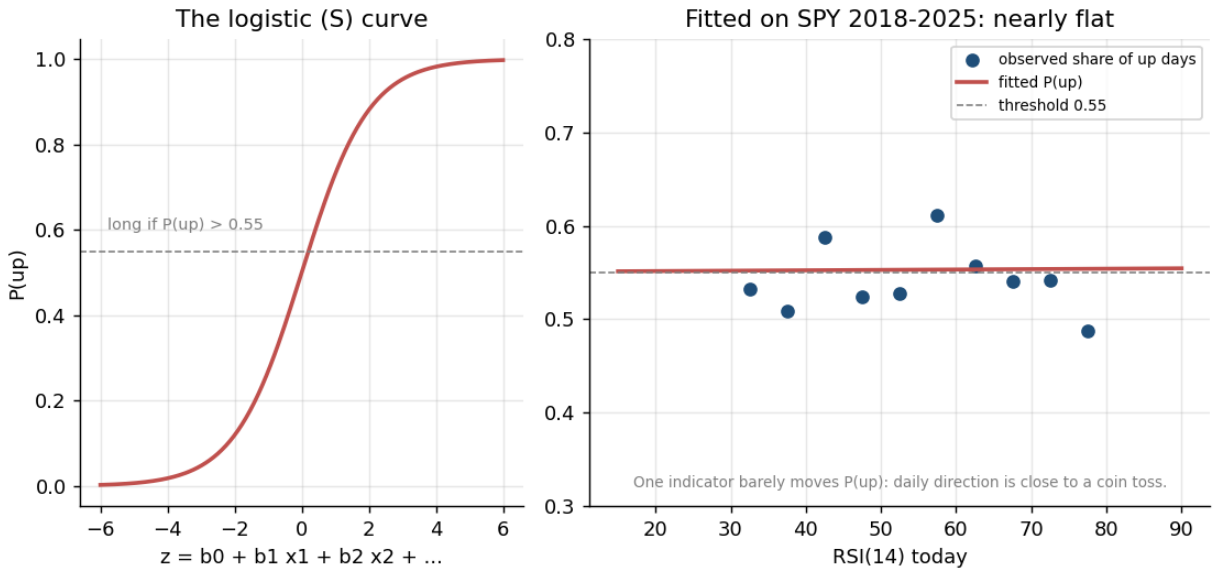


Figure: Algorithm: Logistic Regression for Direction - illustrated on real market data.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

How it works	Fits P(up tomorrow) = 1 / (1 + e^-(b0 + b1 x1 + ...)) from features known today.
Rule	Go long when P(up) exceeds a threshold such as 0.55; otherwise stay flat.
Works best when	A transparent baseline: each coefficient shows how a feature moves the probability.
Watch out for	Only linear effects unless you engineer features; daily direction is close to a coin toss.

How It Works: Decision Tree and Random Forest

Decision tree (depth 2) on SPY: split on feature thresholds; a random forest averages many such trees

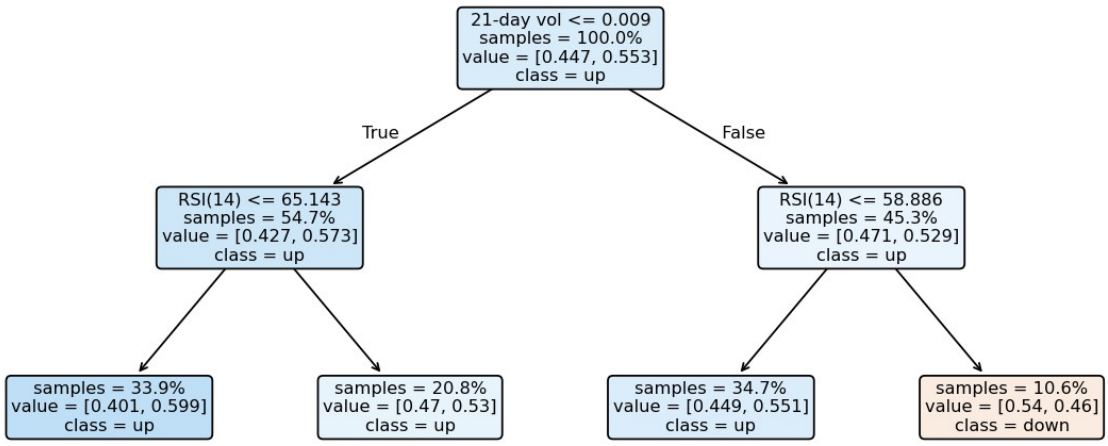


Figure: Algorithm: Decision Tree and Random Forest - illustrated on real market data.

How it works	A tree splits the data on feature thresholds (e.g. RSI < 30) into leaves with an up/down share.
Rule	A random forest averages hundreds of trees, each grown on a random sample of rows and features.
Works best when	Non-linear, interacting signals - e.g. oversold AND low volatility.
Watch out for	Deep trees memorise noise; limit depth and leaf size and validate walk-forward.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

How It Works: Gradient Boosting (XGBoost)

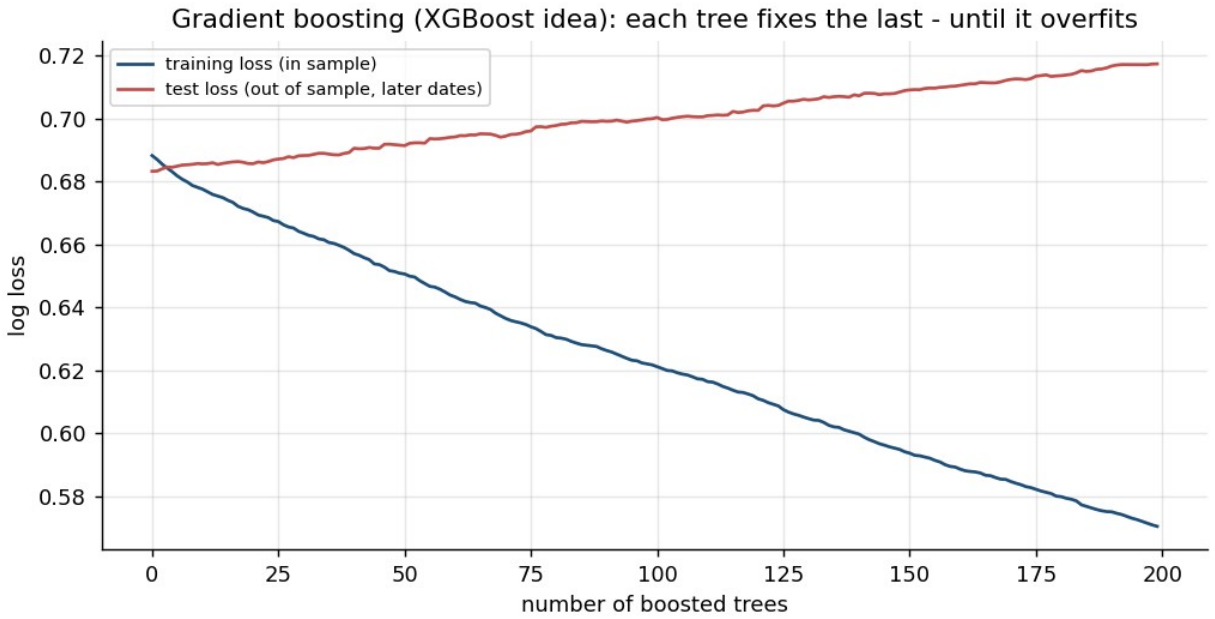


Figure: Algorithm: Gradient Boosting (XGBoost) - illustrated on real market data.

How it works	Builds trees one after another; each new tree corrects the errors of the ensemble so far.
Rule	Prediction = sum of many small trees x learning rate; stop early when test loss stops falling.
Works best when	Tabular features such as returns, indicators and volatility - usually the strongest model.
Watch out for	Training loss keeps falling while test loss rises: classic overfitting (see the chart).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

How It Works: K-Means Clustering for Market Regimes

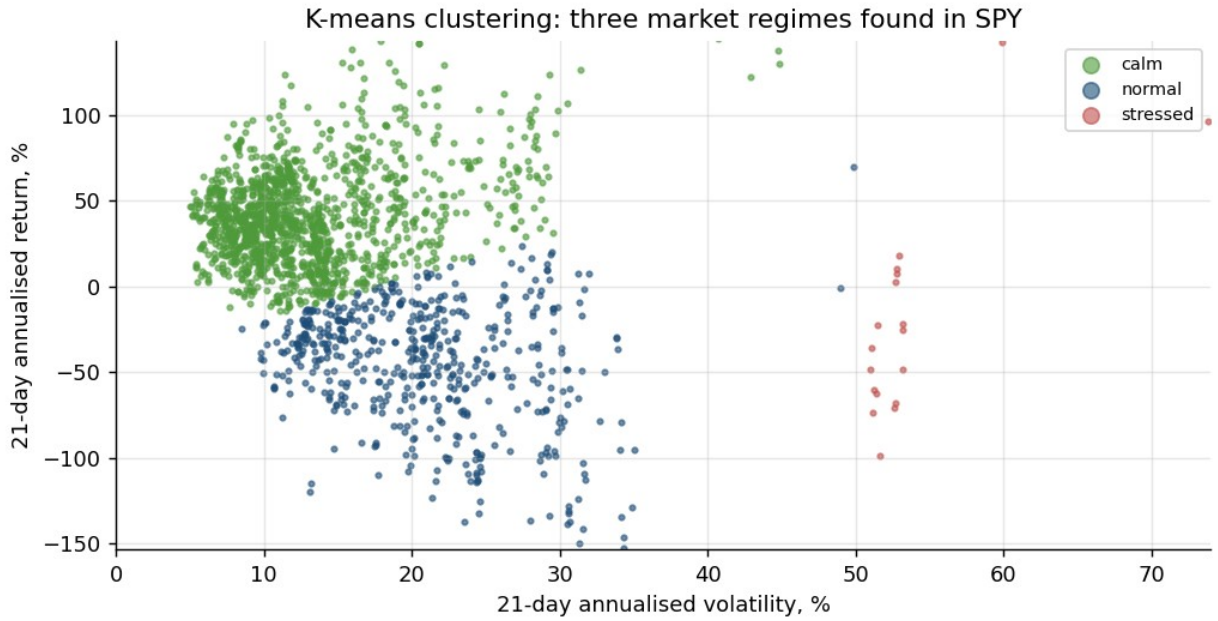


Figure: Algorithm: K-Means Clustering for Market Regimes - illustrated on real market data.

How it works	Groups days into k clusters by minimising the distance of each day to its cluster centre.
Rule	Features: rolling return and volatility (standardised). Name clusters by their averages.
Works best when	Switching or sizing strategies by regime: trend in calm markets, cut risk when stressed.
Watch out for	Fit on past data only; choose k with the elbow or silhouette method; regimes shift.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

How It Works: Principal Component Analysis (PCA)

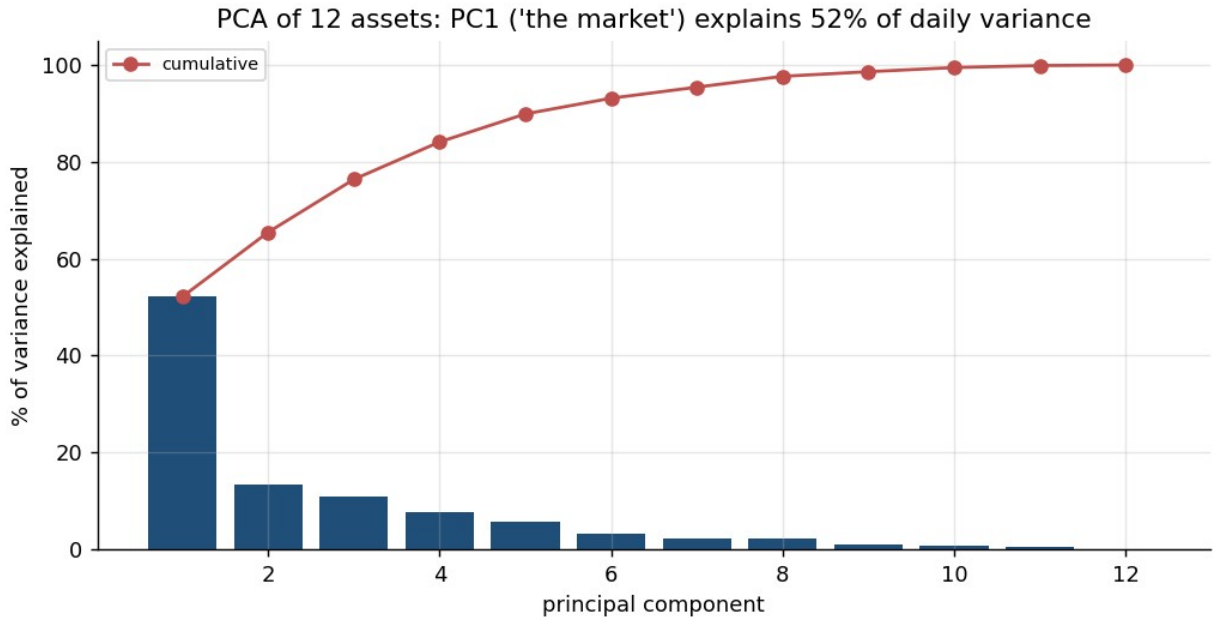


Figure: Algorithm: Principal Component Analysis (PCA) - illustrated on real market data.

How it works	Finds uncorrelated combinations of assets that explain the most variance of returns.
Rule	PC1 of a stock universe is 'the market'; later PCs are sector or style factors.
Works best when	Reducing many correlated features; building factor-neutral and pairs portfolios.
Watch out for	Standardise first; components change over time; PCs are statistical, not economic, factors.

How It Works: Walk-Forward Validation

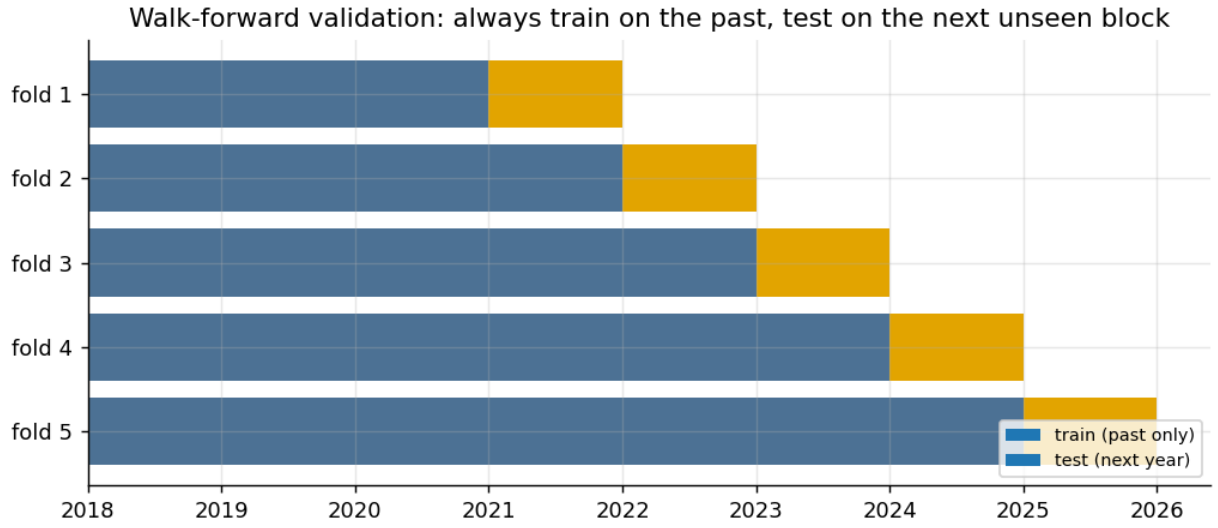


Figure: Method: Walk-Forward Validation - illustrated on real market data.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

How it works	Train on a past window, test on the next unseen block, then roll forward and repeat.
Rule	Never shuffle time series; each test block is genuinely out of sample.
Works best when	Estimating how a model or parameter set would really have performed live.
Watch out for	Overlapping targets leak information - leave a gap between train and test.

How It Works: A/B Testing Two Strategy Variants

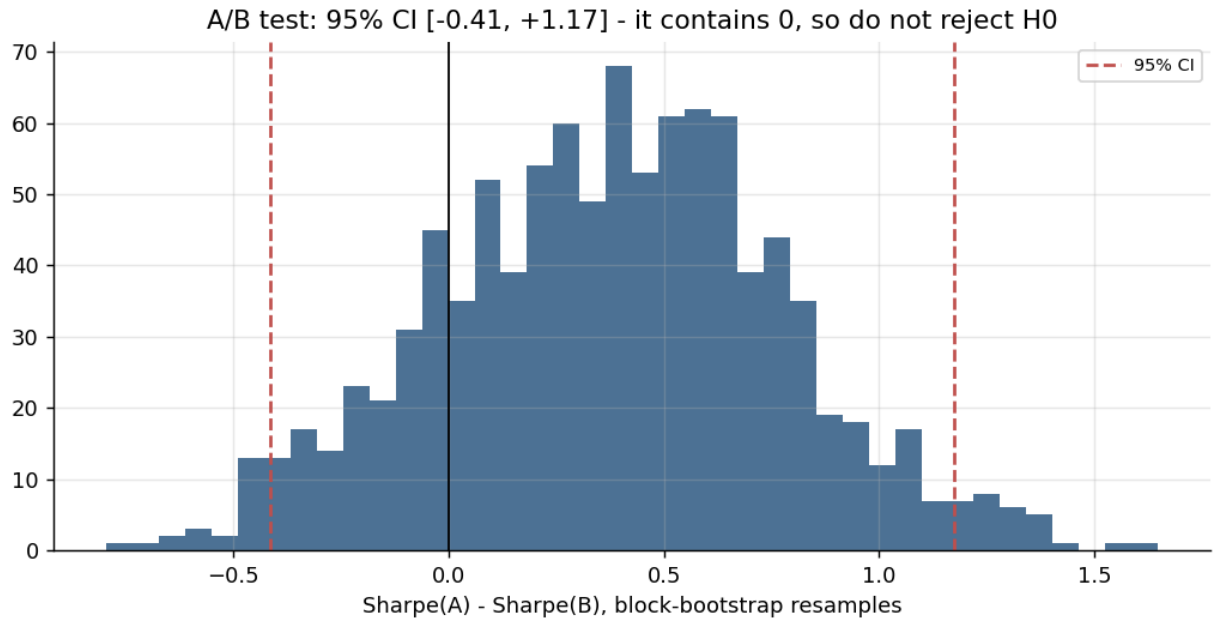


Figure: Method: A/B Testing Two Strategy Variants - illustrated on real market data.

How it works	Compare variant A and B on the same days: H0 'no difference' vs H1 'B is better'.
Rule	Welch / paired t-test on daily returns; bootstrap a 95% CI of the Sharpe difference.
Works best when	Deciding which parameter set or filter to promote to paper trading.
Watch out for	If the CI contains 0, do not reject H0; testing 100 variants gives ~5 false winners.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

How It Works: Pairs Trading (Hedged Statistical Arbitrage)

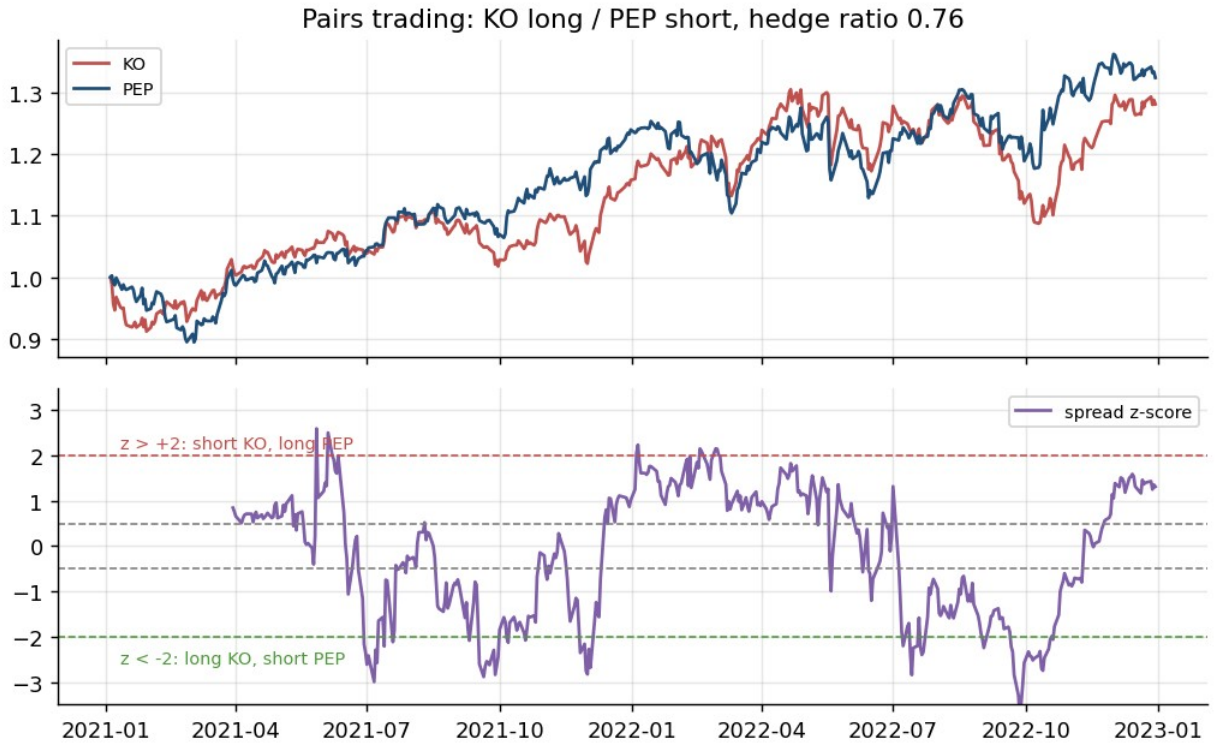


Figure: Algorithm: Pairs Trading (Hedged Statistical Arbitrage) - illustrated on real market data.

How it works	Long one stock, short a cointegrated partner in hedge-ratio proportion: market moves cancel.
Rule	Spread z > +2: short A / long B. z < -2: long A / short B. Exit as z returns to 0.
Works best when	Economically linked stocks (KO/PEP) whose spread has a stable mean.
Watch out for	Cointegration can break; costs on both legs; short-borrow; screening many pairs.

3.7 Lab 10: Strategy-Search Stage: Grid Search, Out-of-Sample Ranking and Decay

Maps to: A2, A4, K2 | Duration: 20 min | Slide 163

Scenario

A junior analyst wants the pipeline to 'find the best parameters'. The head of research agrees on one condition: parameters are ranked on 2018-2021 only and judged on the untouched 2022-2025 period (including the 2022 bear market), and the stage must report how much of the in-sample result decays.

Objective

Build `python -m pipeline search` to grid-search three signal families, rank parameter sets in sample, measure out-of-sample decay, prefer a robust parameter region, and decide whether anything deserves promotion into config.yaml.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Files

labs_fullcode/Lab10-strategy-grid-search/ - Lab10-strategy-grid-search/search_stage.py, pipeline/search.py, Lab10-strategy-grid-search/solution/search.py

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 10	python3 Lab10-strategy-grid-search/search_stage.py (same as: python -m pipeline search)
The complete code	pipeline/search.py (reference copy: Lab10-strategy-grid-search/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab10-strategy-grid-search/Lab10_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

Step 1: Define the search space (A2, K2)

Read GRIDS in Lab10-strategy-grid-search/solution/search.py: sma_cross fast x slow (24 valid pairs), rsi_reversion n x entry x exit (36), bollinger_breakout window x k (16) - 76 sets.

Check: You can say why fast 50 / slow 50 is skipped and why every extra set increases the chance of a lucky winner.

Step 2: Build the grid search (A2, A4)

Ask your assistant for the search engine.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/search.py. IS_END = '2021-12-31', OOS_START = '2022-01-01'. GRIDS = {'sma_cross': {'fast': [5,10,20,30,50], 'slow': [50,100,150,200,250]}, 'rsi_reversion': {'n': [2,3,5,14], 'entry': [10,20,30], 'exit': [50,60,70]}, 'bollinger_breakout': {'window': [10,20,30,50], 'k': [1.0,1.5,2.0,2.5]}}. grid_points(name) yields parameter dicts (skip sma_cross with fast >= slow). search(bars, costs) runs ONE backtest per set with pipeline.signals.target_position and common.backtest.backtest (signals are causal, so one run covers both windows), then records IS and OOS Sharpe and CAGR (common.metrics) and trades; add IS_rank and OOS_rank (method='min', NaN last) and decay = OOS - IS Sharpe. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: search(...) returns a DataFrame with 76 rows.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 3: Add ranking, robustness and the decay report (A4, K2)

Ask your assistant for run() and main().

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/search.py add robust_choice(ma) - for the sma_cross grid, the cell with the best AVERAGE in-sample Sharpe over itself and its neighbours - and run(symbol='SPY') for `python -m pipeline search`: print buy-and-hold Sharpe IS and OOS; a decay report with the best IS set per family (IS -> OOS Sharpe, decay, OOS rank of 76, trades); the top 5 by IS Sharpe; the Spearman rank correlation between IS and OOS Sharpe; the average decay of the top 5; the share of all sets that beat buy and hold OOS; the single best MA cell versus the robust MA region; and a suggested config.yaml strategy line (do NOT edit the config automatically). Save _outputs/Lab10_grid_results.csv, _outputs/Lab10_decay_report.csv and chart Lab10_grid_heatmap.png (IS and OOS Sharpe heatmaps of the MA grid with values in each cell). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline search --help shows --symbol.

Step 4: Run the search (A4)

Run: python -m pipeline search --symbol SPY (same as: python3 Lab10-strategy-grid-search/search_stage.py).

Check: 76 sets; buy-and-hold Sharpe 0.87 IS and 0.67 OOS. Best per family: sma_cross 10/100 IS 1.23 -> OOS 0.70 (decay -0.53, OOS rank 26); rsi_reversion n 2, entry 30, exit 70: 0.60 -> 0.74; bollinger_breakout 20 / 1.0: 0.91 -> 1.04 (OOS rank 3).

Step 5: Read the overfitting evidence (A4, K2)

Read the top-5 table and the three summary lines.

Check: Top 5 are all sma_cross; average decay -0.62 Sharpe points; IS/OOS rank correlation 0.06 (the in-sample ranking has almost no predictive value); 43% of all sets beat buy and hold out of sample.

Step 6: Prefer a robust region to a single peak (A2, A4)

Compare the single best MA cell with the robust MA region and open _outputs/Lab10_grid_heatmap.png.

Check: Best cell 10/100: IS 1.23 -> OOS 0.70; robust region 5/50: neighbour-average IS 1.04, own IS 0.92 -> OOS 0.44 - robustness reduces luck but guarantees nothing.

Step 7: Decide whether to promote anything (A2, K2)

Do not edit config.yaml yet. Weigh the evidence first.

AI prompt - paste into your AI coding assistant
Using labs_fullcode/_outputs/Lab10_decay_report.csv and labs_fullcode/_outputs/Lab10_grid_results.csv, argue for and against replacing the pipeline's configured ema_cross 10/30 with the best sma_cross or bollinger_breakout set. Mention the number of sets tried (76), the decay, the rank correlation and what the diagnostics stage (Lab 18) should check before any change. Finish with a one-line recommendation.

Check: A balanced recommendation that keeps ema_cross until a diagnostics run and an A/B test (Lab 13) support a change.

Expected Output (Full Code)

python -m pipeline search --symbol SPY: 76 parameter sets; buy-and-hold Sharpe 0.87 in sample and 0.67 out of sample; best sma_cross (10/100) 1.23 -> 0.70; best rsi_reversion (2, 30, 70) 0.60 -> 0.74; best bollinger_breakout (20, 1.0) 0.91 -> 1.04; top-5 average decay -0.62; IS/OOS rank correlation 0.06;

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

43% of sets beat buy and hold out of sample; files _outputs/Lab10_grid_results.csv, _outputs/Lab10_decay_report.csv and chart _outputs/Lab10_grid_heatmap.png.

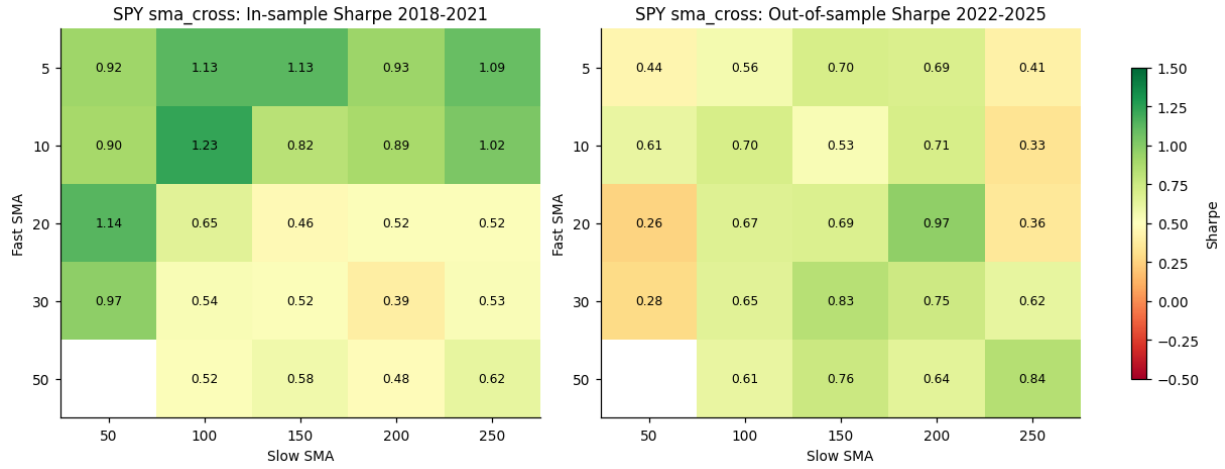


Figure: Lab 10 output (labs_fullcode/_outputs/Lab10_grid_heatmap.png), offline mode on the bundled data.

Debrief

Grid search is a statistical search over model structures (A2, A4); comparing in-sample with out-of-sample results (K2) exposes how much of the 'best' result was luck. A hold-out period, a count of trials and a preference for robust regions are the pipeline's defences against fooling itself.

Troubleshooting

Symptom	What to do
IntCastingNaNError on ranks	Some sets never trade in one window (NaN Sharpe); rank with na_option='bottom'.
Heatmap has a blank cell	Expected: fast 50 / slow 50 is not a valid pair.
Different results online	Alpaca's IEX data differ slightly from the bundled data; compare patterns, not decimals.

3.8 Lab 11: ML-Signal Stage: a Walk-Forward Classifier from Probability to Position

Maps to: A4, K4, K6 | Duration: 20 min | Slide 166

Scenario

The desk's data scientist claims a machine-learning model reading technical features can predict whether SPY closes up tomorrow. The pipeline will only accept an ML signal that is validated walk-forward, turned into positions with an explicit probability threshold and traded next bar with the same costs as every other signal - and that still beats buy and hold.

Objective

Build pipeline/ml.py: engineer causal features, train logistic regression, random forest and XGBoost with walk-forward TimeSeriesSplit validation, turn P(up) into a target position and backtest it after costs.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Files

labs_fullcode/Lab11-ml-signal-classifier/ - Lab11-ml-signal-classifier/ml_stage.py, pipeline/ml.py, Lab11-ml-signal-classifier/solution/ml.py

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 11	python3 Lab11-ml-signal-classifier/ml_stage.py (same as: python -m pipeline ml)
The complete code	pipeline/ml.py (reference copy: Lab11-ml-signal-classifier/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab11-ml-signal-classifier/Lab11_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

Step 1: Build causal features and the label (K4)

Ask your assistant for build_features().

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/ml.py with build_features(bars): features known at the close of day t only - ret_lag1/2/3/5 (r.shift(lag-1), so ret_lag1 = today's return), mom_5, mom_21, rsi_14 and rsi_2 (pipeline.indicators.ta_rsi), macd_hist / close (ta_macd), Bollinger %B from ta_bb, vol_21 annualised, dist_sma50 and dist_sma200, volume z-score over 21 days; label target = 1 if close t+1 > close t. Drop the last row (no label yet) and rows with NaN. The label may use shift(-1); no FEATURE may. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: build_features(load_bars('SPY','2018-01-01','2025-12-31')) has 1,811 rows and 15 columns (14 features + target).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 2: Add walk-forward validation (K4, K6)

Ask your assistant for the models and the walk-forward loop.

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/ml.py add MODELS = {'Logistic regression': StandardScaler + LogisticRegression(C=0.1, max_iter=1000), 'Random forest': RandomForestClassifier(n_estimators=300, max_depth=4, min_samples_leaf=20, random_state=42), 'XGBoost': XGBClassifier(n_estimators=200, max_depth=3, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8, random_state=42, eval_metric='logloss') imported lazily}, and walk_forward_proba(X, y, make, splits=5) using TimeSeriesSplit(n_splits=5, gap=1): fit on each training block, predict_proba on the following test block, return the out-of-sample probabilities and the last model. Never fit on future data. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: walk_forward_proba returns probabilities only for the five test blocks (2020-01-06 onwards).

Step 3: Turn probability into a position and backtest it (A4)

Ask your assistant for position_from_probability(), run() and main().

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/ml.py add position_from_probability(prob, threshold, index=None) = 1.0 where P(up) > threshold (decided at close t, so the backtester fills it at the next open), and run(symbol='SPY', threshold=0.55, models=None) for `python -m pipeline ml --threshold --models`: print samples, features, base rate and the five folds; out-of-sample accuracy, precision, AUC and share predicted up per model, with the naive 'always up' benchmark; backtest each model's positions from the first test date with common.backtest.backtest and the config cost model against buy and hold (CAGR, Sharpe, max drawdown, exposure, trades, turnover); print a VERDICT ('NO EDGE' if the best AUC < 0.53); save _outputs/Lab11_ml_probabilities.csv and chart Lab11_ml_strategy.png (equity curves and XGBoost feature importance from the last fold). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline ml --help shows --threshold and --models.

Step 4: Run the ML stage (A4, K4)

Run: python -m pipeline ml --threshold 0.55 (same as: python3 Lab11-ml-signal-classifier/ml_stage.py).

Check: 1,811 samples, 14 features, base rate 55.4%; folds test from 2020-01-06 to 2025-12-30. Out-of-sample AUC: logistic regression 0.481, random forest 0.483, XGBoost 0.497 (accuracy 0.535 / 0.542 / 0.519 versus 0.551 for always 'up').

Step 5: Read the strategy result (A4)

Read the backtest table and the VERDICT.

Check: 2020-2025, threshold 0.55: CAGR 9.19% / 7.70% / 4.00% versus buy and hold 14.98%; Sharpe 0.61 / 0.54 / 0.32 versus 0.78; turnover 83 / 75 / 96 versus 0.17; 'VERDICT: NO EDGE - keep the ML signal out of the live strategy'.

Step 6: Test the threshold - and notice the snooping risk (A4, K6)

Run: python -m pipeline ml --threshold 0.5 --models 'Logistic regression', then --threshold 0.6.

Check: Threshold 0.50: CAGR 16.82%, Sharpe 0.93, exposure 87%, 128 trades - better than buy and hold; threshold 0.60: 7.67%, Sharpe 0.60, exposure 23%, 205 trades. Choosing 0.50 AFTER seeing this table is data snooping: it must be re-tested out of sample and counted as an extra trial.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 7: Prove that leakage is detectable (K6)

Ask your assistant for a throw-away leakage experiment (do not save it into pipeline/ml.py).

AI prompt - paste into your AI coding assistant
In a Python shell started in labs, import pipeline.ml, wrap build_features so it also adds a feature 'leak' = tomorrow's return (bars['close'].pct_change().shift(-1)), run walk_forward_proba with the 'Logistic regression' model and print the out-of-sample AUC. Explain why the result proves the feature leaks the future, then discard the code.

Check: The leaky model scores AUC 0.997 - far too good; the honest model scores 0.481. Accuracy above about 60% on daily direction is a leakage alarm.

Step 8: Inspect the feature importance (K4)

Open _outputs/Lab11_ml_strategy.png. Which features does XGBoost rely on, and does that make economic sense?

Check: The importance bars are spread across many features with no dominant one - consistent with no reliable edge.

Expected Output (Full Code)

python -m pipeline ml --threshold 0.55: 1,811 samples, 14 features, base rate 55.4%; out-of-sample AUC 0.481 (logistic regression), 0.483 (random forest), 0.497 (XGBoost); strategies 2020-2025: CAGR 9.19% / 7.70% / 4.00% versus buy and hold 14.98%, Sharpe 0.61 / 0.54 / 0.32 versus 0.78, turnover 75-96 versus 0.17; VERDICT: NO EDGE; a leaky feature pushes AUC to 0.997; files _outputs/Lab11_ml_probabilities.csv and chart _outputs/Lab11_ml_strategy.png.

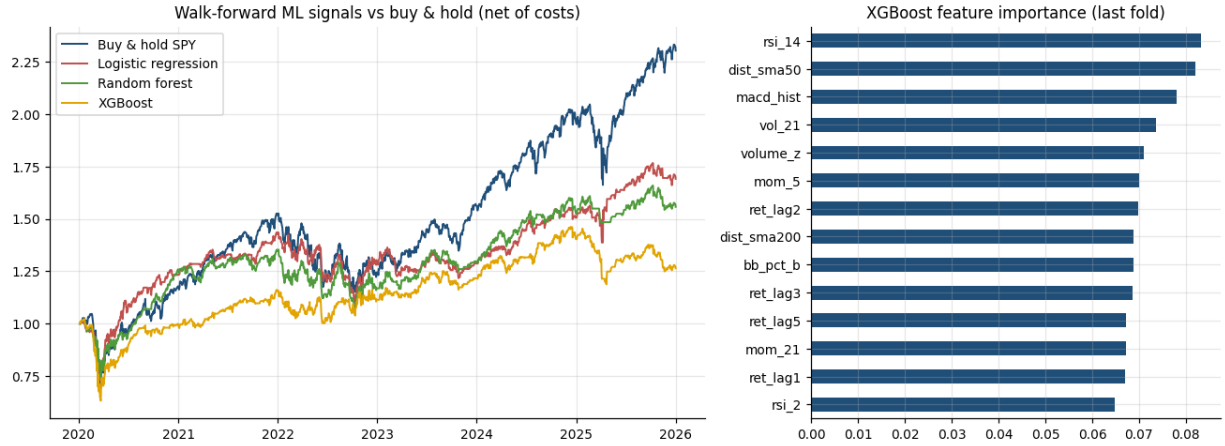


Figure: Lab 11 output (labs_fullcode/_outputs/Lab11_ml_strategy.png), offline mode on the bundled data.

Debrief

Supervised models (K4) can be trained and validated correctly (K6) and still find no edge - an honest negative result is valuable evidence (A4). Walk-forward validation stops the model learning from the future, a leakage test proves the check works, and the cost-aware backtest shows what an AUC of 0.5 is worth in money.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Troubleshooting

Symptom	What to do
xgboost not installed	Run pip install xgboost, or use --models 'Logistic regression' 'Random forest'.
Accuracy above 60%	Suspect leakage: no feature may use shift(-1), and TimeSeriesSplit needs gap=1.
Results differ in the third decimal	Random forest and XGBoost use random_state=42; library versions can still differ slightly.

3.9 Lab 12: Regime-Filter Stage: PCA and K-Means as a Gate on the Signals

Maps to: A4, K4, K6 | Duration: 15 min | Slide 168

Scenario

The risk team suspects the desk's signals behave differently in calm and stressed markets. It asks for an unsupervised regime model that can switch signals off in stress - fitted walk-forward so that the gate never uses information from the future.

Objective

Build pipeline/regime.py: PCA on the universe's returns, K-means regimes on backward-looking SPY features, walk-forward regime labels, and a gate that forces signals flat in the stress regime, evaluated after costs.

Files

labs_fullcode/Lab12-market-regimes/ - Lab12-market-regimes/regime_stage.py, pipeline/regime.py, Lab12-market-regimes/solution/regime.py

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 12	python3 Lab12-market-regimes/regime_stage.py (same as: python -m pipeline regime)
The complete code	pipeline/regime.py (reference copy: Lab12-market-regimes/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab12-market-regimes/Lab12_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

Step 1: Build PCA and the regime features (K4)

Ask your assistant for the first part of pipeline/regime.py.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/regime.py. regime_features(closes) builds backward-looking SPY features: vol_21 (annualised 21-day volatility), ret_21, ret_63, dist_sma200 (close / ta_sma(200) - 1) and corr_spy_tlt (63-day rolling correlation of SPY and TLT returns), dropping NaN rows. fit_regimes(train, k) fits StandardScaler + KMeans(n_clusters=k, n_init=20, random_state=42) and names clusters by average vol_21: 'Low vol', 'Moderate vol', 'High vol (stress)' for k=3, otherwise 'Regime i (vol rank)' with the highest named 'High vol (stress)'. In run() also fit sklearn PCA on standardised daily returns of config universe.trade (pipeline.ingest.load_closes) and print variance explained by PC1-PC4 and the loadings. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: fit_regimes returns a scaler, a fitted KMeans and a dict of names ordered from low to high volatility.

Step 2: Add walk-forward labels and the gate (K6, A4)

Ask your assistant for the walk-forward loop, regime_gate(), run() and main().

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/regime.py add walk_forward_regimes(feat, k, first_year=2020): for each calendar year fit_regimes on data up to 31 December of the previous year only and label that year's days; regime_gate(target, regimes, blocked=('High vol (stress)',)) forces the target to 0 on blocked days (the regime is known at the close, so the gate acts on the next bar like any signal). run(k=3) for `python -m pipeline regime --clusters`: print [1] PCA, [2] full-sample cluster profiles (labelled hindsight, description only) with next-day SPY Sharpe and trend-filter Sharpe by regime, [3] walk-forward regime counts and the latest regime and gate state, [4] gated versus ungated CAGR, Sharpe, max drawdown, exposure and trades from 2020 for the configured strategy, trend_filter and dip_buy (backtest with config costs). Save _outputs/Lab12_regime_labels.csv and chart Lab12_regimes.png (PCA variance, SPY coloured by walk-forward regime, gated vs ungated equity). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline regime --help shows --clusters.

Step 3: Run the regime stage and read the PCA (A4, K4)

Run: python -m pipeline regime --clusters 3 (same as: python3 Lab12-market-regimes/regime_stage.py).

Check: PCA on 9 assets x 2,010 days: PC1 56.8%, PC2 15.5%, PC3 10.3% (cumulative 82.5%). PC1 loads 0.33-0.43 on the equities and -0.09 on TLT: one broad equity factor; TLT and GLD are the diversifiers.

Step 4: Read the regime profiles (K4, K6)

Read section [2]. In which regime does the trend filter work, and where does it fail?

Check: Low vol 654 days (vol 12.7%), Moderate vol 786 days (vol 13.7%, stock-bond correlation -0.33), High vol (stress) 372 days (vol 30.4%). Trend-filter Sharpe 1.23 / 1.04 / -0.45 by regime.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 5: Check the walk-forward labels (K6)

Read section [3]. Why does the walk-forward model label far fewer stress days than the full-sample fit?

Check: 2020-2025: Low vol 1,061, Moderate vol 383, High vol (stress) 64 days; latest regime (2025-12-31) Low vol -> gate OPEN.

Step 6: Evaluate the gate after costs (A4)

Read section [4] and _outputs/Lab12_regimes.png. Should the gate go into the live pipeline?

AI prompt - paste into your AI coding assistant
Using the gated versus ungated table from python -m pipeline regime, write a short recommendation to the risk team on whether to switch the regime gate on for ema_cross, trend_filter and dip_buy. Quote Sharpe and max drawdown, mention that the gate is itself a model fitted on few stress days, and say which test from Lab 13 should decide.

Check: ema_cross 0.91 -> 0.89 and trend_filter 0.89 -> 0.80 (gate hurts); dip_buy 0.63 -> 0.72 with max drawdown -17.98% -> -12.37% (gate helps) - so the gate is signal-specific.

Step 7: Try four clusters (A4)

Run: python -m pipeline regime --clusters 4 and compare the profiles.

Check: Four regimes; the highest-volatility cluster has only 23 days (vol 80.8%, the March 2020 crash) - too few to trust a gate built on it.

Expected Output (Full Code)

python -m pipeline regime --clusters 3: PC1 56.8%, PC2 15.5%, PC3 10.3% (82.5% cumulative); regimes Low vol (654 days, vol 12.7%), Moderate vol (786, 13.7%), High vol (stress) (372, 30.4%); trend-filter Sharpe 1.23 / 1.04 / -0.45 by regime; walk-forward 2020-2025: 1,061 / 383 / 64 days, latest Low vol (gate open); gate effect on Sharpe: ema_cross 0.91 -> 0.89, trend_filter 0.89 -> 0.80, dip_buy 0.63 -> 0.72; files _outputs/Lab12_regime_labels.csv and chart _outputs/Lab12_regimes.png.

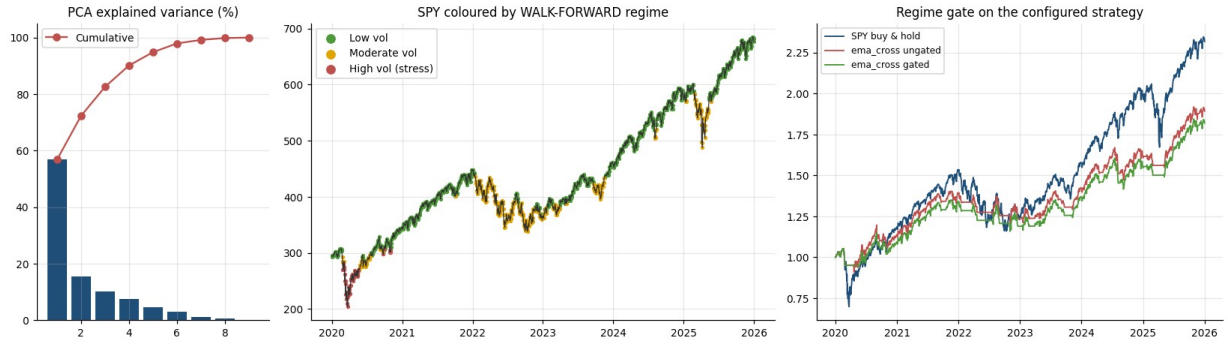


Figure: Lab 12 output (labs_fullcode/_outputs/Lab12_regimes.png), offline mode on the bundled data.

Debrief

Unsupervised learning (K4) finds structure without labels: PCA compresses correlated assets into a few drivers and K-means groups days into regimes. Fitting the regimes walk-forward is part of validating the model (K6), and testing the gate signal by signal after costs (A4) shows that a filter can help one strategy and hurt another.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Troubleshooting

Symptom	What to do
Cluster names swap between runs	K-means numbers clusters arbitrarily; fit_regimes names them by volatility so labels stay meaningful.
ConvergenceWarning	Increase n_init or max_iter in KMeans; results are usually unaffected.
Walk-forward starts in 2020	The first fit needs at least 200 days of features after the 200-day warm-up.

3.10 Lab 13: A/B-Test Stage: a Statistical Decision before Capital Moves

Maps to: A5, K2 | Duration: 15 min | Slide 170

Scenario

Two portfolio managers each champion a variant for the SPY sleeve: the 200-day trend filter and the 3-day dip buy. The pipeline's rule is that capital only moves between variants when an A/B test on the same days, with the same costs, says the difference is reliable - and the decision is recorded.

Objective

Build `python -m pipeline abtest` to compare any two library signals with a Welch t-test, a paired t-test and a block-bootstrap confidence interval for the Sharpe difference, and to record an accept or reject decision with a multiple-testing caution.

Files

labs_fullcode/Lab13-ab-testing/ - Lab13-ab-testing/abtest_stage.py, pipeline/abtest.py, Lab13-ab-testing/solution/abtest.py

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 13	python3 Lab13-ab-testing/abtest_stage.py (same as: python -m pipeline abtest)
The complete code	pipeline/abtest.py (reference copy: Lab13-ab-testing/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab13-ab-testing/Lab13_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

Step 1: State the hypotheses (A5, K2)

Write H0: both variants have the same expected daily return and Sharpe; H1: they differ. Choose alpha = 0.05 and note how many variants the desk has tried this quarter (default 20).

Check: H0, H1, alpha and the number of variants tried are written down before the test runs.

Step 2: Build the statistics (A5, K2)

Ask your assistant for the statistical core of pipeline/abtest.py.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/abtest.py. block_bootstrap_sharpe_diff(a, b, n_boot=2000, block=21, seed=7) resamples blocks of 21 consecutive days with numpy default_rng(seed) and returns Sharpe(A) - Sharpe(B) for each draw (annualised with sqrt(252)). ab_test(ra, rb, alpha=0.05, tried=20) returns the Welch t-test (scipy.stats.ttest_ind equal_var=False), the paired t-test (ttest_rel) and mean daily difference, the Sharpe difference and its (1 - alpha) bootstrap CI, the share of draws where A beats B, reject_h0 = paired p < alpha AND the CI excludes 0, and the Bonferroni threshold alpha / tried. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: ab_test returns a dict with welch_p, paired_p, ci and reject_h0.

Step 3: Build the A/B command (A5)

Ask your assistant for run() and main().

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/abtest.py add run(a='trend_filter', b='dip_buy', alpha=0.05, tried=20, symbol='SPY') for `python -m pipeline abtest --a --b --alpha --tried --symbol` (a and b chosen from pipeline.signals.SIGNALS): backtest both with the config cost model, use the SAME window from the first valid 200-day SMA, print CAGR, volatility, Sharpe, max drawdown, exposure and trades, the three tests, a DECISION line ('REJECT H0 - ...' or 'DO NOT REJECT H0 - <winner> looks better on paper, but the evidence is not reliable; keep both on paper') and the Bonferroni caution; write _outputs/Lab13_abtest_decision.yaml and chart Lab13_ab_test.png (both equity curves and the bootstrap histogram with the CI). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline abtest --help shows --a, --b, --alpha, --tried and --symbol.

Step 4: Run the A/B test (A5)

Run: python -m pipeline abtest --a trend_filter --b dip_buy --alpha 0.05 (same as: python3 Lab13-ab-testing/abtest_stage.py).

Check: Window 2018-10-16 to 2025-12-31 (1,812 days). A trend_filter vs B dip_buy: CAGR 9.70% vs 4.97%, Sharpe 0.81 vs 0.49, max drawdown -19.36% vs -23.93%, trades 24 vs 102.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 5: Compare the independent and paired tests (K2)

Read tests 1 and 2. Why does the paired design suit two strategies measured on the same days?

Check: Welch t = 0.74 (p = 0.462); paired t = 0.85 (p = 0.398), mean difference 0.00018 a day.

Step 6: Read the bootstrap CI and the decision (A5)

Read test 3 and the DECISION line, then explain the decision in plain words for the two portfolio managers.

AI prompt - paste into your AI coding assistant
Rewrite this A/B test result for two portfolio managers who are not statisticians, in under 100 words: <paste the three test lines and the DECISION line>. Explain why 'A looks better' is not the same as 'A is better', and what evidence would change the decision.

Check: Sharpe difference +0.31, 95% CI [-0.33, +1.05], A beats B in 82% of bootstrap samples; DECISION: DO NOT REJECT H0. Bonferroni with 20 variants: p < 0.0025.

Step 7: Test the pipeline's own strategy (A5, K2)

Run: python -m pipeline abtest --a ema_cross --b trend_filter.

Check: ema_cross Sharpe 0.95 vs trend_filter 0.81; Welch p 0.852, paired p 0.715, Sharpe difference +0.14, CI [-0.42, +0.67], A beats B in 66% - again DO NOT REJECT H0: the configured strategy is not proven better.

Expected Output (Full Code)

python -m pipeline abtest --a trend_filter --b dip_buy: window 2018-10-16 to 2025-12-31 (1,812 days); CAGR 9.70% vs 4.97%, Sharpe 0.81 vs 0.49; Welch t = 0.74 (p = 0.462); paired t = 0.85 (p = 0.398); Sharpe difference +0.31, 95% CI [-0.33, +1.05]; A beats B in 82% of bootstrap samples; DECISION: do not reject H0; Bonferroni p < 0.0025 for 20 variants; files _outputs/Lab13_abtest_decision.yaml and chart _outputs/Lab13_ab_test.png.

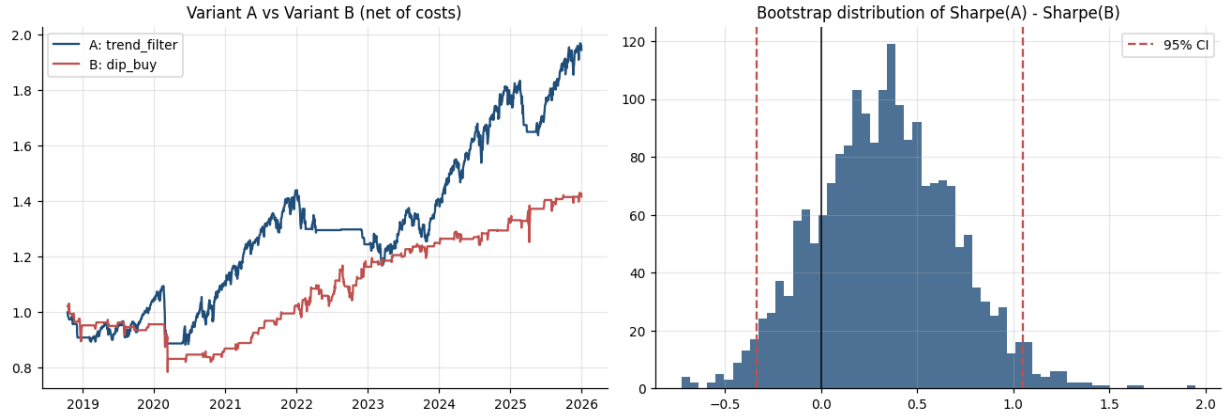


Figure: Lab 13 output (labs_fullcode/_outputs/Lab13_ab_test.png), offline mode on the bundled data.

Debrief

A/B testing (A5) applies statistical inference to a business choice: the trend filter looks better, but the evidence cannot reject 'no difference'. Statistical tests (K2) - paired where the data are paired, bootstrapped where returns are not normal, corrected for the number of variants tried - protect capital from being moved on noise.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Troubleshooting

Symptom	What to do
p-value differs slightly	The bootstrap depends on the seed (7) and the number of draws; the decision should not change.
t-tests and the CI disagree	The t-tests compare mean returns, the CI compares Sharpe ratios; report both.
Returns are not normal	Daily returns have fat tails - that is why the block bootstrap is used alongside the t-tests.

3.11 Lab 14: Pairs Stage: a Hedged, Market-Neutral Statistical-Arbitrage Sleeve

Maps to: A4, K6 | Duration: 20 min | Slide 172

Scenario

Lion City Capital's equity desk wants a sleeve whose returns do not depend on the direction of the market. The head of trading asks for the classic statistical-arbitrage hedge as a pipeline stage: hold one stock long and an economically linked stock short in hedge-ratio proportion, trade the spread when it stretches, and output per-leg target weights. Candidates: KO/PEP, JPM/BAC and XLF/JPM.

Objective

Build pipeline/pairs.py: screen pairs with the Engle-Granger cointegration test, estimate the hedge ratio, trade the spread z-score out of sample with next-bar execution and costs on both legs, confirm market neutrality and output leg targets.

Files

labs_fullcode/Lab14-pairs-trading/ - Lab14-pairs-trading/pairs_stage.py, pipeline/pairs.py, data/KO.csv, data/PEP.csv, data/BAC.csv, Lab14-pairs-trading/solution/pairs.py

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 14	python3 Lab14-pairs-trading/pairs_stage.py (same as: python -m pipeline pairs)
The complete code	pipeline/pairs.py (reference copy: Lab14-pairs-trading/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab14-pairs-trading/Lab14_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

Step 1: Read the method (K6)

Read the docstring of Lab14-pairs-trading/solution/pairs.py: the formation period 2018-2022 is used only to estimate the hedge ratio and test cointegration; the trading period 2023-2025 is out of sample with the hedge ratio held fixed.

Check: You can explain why the hedge ratio must be frozen before the trading period starts.

Step 2: Build the cointegration tools (K6)

Ask your assistant for the statistics part of pipeline/pairs.py.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/pairs.py with numpy-only statistics: ols(y, X) returning coefficients, standard errors and residuals; adf_t(series, lags=1) - the ADF t-statistic of g in d e_t = g e_(t-1) + phi d e_(t-1); engle_granger(log_a, log_b) - step 1 OLS log A = a + beta log B, step 2 adf_t on the residual - returning (alpha, beta, adf_t); half_life(spread) from an AR(1) fit of spread changes; EG_CRIT = {'1%': -3.90, '5%': -3.34, '10%': -3.04} (MacKinnon, two variables with constant). FORMATION = ('2018-01-01', '2022-12-31'), TRADING = ('2023-01-01', '2025-12-31'). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: engle_granger on log KO and log PEP over the formation period returns beta about 0.756 and ADF t about -3.36.

Step 3: Build the spread trading rules and leg targets (A4)

Ask your assistant for trade_pair(), leg_targets(), run() and main().

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/pairs.py add trade_pair(closes, a, b, alpha, beta, entry, exit_, window, cost_bps): spread = log A - beta log B - alpha; z = (spread - rolling mean) / rolling std over window; state machine: short spread (-1: short A, long B) when z > +entry, long spread (+1) when z < -entry, exit when |z| < exit_; held = state.shift(1) (decided at close t, held over t+1); gross = held x w x (rA - beta rB) with w = 1 / (1 + |beta|); costs = |change in held| x w x (1 + |beta|) x cost_bps per leg per side; return spread, z, held, state, gross, net. leg_targets(state, a, b, beta) -> {a: state x w, b: -state x beta x w}. run(pair=('KO', 'PEP'), entry=2.0, exit_=0.5, window=60, cost_bps=5.0) for `python -m pipeline pairs --pair --entry --exit --window --cost-bps`: screen config universe.pairs (correlation of returns, beta, ADF t, cointegrated at 5%?), print the chosen pair's hedge ratio and half-life (warn if not cointegrated), trade 2023-2025 using 120 days of history for the z warm-up, print gross, net and SPY return, CAGR, volatility, Sharpe and max drawdown, trades, time in market, beta and correlation to SPY, the latest leg targets, and a note that short legs need a margin account while the automation stages are long only. Save chart Lab14_pairs_trading.png (normalised prices, z with bands and open positions, equity vs SPY). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline pairs --help shows --pair, --entry, --exit, --window and --cost-bps.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 4: Run the pair screen (A4, K6)

Run: python -m pipeline pairs (same as: python3 Lab14-pairs-trading/pairs_stage.py). Compare correlation with the ADF t-statistic for each pair.

Check: KO/PEP: correlation 0.75, beta 0.76, ADF t -3.36 - cointegrated at 5% (critical -3.34). JPM/BAC 0.93, ADF -2.08 and XLF/JPM 0.94, ADF -1.89 - highly correlated but NOT cointegrated.

Step 5: Interpret the hedge ratio (K6)

Read section [2]. Explain in one sentence why the hedge ratio, not a 1:1 position, makes the spread stationary.

Check: KO/PEP hedge ratio 0.756 (short 0.76 of PEP per 1 of KO), half-life 39 days.

Step 6: Evaluate the out-of-sample result (A4)

Read section [3] and [4]: trades, time in market, beta to SPY and the latest leg targets.

Check: 2023-2025: 13 trades, 59% time in market, gross -9.4%, net -10.6% (Sharpe -0.60), beta to SPY +0.02, correlation +0.04, versus SPY +86.3%; latest z +0.65 -> spread position 0 -> leg targets KO 0.0, PEP 0.0. The hedge worked (beta near 0), but the spread did not revert profitably after 2022.

Step 7: Test a failed pair and other thresholds (A4, K6)

Run: python -m pipeline pairs --pair JPM BAC --entry 1.5 --exit 0. Explain why trading a pair that failed the cointegration test is a statistical error and why trying many pairs and thresholds is data snooping.

Check: JPM/BAC: 'WARNING: NOT cointegrated' (ADF -2.08, half-life 93 days); 1 trade held 100% of the time, net -21.9%, beta to SPY -0.03.

Step 8: Study the chart (A4)

Open _outputs/Lab14_pairs_trading.png: normalised prices, the z-score with entry and exit bands and open positions shaded, and the equity curve against SPY.

AI prompt - paste into your AI coding assistant
Looking at labs_fullcode/_outputs/Lab14_pairs_trading.png and the stage output, explain to the head of trading how the desk should re-test cointegration on a rolling basis and size each pair small before this sleeve could ever be traded, and why it stays a paper sleeve while the automation stages are long only.

Check: A rolling re-test plan (for example every quarter on the last 3 years) and a position-size limit per pair.

Expected Output (Full Code)

python -m pipeline pairs: screening KO/PEP ADF t -3.36 (cointegrated at 5%), JPM/BAC -2.08 and XLF/JPM -1.89 (not); KO/PEP hedge ratio 0.756, half-life 39 days; out of sample 2023-2025: 13 trades, 59% time in market, gross -9.4%, net -10.6% (Sharpe -0.60), beta to SPY +0.02, versus SPY +86.3%; leg targets KO 0.0 / PEP 0.0 at 2025-12-31; chart _outputs/Lab14_pairs_trading.png.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

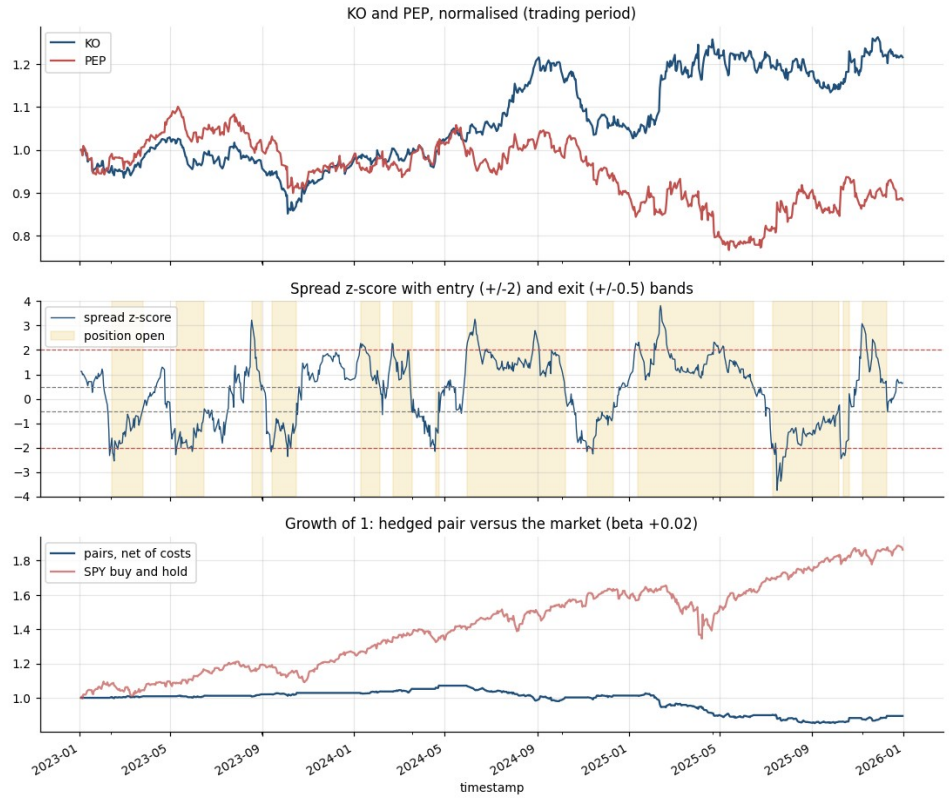


Figure: Lab 14 output (labs_fullcode/_outputs/Lab14_pairs_trading.png), offline mode on the bundled data.

Debrief

Pairs trading is a hedging algorithm: the short leg cancels most of the market exposure of the long leg, which is why the beta to SPY is near zero (A4). The statistical model is cointegration, not correlation (K6). A pair that is cointegrated in one period can stop reverting in the next, so professional desks re-test cointegration on a rolling basis and size each pair small.

Troubleshooting

Symptom	What to do
KeyError for KO, PEP or BAC	The bundled data folder is out of date; copy data/KO.csv, data/PEP.csv and data/BAC.csv from the course lab folder.
ADF t differs slightly online	Alpaca's IEX feed and the bundled Yahoo data differ a little; use the bundled data for the class comparison.
All pairs show 'no'	Check the formation period is 2018-2022 and that log prices are used.

3.12 Topic Summary

- A4 - Conduct statistical modelling of data to derive patterns and solutions
- A5 - Perform A/B testing based on different hypotheses
- K2 - Types of statistical analyses, data models, algorithms and advanced computational methods
- K4 - Range and application of various types of data models
- K6 - Statistical modelling techniques

Assessed by: Practical Performance Task 3 | Written Assessment Q2, Q4, Q6

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Topic 4: Building an End-to-End Algorithmic Trading Pipeline: Backtest, Forward Test and Performance Analysis

Day	Abilities	Knowledge	Labs	Slides
Day 2	A7, A8, A9, A10	K7, K8	15 - 20	175 - 198

Learning Outcome

Perform coding and configuration of an automated trading program based on selected models or algorithms, conduct tests on its actions and outcomes, diagnose unintended outcomes, and draw insights from the results to support decisions and report accordingly.

4.1 Architecture of an End-to-End Algorithmic Trading Pipeline

Maps to: A7, K7 | Slides 176, 182

- The pipeline: (1) strategy search and research, (2) data ingestion, (3) signal generation, (4) backtest with costs, (5) risk checks, (6) order execution, (7) forward test on paper, (8) monitoring, logging and performance reporting.
- Configuration lives outside the code: symbols, parameters, position limits and API keys sit in a config file or environment variables, so a change is a reviewed configuration change rather than an edit to the logic.
- Pre-trade risk controls: maximum position size, maximum order value, maximum daily loss, market-hours check, duplicate-order check and a kill switch that halts all trading - the controls regulators require of algorithmic traders.
- Event sources: a scheduled job (every bar), a streaming feed (every quote), or an external alert - such as a TradingView alert sent by webhook to your own server, which then places the order through Alpaca.
- Agentic extension: analyst agents produce a view, a trader agent proposes an order, and a risk-manager agent with hard limits approves or rejects it - the LLM never bypasses the coded risk checks.
- Every decision is logged: timestamp, inputs, signal, order request, broker response and fill - the audit trail for diagnosing problems and for compliance review.

4.2 Backtesting with Realistic Trading Costs

Maps to: A8, K7 | Slides 177-179

- A backtest replays a strategy on historical data. Vectorised backtests are fast (whole-array operations); event-driven backtests replay bar by bar and model orders more realistically.
- Execute on the NEXT bar: a signal computed on today's close can only be traded at tomorrow's open (or later). Same-bar execution is the commonest source of look-ahead bias.
- Trading costs: commission (Alpaca charges none on US equities, but most brokers do), the bid-ask spread, slippage and market impact, and regulatory fees on sells (the US SEC Section 31 fee and FINRA Trading Activity Fee). Short positions also incur borrow costs.
- Model costs explicitly: cost per trade = commission + (half-spread + slippage) x shares x price + regulatory fees. Express them in basis points of traded value so they scale with turnover.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

- Report gross and net results side by side, the cost breakdown, and the break-even cost - the cost per trade at which the strategy stops making money.
- A strategy whose edge disappears at 5 to 10 basis points per trade is not tradeable for most desks, however good its gross Sharpe ratio looks.

4.3 Forward Testing with Automated Paper Trading

Maps to: A7, A8 | Slides 183-184

- Forward testing (paper trading) runs the strategy in real time on data it has never seen, placing simulated orders - the bridge between a backtest and live capital.
- The automated bot loop: wait for the bar to close; fetch the latest bars; compute the signal; run the risk checks; compare the target position with the actual position; submit the buy or sell order; read the order status; log everything; sleep until the next bar.
- Alpaca's market clock (is the market open?) and calendar prevent orders outside trading hours; the pattern-day-trader rule applies to paper accounts under USD 25,000 as it does to live ones.
- Run the forward test long enough to see a meaningful number of trades - weeks for a daily strategy - and compare its fills, slippage and returns with what the backtest predicted for the same period.
- Run variants side by side in separate paper accounts as a live A/B test, and only promote a strategy to live trading through a documented review.

4.4 Potential Reasons for Unintended Outcomes and How to Diagnose Them

Maps to: A9, K8 | Slides 180, 185

- Overfitting: parameters tuned to noise. Signs - a spectacular in-sample Sharpe that collapses out of sample, and performance that is very sensitive to small parameter changes.
- Look-ahead bias: using information not available at the decision time - same-bar execution, centred rolling windows, or features normalised with full-sample statistics.
- Survivorship bias: testing only on today's index members, which excludes the companies that failed. Data-snooping bias: reporting the best of many tests without adjustment.
- Cost and execution drag: under-estimated spread and slippage, partial fills and rejected orders. Signs - paper or live returns persistently below the backtest by roughly the turnover x cost.
- Regime change and concept drift: the market behaves differently from the training period (2020 COVID volatility, the 2022 rate-hiking cycle). Monitor rolling performance and feature distributions.
- Data and software errors: unadjusted splits, time-zone mismatches, missing bars, duplicated orders, and stale positions after a restart. Knight Capital's 2012 loss came from obsolete code activated by a deployment error.
- Diagnosis method: reproduce the result, isolate the first step where it diverges from expectation (data, signal, order, fill), compare against a benchmark or a known-good run, fix, then re-test.

4.5 Performance Analysis, Insights and Reporting

Maps to: A10 | Slides 181

- A tearsheet presents the equity curve against the benchmark, the drawdown chart, monthly returns, rolling Sharpe ratio, the metrics table and the cost breakdown on one page.
- Compare backtest and forward test for the same period: did the live fills, number of trades and returns behave as the backtest predicted? Explain every material gap.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

- Draw insights, not just numbers: in which regimes did the strategy earn its returns, what drove its drawdowns, how sensitive is it to costs and parameters, and how correlated is it with the desk's existing book?
- Report for the decision-maker: lead with the recommendation (promote, extend the forward test, modify or retire), support it with three or four key figures, and state the risks and limitations plainly.
- Keep a model record: hypothesis, data, parameters, test results, version and approval - the documentation an investment committee and model-risk reviewer expect.

Historical Data: Backtest, Tune, Then Forward Test



Figure: one history, three jobs - SPY 2018-2025.

Backtest on history	Replay the strategy's rules over past prices - next-bar fills, trading costs - to see how it would have performed.
Tune hyperparameters (in-sample)	Search the settings (MA lengths, RSI levels, model parameters) on the OLDER history only, by grid search or walk-forward.
Confirm out-of-sample	Backtest the tuned settings ONCE on later history the search never saw. A big drop from in-sample = overfitting.
Forward test (paper)	Run the tuned strategy on NEW data as it arrives - paper orders, real fills and risk checks - before any capital.

The same rule moves through four tests; each uses data the previous step never touched.

1. Historical data (2018-2025)
2. Tune in-sample (2018-2021)
3. Backtest once out-of-sample (2022-2024)
4. Forward test on paper (2025 onward)
5. Go-live decision

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Tuning on the test period, or re-tuning after seeing out-of-sample results, turns the test into more tuning - the result will look better than reality (Labs 10, 15, 16).

4.6 Lab 15: Backtest-Engine Stage: Cost Models, Execution Timing and Break-Even

Maps to: A7, A8, K7 | Duration: 25 min | Slide 187

Scenario

Alpaca advertises commission-free trading and a trader concludes that costs do not matter. Before the pipeline is automated, the head of execution wants the backtest engine to show gross versus net results under four cost scenarios, where the money goes, the break-even cost of a fast and a slow strategy, and how sensitive results are to the assumed fill time.

Objective

Build pipeline/backtest.py around the common backtester: run any library signal under every cost scenario (and a custom one), break the cost into commission, spread, slippage and regulatory fees, find each strategy's break-even cost and test next-open versus next-close execution.

Files

labs_fullcode/Lab15-trading-costs/ - Lab15-trading-costs/backtest_stage.py, pipeline/backtest.py, common/backtest.py, Lab15-trading-costs/solution/backtest.py

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 15	python3 Lab15-trading-costs/backtest_stage.py (same as: python -m pipeline backtest)
The complete code	pipeline/backtest.py (reference copy: Lab15-trading-costs/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab15-trading-costs/Lab15_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 1: Read the cost model (K7)

Open common/backtest.py and read CostModel and COST_PRESETS (zero, alpaca_retail, per_share_broker, high_cost). Note that regulatory fees apply to sells only and that a trade costs half the spread plus slippage.

AI prompt - paste into your AI coding assistant
Explain CostModel.trade_cost in labs_fullcode/common/backtest.py with a worked example: a USD 95,000 buy and the matching sell of SPY at 676.64 under the alpaca_retail preset. Show each component in dollars. Do not change the file.

Check: A worked example where spread and slippage dominate and commission is zero.

Step 2: Build the engine helpers (A7)

Ask your assistant for the engine part of pipeline/backtest.py.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/backtest.py. run_backtest(bars, name, costs=None, execution='next_open', **params) = common.backtest.backtest(bars, pipeline.signals.target_position(name, bars, **params), costs, execution) - the single entry point later stages use. break_even_bps(bars, signal, lo=0, hi=1000): bisection (30 steps) on a one-way cost expressed as CostModel(slippage_bps=x) until net CAGR = 0 (return 0 if gross CAGR <= 0). parse_custom(['spread_bps=3', 'slippage_bps=3']) -> CostModel('Custom route', ...). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: break_even_bps on dip_buy returns about 19.5.

Step 3: Build the scenario report (A7, A8)

Ask your assistant for run() and main().

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/backtest.py add run(strategies=None, symbol='SPY', custom=None) for `python -m pipeline backtest --strategies --symbol --custom`: default strategies dip_buy and trend_filter; scenarios from config costs.scenarios plus the custom one; for each strategy and scenario print gross CAGR, net CAGR, cost drag, net Sharpe, executions and dollars of commission, spread, slippage, reg fees and total (from BacktestResult.cost_summary()); print each strategy's break-even bps per side next to its number of round trips and average gross trade in bps; print next-bar OPEN versus next-bar CLOSE CAGR and Sharpe with alpaca_retail costs; save _outputs/Lab15_cost_scenarios.csv and chart Lab15_costs.png (equity by scenario for the first strategy with the gross curve dashed, and a stacked bar of where the costs go). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline backtest --help shows --strategies, --symbol and --custom.

Step 4: Run the cost scenarios (A7, A8)

Run: python -m pipeline backtest --strategies dip_buy trend_filter (same as: python3 Lab15-trading-costs/backtest_stage.py).

Check: dip_buy: gross CAGR 5.09%; net 4.65% (Alpaca), 4.21% (per-share), 0.47% (high-cost, USD 33,560 of costs); 203 executions. trend_filter: gross 8.80%; net 8.70% / 8.59% / 7.68%; 47 executions.

Step 5: Analyse the cost breakdown (K7)

For the Alpaca scenario, which component is largest even though commission is zero?

Check: dip_buy with Alpaca costs: commission USD 0, spread 1,127, slippage 2,254, reg fees 317 - total USD 3,698; slippage and spread scale with turnover.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 6: Interpret break-even and execution timing (A8, K7)

Compare each break-even cost with the average trade, then read the execution-timing lines.

Check: Break-even: dip_buy 19.5 bps per side (102 round trips, average gross trade 41.7 bps) versus trend_filter 142.3 bps (24 round trips, 347.3 bps). Timing: dip_buy 4.65% at the next OPEN but 7.22% at the next CLOSE - the fill assumption alone moves the result; trend_filter 8.70% vs 8.66%.

Step 7: Configure your own execution route (A7)

Run: python -m pipeline backtest --custom spread_bps=3 slippage_bps=3 and record the new net CAGR for both strategies.

Check: Custom route: dip_buy net 3.89% (USD 9,833 of costs); trend_filter 8.51% (USD 2,445).

Expected Output (Full Code)

python -m pipeline backtest: dip_buy gross CAGR 5.09%, net 4.65% (Alpaca) / 4.21% (per-share) / 0.47% (high-cost, USD 33,560 of costs); trend_filter gross 8.80%, net 8.70% / 8.59% / 7.68%; break-even 19.5 bps per side for dip_buy (102 round trips, average trade 41.7 bps) versus 142.3 bps for trend_filter; next-close fills lift dip_buy to 7.22%; custom route 3.89% / 8.51%; files _outputs/Lab15_cost_scenarios.csv and chart _outputs/Lab15_costs.png.

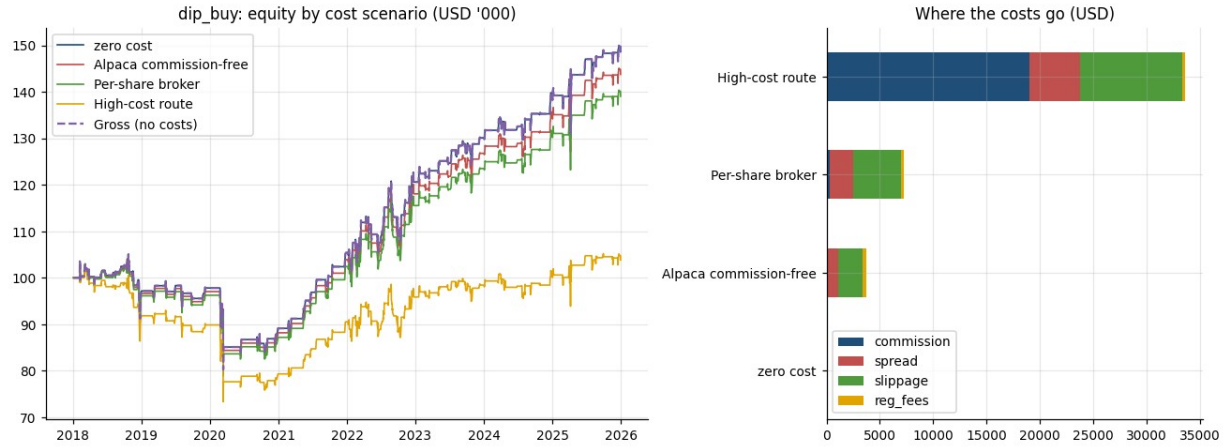


Figure: Lab 15 output (labs_fullcode/_outputs/Lab15_costs.png), offline mode on the bundled data.

Debrief

Commission is only one trading cost (K7): spread and slippage scale with turnover, and the assumed fill time can matter as much as the fee schedule. Coding the cost model into the engine (A7) and testing outcomes under several scenarios (A8) shows whether an edge survives real-world execution - the break-even cost is the single number that says how much room the strategy has.

Troubleshooting

Symptom	What to do
Net CAGR equals gross CAGR	You are using the zero preset; check costs.preset and costs.scenarios in config.yaml.
Regulatory fees look tiny	They are: a few dollars per million of sell notional. Rates change - check the current SEC and FINRA rates.
Break-even hits the upper bound	Increase hi in break_even_bps() for very slow strategies.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

4.7 Lab 16: Automation Stage: a Scheduled, Risk-Checked Forward Test and Live Paper Trading

Maps to: A7, A8, K7 | Duration: 25 min | Slide 189

Scenario

The desk is ready to automate the configured EMA 10/30 strategy on SPY. Before any capital is considered, the pipeline must run unattended as a forward test: one decision cycle at a fixed time, completed bars only, pre-trade risk checks, a kill switch, idempotent paper orders and an audit row for every decision - first as a 2025 replay, then live on the Alpaca paper account during class.

Objective

Build pipeline/risk.py and pipeline/runner.py: `python -m pipeline run --once | --loop` with the market clock, risk checks, kill switch and trade log; run a 2025 forward test; run the strategy live on the paper account for a few cycles; and schedule it with cron, launchd or Windows Task Scheduler.

Files

labs_fullcode/Lab16-paper-trading-bot/ - Lab16-paper-trading-bot/run_stage.py, pipeline/runner.py, pipeline/risk.py, Lab16-paper-trading-bot/solution/runner.py, Lab16-paper-trading-bot/solution/risk.py

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 16	python3 Lab16-paper-trading-bot/run_stage.py (same as: python -m pipeline run)
The complete code	pipeline/risk.py, pipeline/runner.py (reference copy: Lab16-paper-trading-bot/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab16-paper-trading-bot/Lab16_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 1: Build the risk layer and the kill switch (A7)

Ask your assistant for pipeline/risk.py.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/risk.py with class RiskManager(cfg). kill_switch_on() = the file named by risk.kill_switch_file exists under labs. check(broker, status, symbol, wants_buy, allow_queue=False) returns (ok_to_trade, ok_to_buy, notes): kill switch -> (False, False, ['KILL_SWITCH present - trading halted']); session 'closed' -> blocked with 'market closed' unless allow_queue (then note 'order will queue (GTC) for the next session'); equity / last_equity - 1 <= -risk.max_daily_loss_pct -> no new buys; symbol not in risk.allowed_symbols -> no buy; account trading_blocked -> no buy; otherwise ['all checks passed']. cap_weight(weight, equity, price) applies max_position_pct and max_order_value. kill_main(argv) for `python -m pipeline kill --on | --off | --status`: --on writes the file with a timestamp; --off MOVES it to _state/kill_switch_history/KILL_SWITCH.<timestamp> (an audit record - never delete it). Risk is deterministic code; no model may override it. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline kill --status prints 'KILL SWITCH OFF'.

Step 2: Build one decision cycle (A7)

Ask your assistant for decide() and run_cycle() in pipeline/runner.py.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/runner.py. decide(bars, cfg) = the configured strategy's target from pipeline.signals.target_position on COMPLETED bars only, returning bar_date, close, signal and (for ema_cross) ema_fast and ema_slow. run_cycle(broker, om, risk, bars, run_time, cfg, weight, allow_queue=False, poll_seconds=0): read position and pending shares from the pipeline.execution.OrderManager; wants_buy = signal 1 and nothing held or pending; wants_sell = signal 0 and shares held; run RiskManager.check; if allowed call om.rebalance({symbol: weight}) or om.rebalance({symbol: 0.0}) with tag = the bar date (one decision per bar, so a double-fired job cannot double-trade); otherwise action BLOCKED if a trade was wanted; return a log row with run_time, bar_date, symbol, close, ema_fast, ema_slow, signal, position_before, action, qty, order_id, status, fill_price, equity, checks. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: run_cycle returns a dict with the 15 log fields.

Step 3: Build the forward-test replay and the logs (A8)

Ask your assistant for replay(), once() and the log writers.

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/runner.py add replay(cfg, start, end): offline forward test on PaperBrokerSim (state_file bot_sim.json, reset) - for each 2025 session set the simulator clock to 09:35, mark the price at today's open, run_cycle on bars up to YESTERDAY's close with weight risk.max_position_pct, then process the day's bar and roll end_of_day; stop if the kill switch appears. Write _outputs/trade_log.csv (fresh, one row per session) and _outputs/Lab16_equity_log.csv (date, equity, position, close), print every non-HOLD row, the number of orders, start and end equity and SPY buy and hold over the same days, and save chart Lab16_forward_test.png. once(cfg): one cycle; offline it acts at the open after the last bundled bar and appends to trade_log.csv; online it appends to _outputs/live_paper_log.csv. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline run --help lists --once, --loop, --live-paper, --interval, --max-cycles, --start and --end.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 4: Add the live paper loop and scheduling (A7)

Ask your assistant for live_loop(), main() and schedule_main().

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/runner.py add live_loop(cfg, interval, max_cycles): on the Alpaca PAPER account (or the simulator with a clear message if no keys) run up to max_cycles cycles: print the clock line, stop with 'KILL SWITCH ON - no orders sent; loop stopped' if the kill switch exists, run_cycle with weight live_paper.max_position_pct, allow_queue=True and poll_seconds live_paper.poll_seconds online (the OrderManager sends extended-hours limit orders outside regular hours), append to _outputs/live_paper_log.csv, print the row and every position with unrealised P&L, then sleep interval seconds; finish with the dashboard address. main(argv): --loop --live-paper -> live_loop; --loop offline -> replay; --loop online -> live_loop; otherwise once. schedule_main(argv) for `python -m pipeline schedule`: print and write to _outputs/Lab16_schedule/ a crontab line with CRON_TZ=America/New_York for schedule.run_at_exchange_time Mon-Fri (with the Singapore time in US summer and winter), a launchd plist (com.lioncitycapital.pipeline, WorkingDirectory labs) and a Windows schtasks command, each running `python -m pipeline run --once` from labs and appending to _outputs/scheduler.log. Do not install anything. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline schedule --help works.

Step 5: Run the 2025 forward test (A8, K7)

Run: python -m pipeline run --loop (same as: python3 Lab16-paper-trading-bot/run_stage.py --loop). Offline this replays 2025 day by day; each order fills at that day's open in the simulator.

Check: 250 sessions, 5 orders: BUY 159 after the 2025-01-21 close, SELL after 2025-02-27, BUY 166 after 2025-05-02, SELL after 2025-11-19, BUY 163 after 2025-11-28; equity USD 100,000.00 -> 116,249.79 (+16.25%) versus SPY buy and hold +17.06%; trade_log.csv with 250 rows; Lab16_forward_test.png.

Step 6: Audit the decision log (A8)

Open _outputs/trade_log.csv. Check bar_date, signal, action, qty, status, fill_price, equity and checks for one BUY day and one HOLD day.

AI prompt - paste into your AI coding assistant
Read labs_fullcode/_outputs/trade_log.csv and write an audit summary for the compliance officer: number of sessions, decisions by action, every order with its bar date, fill price and equity after the fill, and any row where a risk check did not say 'all checks passed'. Do not change any file.

Check: 250 rows: 5 BUY/SELL rows with status filled and 245 HOLD rows; 249 rows say 'all checks passed' and one (2025-01-27) says 'daily loss -2.05% breaches 2% limit - no new buys' - a HOLD day with the position already held, so the rule blocked nothing.

Step 7: Test the kill switch (A7)

Run: python -m pipeline kill --on, then python -m pipeline run --once, then python -m pipeline kill --off.

Check: The --once row shows checks 'KILL_SWITCH present - trading halted' (no order can be sent); --off prints 'moved to _state/kill_switch_history/KILL_SWITCH.<timestamp>' - the halt is kept as an audit record.

Step 8: Run the strategy LIVE on the paper account (A7, A8, K7)

With paper keys in .env, during class run: python -m pipeline run --loop --interval 60 --live-paper --max-cycles 5. Watch decision -> order -> fill -> position -> log for each cycle. After two or three cycles, in a SECOND terminal run python -m pipeline kill --on and watch the next cycle halt. Then run python -m pipeline kill --off and open _outputs/live_paper_log.csv and the Alpaca dashboard.

Check: Cycle 1 sends one SPY buy for 10% of equity (an extended-hours LIMIT DAY order in Singapore class hours; about 14 shares on USD 100,000) and prints its lifecycle and the position; later cycles HOLD ('buy

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

order ... still working' or position held); after kill --on: 'KILL SWITCH ON - no orders sent; loop stopped'. Without keys the same loop runs on the simulator (at a weekend the buy queues as a GTC limit order).

Step 9: Schedule the runner (A7)

Run: python -m pipeline schedule and read the three entries. Only install one if your instructor approves (crontab -e, launchctl load, or schtasks in a Command Prompt).

Check: One cycle at 09:35 New York = 21:35 Singapore (US summer) / 22:35 (US winter), Mon-Fri; files crontab.txt, com.lioncitycapital.pipeline.plist and schtasks.cmd in _outputs/Lab16_schedule/; nothing is installed.

Expected Output (Full Code)

python -m pipeline run --loop (offline 2025 replay): 250 sessions, 5 orders (BUY 159, SELL, BUY 166, SELL, BUY 163), equity USD 100,000.00 -> 116,249.79 (+16.25%) versus SPY +17.06%; trade_log.csv with 250 rows; kill switch halts the next cycle and is archived on --off; run --loop --live-paper sends a 10% SPY order as an extended-hours limit during Singapore hours and stops on the kill switch; schedule entries for 09:35 New York (21:35 / 22:35 Singapore); chart _outputs/Lab16_forward_test.png.

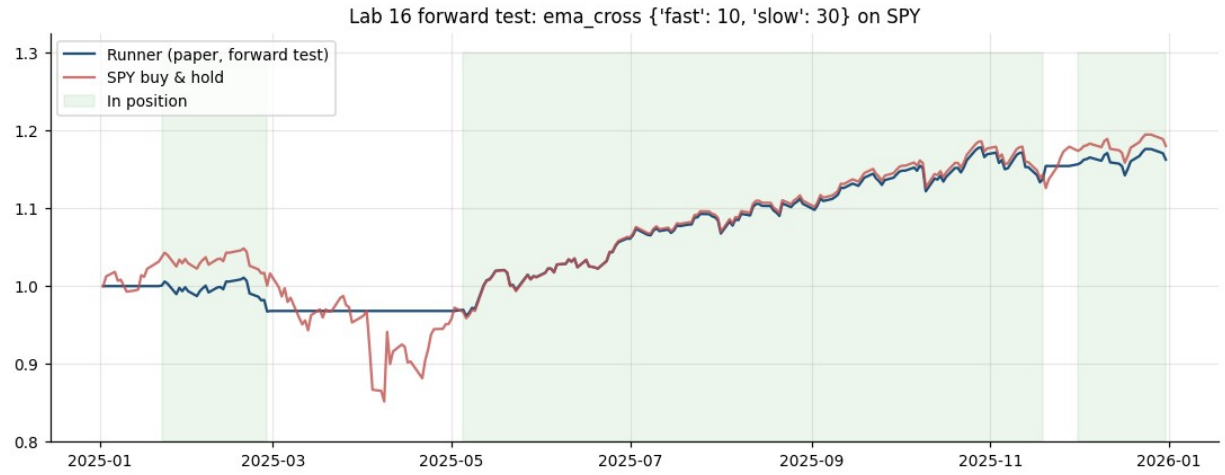


Figure: Lab 16 output (labs_fullcode/_outputs/Lab16_forward_test.png), offline mode on the bundled data.

Debrief

The runner is a software agent coded from a selected model (A7): signal on completed bars, deterministic risk limits, idempotent execution, a kill switch and an audit log. The forward test and the live paper loop check the actions it takes and their outcomes on unseen data (A8), including the real effect of whole shares, fills and session rules (K7).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Troubleshooting

Symptom	What to do
Action BLOCKED with 'market closed'	Expected for --once at a weekend online; the live loop queues a GTC order instead (allow_queue).
No trades in the replay	Check that labs_fullcode/KILL_SWITCH does not exist (python -m pipeline kill --status).
A cron job ran twice	The client_order_id is unique per decision bar, so the second run finds the order and sends nothing.
macOS cron fires at the wrong hour	macOS cron ignores CRON_TZ; use the Singapore-time line or launchd.

4.8 Lab 17: Event-Driven Automation: TradingView Webhook to Signal, Risk and Execution

Maps to: A7, K5, K7 | Duration: 20 min | Slide 191

Scenario

Some Lion City Capital traders design signals in TradingView. The desk wants their alerts to place paper orders automatically - but only through the pipeline: a gateway that checks a shared secret, the allow-list, a maximum size and duplicate alerts, asks the pipeline's own signal, runs the risk checks and then hands the order to the order manager, logging every request.

Objective

Build pipeline/webhook.py: a webhook receiver that turns TradingView alert JSON into a pipeline decision (validation -> signal -> risk -> execution), test it with a local alert simulator and a self-test, and configure a real TradingView strategy alert.

Files

labs_fullcode/Lab17-tradingview-webhook/ - Lab17-tradingview-webhook/webhook_stage.py, Lab17-tradingview-webhook/send_test_alert.py, Lab17-tradingview-webhook/lcc_webhook_strategy.pine, Lab17-tradingview-webhook/tradingview_alert_message.json, pipeline/webhook.py, Lab17-tradingview-webhook/solution/webhook.py

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 17	python3 Lab17-tradingview-webhook/webhook_stage.py (same as: python -m pipeline webhook)
Also in this lab: send_test_alert.py	python3 Lab17-tradingview-webhook/send_test_alert.py
The complete code	pipeline/webhook.py (reference copy: Lab17-tradingview-webhook/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab17-tradingview-webhook/Lab17_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

Step 1: Set the shared secret (A7)

In labs_fullcode/.env set WEBHOOK_SECRET to a long random string (for example the output of python3 -c "import secrets; print(secrets.token_urlsafe(24))"). Without it the receiver uses a demo secret and prints a warning.

Check: python -m pipeline check shows WEBHOOK_SECRET 'found (hidden)'.

Step 2: Build alert validation (A7, K5)

Ask your assistant for the Gateway class in pipeline/webhook.py.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/webhook.py with class Gateway(cfg, session=None): secret from pipeline.config.secret(cfg, 'webhook_secret') (demo value 'lcc-demo-secret' if unset), broker = common.broker.get_broker(state_file='webhook_sim.json', reset=True), an OrderManager, a RiskManager and a set of processed alert ids. handle(payload) -> (http_code, body): 401 if hmac.compare_digest of the secret fails; ticker 'NASDAQ:AAPL' -> 'AAPL'; 422 if the ticker is not in risk.allowed_symbols, the action is not buy/sell, qty is not a number or not 0 < qty <= risk.max_order_qty; alert id = payload 'id' or a hash of ticker, action, qty and time; a repeated id returns 200 'duplicate ignored' (TradingView retries are safe). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: Gateway(load_config()).handle({'secret': 'guess'}) returns (401, ...).

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 3: Route valid alerts through signal, risk and execution (A7, K7)

Ask your assistant for the second half of handle().

AI prompt - paste into your AI coding assistant
In Gateway.handle add: pipeline_signal(ticker) = the configured strategy's latest target from pipeline.signals for that ticker; if webhook.require_signal_agreement is true and the alert disagrees, return 409; a sell with no position returns 422 (long only); RiskManager.check (kill switch, session, daily loss) failing returns 423 'blocked by risk'; otherwise target shares = held + qty for a buy or held - qty for a sell and call OrderManager.rebalance({ticker: target}, tag='tv<alert id>', units='shares') so the order is session-aware and idempotent; return 200 with order id, status, side, qty, fill price, pipeline_signal and signal_agrees. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: A valid buy returns 200 'order submitted' with pipeline_signal and signal_agrees in the body.

Step 4: Build the server, the simulator and the self-test (A7)

Ask your assistant for the HTTP layer.

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/webhook.py add a standard-library http.server receiver (no Flask): POST webhook.path parses JSON (400 'body is not valid JSON' otherwise), calls Gateway.handle, appends a row to _outputs/Lab17_webhook_log.csv (time, client, alert_id, ticker, action, qty, http, result, pipeline_signal, order_id, status, detail; header written fresh when the server starts) and replies JSON; GET /health returns status, broker mode and kill switch. serve(cfg, host, port, max_requests) handles one request at a time; offline it uses a simulated regular session. test_alerts(secret) returns seven cases with expected codes: valid BUY 5 SPY (200), valid SELL 5 AMEX:SPY (200), wrong secret (401), TSLA (422), qty 5000 (422), malformed text (400), a retry of alert 1 (200 duplicate). send_test_alerts(url, secret) posts them with urllib and prints OK or MISMATCH; selftest(cfg) runs the server on a free port in a thread, sends the alerts and prints 'RESULT: n/7'. main(argv) for `python -m pipeline webhook [--port] [--max-requests] [--selftest]` and alert_sim_main(argv) for `python -m pipeline alert-sim --url --secret`. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline webhook --help shows --host, --port, --max-requests and --selftest.

Step 5: Run the self-test (A7, K5)

Run: python -m pipeline webhook --selftest.

Check: Seven alerts: 1 BUY 5 SPY -> 200 'order submitted', filled at 676.70, pipeline_signal 1, agrees; 2 SELL 5 SPY -> 200, filled at 676.57, signal_agrees false; 3 -> 401 'bad or missing secret'; 4 -> 422 'TSLA not on the allow-list'; 5 -> 422 'qty must be between 0 and 100'; 6 -> 400 'body is not valid JSON'; 7 -> 200 'duplicate ignored'; 'RESULT: 7/7 alerts handled as expected'; _outputs/Lab17_webhook_log.csv with 7 rows.

Step 6: Run the receiver and the simulator in two terminals (A7)

Terminal 1: python -m pipeline webhook --port 8080 (same as: python3 Lab17-tradingview-webhook/webhook_stage.py --port 8080). Open http://127.0.0.1:8080/health in a browser. Terminal 2: python3 Lab17-tradingview-webhook/send_test_alert.py --url http://127.0.0.1:8080/webhook. Stop the receiver with Ctrl+C.

Check: The health page shows status ok, broker offline-sim (or alpaca-paper) and kill_switch false; the simulator prints the same seven OK lines; the receiver prints each [code] and its stop summary.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 7: Let the pipeline's signal veto an alert (A7, K7)

In pipeline/config.yaml set webhook.require_signal_agreement: true and rerun python -m pipeline webhook --selftest. Set it back to false.

Check: Alert 2 (SELL while the pipeline's ema_cross signal for SPY is 1) now returns 409 'pipeline signal for SPY is 1: it disagrees with the alert' - RESULT 6/7, as designed.

Step 8: Prove the kill switch reaches the webhook (A7)

Run: python -m pipeline kill --on, then python -m pipeline webhook --selftest, then python -m pipeline kill --off.

Check: Alert 1 returns 423 'blocked by risk - KILL_SWITCH present - trading halted' and alert 2 returns 422 (no position to sell); RESULT 4/7 while halted.

Step 9: Optional - connect a real TradingView alert (paid plan) (A7, K5)

Add Lab17-tradingview-webhook/lcc_webhook_strategy.pine to AMEX:SPY (EMA 10/30, 5 shares per order). Expose the receiver with an HTTPS tunnel (ngrok http 8080) because TradingView posts only to ports 80/443. Create one alert on the strategy: 'Order fills only', tick Webhook URL, paste https://<your-tunnel>/webhook and use tradingview_alert_message.json (with your secret) as the message. Webhooks need a paid plan and two-factor authentication.

AI prompt - paste into your AI coding assistant
Review labs_fullcode/Lab17-tradingview-webhook/tradingview_alert_message.json and lcc_webhook_strategy.pine. Explain which TradingView placeholders fill ticker, action, qty and id, why the id makes retries safe, and three ways to keep the shared secret from leaking (who can see alert messages, chart sharing, logs). Do not change the files.

Check: Each real alert appears in the receiver console and in _outputs/Lab17_webhook_log.csv; online the paper order appears in the Alpaca dashboard.

Expected Output (Full Code)

python -m pipeline webhook --selftest: HTTP 200 (BUY 5 SPY filled at 676.70), 200 (SELL 5 SPY), 401 'bad or missing secret', 422 'TSLA not on the allow-list', 422 'qty must be between 0 and 100', 400 'body is not valid JSON', 200 'duplicate ignored'; RESULT: 7/7; with require_signal_agreement the SELL returns 409; with the kill switch on, the BUY returns 423; audit log _outputs/Lab17_webhook_log.csv.

Debrief

A webhook lets a charting platform trigger execution on a trading platform (K5), but the gateway code (A7) is where the controls live: authentication, allow-lists, size limits, duplicate protection, the pipeline's own signal, the risk layer and an audit log. Each automated order still pays spread and slippage (K7), so alert frequency matters.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Troubleshooting

Symptom	What to do
Address already in use	Another program uses port 8080; start with --port 8081 and use the same port in --url.
All alerts return 401	The receiver and sender use different secrets; both read WEBHOOK_SECRET from labs_fullcode/.env, or pass --secret to the simulator.
TradingView shows 'webhook delivery failed'	Use the HTTPS tunnel URL on port 443, keep the receiver running and check the alert message is valid JSON.
Alerts return 423	The kill switch is on or the market is closed online; check python -m pipeline kill --status.

4.9 Lab 18: Diagnostics Stage: Automated Bias Checks before Go-Live

Maps to: A9, K8 | Duration: 20 min | Slide 192

Scenario

A junior quant sends the investment committee a 'smoothed momentum' backtest on SPY with a Sharpe ratio above 4 and asks to go live. The desk responds by adding a diagnostics stage to the pipeline: every strategy - including its own ema_cross - must pass automated checks for look-ahead, cost drag, overfitting and survivorship before it may trade, even on paper.

Objective

Build `python -m pipeline diagnose` to run automated look-ahead (features and execution), cost-drag, overfitting and survivorship checks on any strategy and on the junior quant's exhibit, and write a PASS / WARN / FAIL report with a go-live verdict.

Files

labs_fullcode/Lab18-diagnose-bias/ - Lab18-diagnose-bias/diagnose_stage.py, Lab18-diagnose-bias/buggy_backtest.py, pipeline/diagnose.py, Lab18-diagnose-bias/solution/diagnose.py

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 18	python3 Lab18-diagnose-bias/diagnose_stage.py (same as: python -m pipeline diagnose)
Also in this lab: buggy_backtest.py	python3 Lab18-diagnose-bias/buggy_backtest.py
The complete code	pipeline/diagnose.py (reference copy: Lab18-diagnose-bias/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab18-diagnose-bias/Lab18_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

Step 1: Run the suspicious backtest (A9)

Run: python3 Lab18-diagnose-bias/buggy_backtest.py. Record the CAGR, Sharpe ratio and win rate.

Check: CAGR 71.52%, Sharpe 4.28, win rate 67.5% - far above anything a real SPY timing rule achieves.

Step 2: Find the problems (A9, K8)

Read the three lines marked CHECK 1 to 3 and write down what is wrong with each.

AI prompt - paste into your AI coding assistant
Review labs_fullcode/Lab18-diagnose-bias/buggy_backtest.py as a model-risk reviewer. For each line marked CHECK 1 to 3, explain the bias, how it inflates the result and the correct fix. Do not run or change the file.

Check: CHECK 1 centred window (uses t+1 and t+2), CHECK 2 same-bar returns, CHECK 3 no costs.

Step 3: Build the look-ahead checks (A9, K8)

Ask your assistant for the first part of pipeline/diagnose.py.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/diagnose.py. buggy_signal(bars) reproduces the exhibit (5-day rolling mean with center=True, long when it rises); trailing_signal(bars) is the fix (trailing window). In run(strategy=None, trials=None) add check 1, look-ahead in FEATURES, using pipeline.signals.lookahead_test (25 truncation truncation and future-shock checks) on the configured strategy and on the exhibit (fn=buggy_signal); and check 2, look-ahead in EXECUTION: the strategy's Sharpe with same-bar returns (target_t x return_t) versus the pipeline's next-bar backtest, plus the exhibit fixed one problem at a time: 0 as submitted (centred, same bar), 1 trailing window same bar, 2 + next-bar execution (common.backtest.backtest), 3 + alpaca_retail costs, and SPY buy and hold, as a CAGR / Sharpe / max drawdown table. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: The exhibit fails the truncation test; the configured strategy passes.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 4: Build the cost, overfitting and survivorship checks (A9, K8)

Ask your assistant for checks 3-5, the report and main().

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/diagnose.py add: check 3 cost drag - gross vs net CAGR from cost_summary(), break-even from pipeline.backtest.break_even_bps versus the assumed one-way cost (half spread + slippage of the config preset): PASS if break-even >= 3 x assumed and drag < 25% of gross; check 4 overfitting - random_strategies(spy_returns, 500, seed=1) (skill-free long/flat signals switching with 10% probability a day, traded next bar), the best in-sample Sharpe among the first N where N = the number of sets in _outputs/Lab10_grid_results.csv (default 76), compared with the strategy's 2018-2021 Sharpe, and its 2022-2025 decay; check 5 survivorship / hindsight - the universe was chosen from today's listed symbols, and an equal-weight NVDA/MSFT/AAPL 'AI basket' versus SPY. Print each check as [PASS|WARN|FAIL|INFO] check subject evidence, a VERDICT ('BLOCK - fix the FAIL items' if the strategy has a FAIL, else 'CLEAR for paper trading - review every WARN'), write _outputs/Lab18_diagnostics.csv and charts Lab18_bias_comparison.png and Lab18_overfitting.png. main(argv): --strategy, --trials, --strict (exit 1 on BLOCK). Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline diagnose --help shows --strategy, --trials and --strict.

Step 5: Run the diagnostics (A9)

Run: python -m pipeline diagnose (same as: python3 Lab18-diagnose-bias/diagnose_stage.py).

Check: ema_cross: look-ahead PASS (0 of 25), execution PASS (same-bar Sharpe 1.45 vs next-bar 0.82), cost drag PASS (gross 9.13% -> net 9.02%, break-even 135.8 bps vs 1.5 bps), overfitting WARN (IS Sharpe 0.86 vs best of 76 random 1.42; OOS 0.75), survivorship WARN. Exhibit: look-ahead FAIL (25 of 25), execution FAIL (4.28 -> 0.24). VERDICT for ema_cross: CLEAR for paper trading - review every WARN.

Step 6: Read the exhibit fixed one problem at a time (A9, K8)

Read the fix table and open _outputs/Lab18_bias_comparison.png.

Check: As submitted 71.47% / Sharpe 4.28; trailing window, same bar 66.56% / 4.68; + next-bar execution 2.23% / 0.24; + realistic costs 1.46% / 0.18; SPY buy and hold 14.14% / 0.78 - the same-bar error is the main bias.

Step 7: Read the overfitting and survivorship evidence (K8)

Read the INFO row and open _outputs/Lab18_overfitting.png. Then run python -m pipeline diagnose --strategy dip_buy.

Check: 500 random strategies: best IS Sharpe 1.77 -> OOS 0.55, median OOS 0.46, IS/OOS correlation -0.05, 16% have IS Sharpe > 1. AI basket CAGR 38.84% vs SPY 14.15%. dip_buy: cost drag PASS (break-even 19.5 bps), overfitting WARN (IS Sharpe 0.14, OOS 0.80).

Step 8: Use the stage as a gate and write the committee note (A9)

Run: python -m pipeline diagnose --strict and check the exit code (0 for ema_cross). The e2e capstone uses the same verdict to block orders.

AI prompt - paste into your AI coding assistant
Using labs_fullcode/_outputs/Lab18_diagnostics.csv, write a 4-sentence note for the investment committee: why the junior quant's Sharpe of 4.28 is not credible, which automated check caught each problem, what the configured ema_cross strategy's WARN items mean, and what must happen before any live capital.

Check: Exit code 0 for ema_cross; a note that names look-ahead (features and execution), cost drag, overfitting and survivorship.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Expected Output (Full Code)

python -m pipeline diagnose: ema_cross PASS for look-ahead (0 of 25), execution (same-bar 1.45 vs next-bar 0.82) and cost drag (break-even 135.8 bps vs 1.5 bps), WARN for overfitting (IS 0.86 vs best of 76 random 1.42) and survivorship -> CLEAR for paper trading; exhibit FAIL: 25 of 25 look-ahead checks, Sharpe 4.28 as submitted -> 0.24 fixed -> 0.18 with costs; 500 random strategies best IS 1.77 -> OOS 0.55; AI basket 38.84% vs SPY 14.15%; files _outputs/Lab18_diagnostics.csv and charts _outputs/Lab18_bias_comparison.png and _outputs/Lab18_overfitting.png.

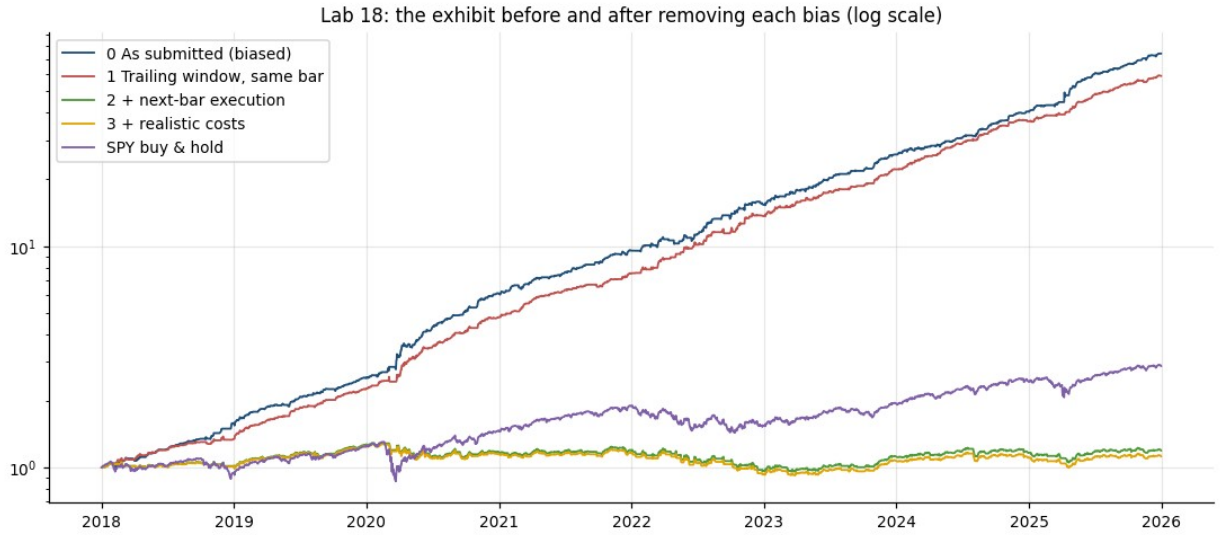


Figure: Lab 18 output (labs_fullcode/_outputs/Lab18_bias_comparison.png), offline mode on the bundled data.

Debrief

Most 'too good to be true' results come from a short list of causes (K8): look-ahead in features or execution, overfitting, survivorship and ignored costs. Automating the checks and using the verdict as a gate (A9) is what stops an unintended outcome from reaching even a paper account - and the same checks run on the desk's own strategy, not only on other people's.

Troubleshooting

Symptom	What to do
Step 1 shows a slightly different CAGR from the fix table	buggy_backtest.py drops the first NaN rows; the stage fills them with 0 - 71.52% vs 71.47%.
Random-strategy numbers differ	Change the seed or --trials and the threshold changes, but the pattern (no persistence) stays.
Is center=True really look-ahead?	Yes: a centred 5-day mean at day t uses the closes of t+1 and t+2.

4.10 Lab 19: Monitoring and Reporting Stage: Automated Tearsheet and Backtest-vs-Forward Report

Maps to: A8, A10 | Duration: 25 min | Slide 194

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Scenario

After a year of paper trading, the investment committee wants a one-page tearsheet and a short report on the automated EMA 10/30 strategy: how the research backtest compares with the forward test, what it cost, whether any monitoring limit is breached, and whether the desk should allocate capital - produced automatically by the pipeline, every time.

Objective

Build `python -m pipeline report` to compare the research backtest with the runner's forward-test logs, run monitoring checks with thresholds, and generate a tearsheet and a markdown report with data-driven insights and a recommendation.

Files

labs_fullcode/Lab19-performance-report/ - Lab19-performance-report/report_stage.py, pipeline/report.py, Lab19-performance-report/solution/report.py

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 19	python3 Lab19-performance-report/report_stage.py (same as: python -m pipeline report)
The complete code	pipeline/report.py (reference copy: Lab19-performance-report/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab19-performance-report/Lab19_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

Step 1: Make sure the forward-test logs exist (A8)

The stage reads _outputs/Lab16_equity_log.csv and _outputs/trade_log.csv from the Lab 16 runner; if either is missing it runs the 2025 replay itself. Run python -m pipeline run --loop if you want fresh logs.

Check: _outputs/Lab16_equity_log.csv has 250 rows ending at USD 116,249.79.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 2: Build the backtest-vs-forward comparison (A8)

Ask your assistant for the first part of pipeline/report.py.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/report.py. load_forward_test(cfg) reads the equity log and the BUY/SELL rows of trade_log.csv whose run_time falls in 2025-01-02 to 2025-12-31 (running pipeline.runner.replay quietly if a log is missing). In run(): backtest the configured strategy 2018-2025 with the config cost model and print a metrics table versus SPY buy and hold (total return, CAGR, volatility, Sharpe, Sortino, max drawdown, Calmar, win rate, profit factor, trades, exposure, alpha, beta) and the cost breakdown; backtest 2025 only (starting from 2024-12-31 so the first decision is warm) and compare it with the forward test: total return, Sharpe, max drawdown, orders and average fill slippage in bps versus the day's open; print the tracking difference and annualised tracking error. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: The comparison table has the columns 'Backtest 2025' and 'Forward test 2025 (paper)'.

Step 3: Add monitoring checks, the tearsheet and the report (A8, A10)

Ask your assistant for the monitoring section and the outputs.

AI prompt - paste into your AI coding assistant
In labs_fullcode/pipeline/report.py add THRESHOLDS = {'tracking_abs': 0.02, 'slippage_bps': 5.0, 'drawdown_stop': 0.15} and monitoring checks with status OK or ALERT: tracking vs backtest, average fill slippage, drawdown stop rule (current and worst forward-test drawdown), kill switch (pipeline.risk.RiskManager) and diagnostics (read _outputs/Lab18_diagnostics.csv rows for the configured strategy: ALERT on any FAIL or if diagnose was not run for it). Save _outputs/Lab19_monitoring.csv; a 3 x 2 tearsheet Lab19_tearsheet.png (equity vs SPY with the forward-test year shaded, drawdowns, monthly-returns heatmap fixed at +/-8%, rolling 6-month Sharpe, backtest vs forward 2025, costs by type); and _outputs/Lab19_report.md with the tables (DataFrame.to_markdown), five data-supported insights and a recommendation: 'CONTINUE PAPER TRADING ...' if every check is OK, otherwise 'PAUSE AND INVESTIGATE: monitoring alerts on ...'. Paper results only - not investment advice. main(argv) for `python -m pipeline report`. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline report --help works.

Step 4: Generate the report (A8, A10)

Run: python -m pipeline diagnose (so diagnostics exist for ema_cross), then python -m pipeline report (same as: python3 Lab19-performance-report/report_stage.py).

Check: 2018-2025: strategy CAGR 9.02% vs SPY 14.15%, Sharpe 0.81 vs 0.78, max drawdown -14.19% vs -33.72%, beta 0.35, alpha +4.03%; costs USD 1,072 (drag 0.11% a year). 2025: backtest +17.04% (Sharpe 1.64) vs forward test +16.25% (Sharpe 1.65), 5 orders each, average slippage 1.0 bps, tracking difference -0.79%.

Step 5: Read the monitoring checks (A8)

Read section [3]. Which limit is closest to being breached?

Check: Five checks OK: tracking -0.79% (limit +/-2%), slippage 1.0 bps (limit 5), drawdown current -1.34% / worst -4.85% (stop at -15%), kill switch off, diagnostics '1 WARN, no FAIL'.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 6: Trigger an alert on purpose (A8)

In pipeline/report.py change THRESHOLDS['tracking_abs'] to 0.005 and rerun python -m pipeline report. Then restore 0.02.

Check: The tracking check shows ALERT and the recommendation becomes 'PAUSE AND INVESTIGATE: monitoring alerts on tracking vs backtest'.

Step 7: Study the tearsheet (A10)

Open _outputs/Lab19_tearsheet.png: equity, drawdown, monthly-returns heatmap, rolling Sharpe, backtest vs forward, costs. Find the weakest months.

Check: The forward-test and backtest lines for 2025 almost overlap; the heatmap shows the weak months in red.

Step 8: Write the committee recommendation (A10)

Open _outputs/Lab19_report.md, read the generated insights and recommendation, and edit them in your own words, adding one risk the report does not mention.

AI prompt - paste into your AI coding assistant
Read labs_fullcode/_outputs/Lab19_report.md. Rewrite sections 5 and 6 for Lion City Capital's investment committee in plain English (under 200 words): what the data show, why the forward test matters, the honest message that the strategy reduces risk but has not beaten buy and hold, one additional risk (for example regime change or capacity), and a clear, conditional recommendation. Keep every number unchanged.

Check: A recommendation that continues paper trading with conditions (second signal, drawdown stop, risk-committee sign-off).

Expected Output (Full Code)

python -m pipeline report: 2018-2025 strategy CAGR 9.02% vs SPY 14.15%, Sharpe 0.81 vs 0.78, max drawdown -14.19% vs -33.72%, beta 0.35, costs USD 1,072 (drag 0.11% a year); 2025 backtest +17.04% (Sharpe 1.64) vs forward test +16.25% (Sharpe 1.65), 5 orders each, average slippage 1.0 bps, tracking -0.79%; 5/5 monitoring checks OK; recommendation: continue paper trading; files _outputs/Lab19_tearsheet.png, _outputs/Lab19_report.md and _outputs/Lab19_monitoring.csv.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

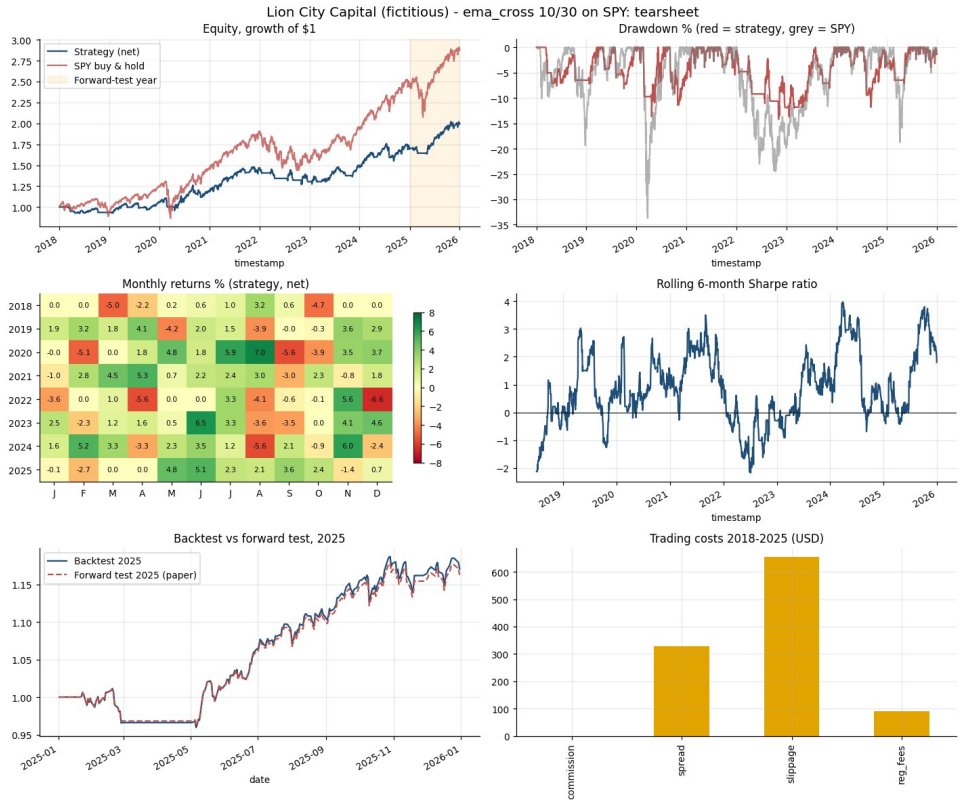


Figure: Lab 19 output (labs_fullcode/_outputs/Lab19_tearsheet.png), offline mode on the bundled data.

Debrief

Comparing the backtest with the forward test is the final test of whether actions and outcomes match the model (A8). Automated monitoring with explicit thresholds turns that comparison into an alarm, and a good report turns numbers into trends, insights and a clear, conditional recommendation (A10) - including the honest message that the strategy reduces risk but has not beaten buy and hold.

Troubleshooting

Symptom	What to do
ImportError: tabulate	Run pip install tabulate (needed for the markdown tables).
Diagnostics check shows ALERT 'not run'	Run python -m pipeline diagnose for the configured strategy first.
Forward-test numbers differ	Rerun python -m pipeline run --loop with the default dates; --once rows outside 2025 are ignored.

4.11 Lab 20: Capstone: One Command Runs the Pipeline End to End, with an Agentic Layer and Human Approval

Maps to: A7, A10, K1 | Duration: 20 min | Slide 196

Scenario

Lion City Capital's head of trading wants the whole pipeline - data, research, signals, execution, automation, diagnostics and reporting - to run with one command, stop sending orders whenever a gate

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

fails, and finish with an optional multi-agent layer (inspired by the TradingAgents framework, arXiv 2412.20138) whose proposals pass hard risk limits and a human's approval before any paper order.

Objective

Build pipeline/agents.py (analyst, researcher, trader and risk agents; rule-based by default, Claude optional) and pipeline/e2e.py (`python -m pipeline e2e`) that runs every stage in order with gates, then evaluate the pipeline and the role of human oversight.

Files

labs_fullcode/Lab20-agentic-capstone/ - Lab20-agentic-capstone/e2e_stage.py, pipeline/e2e.py, pipeline/agents.py, Lab20-agentic-capstone/solution/e2e.py, Lab20-agentic-capstone/solution/agents.py

How to Run the Python Scripts

Everything runs from the labs_fullcode folder, ONLINE with your Alpaca paper keys or OFFLINE on the bundled data. Do the first four rows once; after that each lab is one command.

Step	Command / file
Open a terminal in the labs_fullcode folder	cd labs_fullcode
Create and activate a virtual environment (once)	python3 -m venv .venv source .venv/bin/activate (Windows: .venv\Scripts\activate)
Install the libraries (once)	pip install -r requirements.txt
Optional: your Alpaca paper keys (leave blank to run OFFLINE)	cp .env.example .env (Windows: copy .env.example .env) then paste APCA_API_KEY_ID and APCA_API_SECRET_KEY into .env
Run Lab 20	python3 Lab20-agentic-capstone/e2e_stage.py (same as: python -m pipeline e2e)
The complete code	pipeline/agents.py, pipeline/e2e.py (reference copy: Lab20-agentic-capstone/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

Lab Steps - In Python with Your AI Assistant

AI writes the code; you own the algorithm. Paste each prompt into your AI coding assistant opened in the labs_fullcode folder (the same prompts, ready to copy, are in Lab20-agentic-capstone/Lab20_Prompts.pdf and Prompt_Library.html), run the stage and check the result. The complete, working code is already provided - see 'How to Run the Python Scripts' above. The Full Code Practical Performance uses this code path.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 1: Build the agent layer (K1, A7)

Ask your assistant for pipeline/agents.py. The risk manager must stay deterministic code.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/agents.py. Agents: technical_analyst(symbol, bars) - EMA 20/50, RSI 14, MACD histogram and ATR 14 from pipeline.indicators -> score in [-1, 1]; news_analyst(cfg, symbol, headlines) - lexicon sentiment of pipeline.ingest.load_headlines (POSITIVE / NEGATIVE word sets); debate(cfg, tech, news) - bull and bear points and balance = 0.6 x technical + 0.4 x news; trader(symbol, tech, research, equity, held) - buy if balance > 0.25 and flat, sell if < -0.25 and held, confidence = min(1, |balance| + 0.3), qty = floor(equity x 0.25 x confidence / close); risk_manager(cfg, proposal, tech, equity) - DETERMINISTIC hard limits from config agents (allow-list, min_confidence, cap = min(equity x max_position_pct, max_order_value) with resizing, ATR bracket stop = close - stop_atr x ATR and target = close + target_atr x ATR). Optional LLM: only if the ANTHROPIC_API_KEY named in secrets_from_env is set and the anthropic package imports, news_analyst and debate call client.messages.create(model=cfg['agents']['model'] (claude-sonnet-5-5), max_tokens=1024, system prompt that treats headlines as untrusted data and asks for ONE JSON object, extra_headers={'anthropic-beta': 'server-side-fallback-2026-07-01'}, extra_body={'fallbacks': 'default'}), return None on refusal or any error so the rule-based brain is used. run(symbols, execute_orders, yes, reset, session) also applies pipeline.risk.RiskManager.check, asks a human to type YES (EOF = declined) before OrderManager.rebalance(..., units='shares', order_type='bracket' in the regular session), writes _outputs/Lab20_decision_trace.json and chart Lab20_agent_pipeline.png. main(argv) for `python -m pipeline agents --symbols --execute --yes --reset --session`. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline agents --reset prints three symbol blocks and 'Propose-only run'.

Step 2: Build the end-to-end orchestrator (A7, A10)

Ask your assistant for pipeline/e2e.py.

AI prompt - paste into your AI coding assistant
Create labs_fullcode/pipeline/e2e.py. run(execute_orders=False, yes=False, fast=False) calls each stage's run() in order - check (reset), validate, ingest, hypothesis, indicators, reconcile, signals, execute (equal weights over the long universe targets from signals, capped at 20%), evaluate, search, ml, regime, abtest, pairs, backtest, runner.replay for 2025, webhook.selftest, diagnose, report - with contextlib.redirect_stdout into _outputs/Lab20_e2e_log.txt, timing each stage and printing one line: lab, stage, status (OK, GATE or ERROR), seconds and a short result. Exceptions are caught (ERROR) and the run continues. Gates: check not ready, ingest FAIL, diagnose BLOCK, report monitoring ALERT or kill switch on -> agents run propose-only. Last stage: agents.run; with --execute and gates passed ask the human for YES on the real console (or --yes for a scripted demo). Write _outputs/Lab20_e2e_summary.json and chart Lab20_e2e_pipeline.png (one coloured box per stage). --fast skips search, ml and pairs. Exit code 1 if any stage errored. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline e2e --help shows --execute, --yes and --fast.

Step 3: Run the whole pipeline (A7)

Run: python -m pipeline e2e (same as: python3 Lab20-agentic-capstone/e2e_stage.py).

Check: 20 stages, all OK, in about 6-10 seconds offline: e.g. hypothesis 'p 0.044 -> SUPPORTED', reconcile 'USD 79,878', signals '7/9 universe targets long', execute '7 paper orders', ml 'NO EDGE', run '+16.25% vs buy & hold +17.06%', webhook '7/7', diagnose 'CLEAR for paper trading', report '5/5 monitoring OK', agents 'NVDA APPROVED, AAPL no trade, SPY APPROVED; approval not requested'; 'RESULT: 20 stages ..., 0 errors, gates passed'.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 4: Trace the run (A10)

Open _outputs/Lab20_e2e_log.txt (the full output of every stage), _outputs/Lab20_e2e_summary.json and _outputs/Lab20_e2e_pipeline.png.

Check: The log contains each stage's full printout in order; the chart shows 20 green boxes.

Step 5: Read the agents' decisions (K1, A10)

Run: python -m pipeline agents --reset and open _outputs/Lab20_decision_trace.json and _outputs/Lab20_agent_pipeline.png. Follow one symbol from evidence to decision.

Check: Rule-based on 2025-12-31 data: NVDA technical +0.75, news +0.25, BUY 114 resized to 107 shares (cap USD 20,000) with stop 175.5 / target 201.9 - APPROVED; AAPL HOLD (balance +0.02); SPY technical +0.72, news -0.33, BUY 22 - APPROVED with stop 664.75 / target 694.45.

Step 6: Apply human oversight (A7, K1)

Run: python -m pipeline e2e --execute. When asked, type YES only if you agree with the approved orders; any other answer declines them.

Check: With YES: 2 paper bracket orders (NVDA 107, SPY 22) and 'approval granted, 2 paper orders'; otherwise 'approval declined, 0 paper orders'.

Step 7: Prove the gates stop the orders (A7)

Run: python -m pipeline kill --on, then python -m pipeline e2e --fast --execute --yes, then python -m pipeline kill --off.

Check: 'gates failed ... agents run in propose-only mode', 0 paper orders, 'gates FAILED (no orders)' - even with --yes the kill switch wins.

Step 8: Optional - switch on the LLM agents (K1)

Set ANTHROPIC_API_KEY in .env (with the anthropic package installed) and rerun python -m pipeline agents --reset. The news and debate agents then use Claude (claude-sonnet-5-5); if a call fails or is refused the pipeline falls back to the rules. The risk manager and the pipeline risk checks never change.

Check: The banner shows 'Agent brain: LLM (claude-sonnet-5-5)' and each News line ends with (llm); orders still need the same limits and a human YES.

Step 9: Run the capstone on the live paper account (A7)

With Alpaca paper keys set, run python -m pipeline e2e --execute during class. Approved agent orders are sent as extended-hours limit orders (no bracket outside the regular session) - check them in the Alpaca paper dashboard, then flatten with python -m pipeline execute --live-paper --flatten.

Check: The check stage shows ONLINE and the real session; agent orders appear in the dashboard with client ids starting lcc-agent.

Step 10: Evaluate the pipeline (A10, K1)

Write three sentences for the committee: what the pipeline and the agents add, what could go wrong (hallucinated news, prompt injection in headlines, over-trading, a broken gate) and why hard limits, the kill switch and human approval stay in place.

AI prompt - paste into your AI coding assistant
Using labs_fullcode/_outputs/Lab20_e2e_summary.json and labs_fullcode/_outputs/Lab20_decision_trace.json, draft three sentences for Lion City Capital's investment committee evaluating the end-to-end pipeline: the evidence from the stages, the risks of the agent layer, and the controls that keep it safe. Keep it factual and quote two numbers.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Check: Three sentences that cite stage results (for example forward test +16.25% vs backtest +17.04%) and name the controls.

Expected Output (Full Code)

python -m pipeline e2e (offline): 20 stages OK in about 6-10 seconds, 0 errors, gates passed - from check and validation through the 2025 forward test (+16.25%), webhook self-test (7/7), diagnostics (CLEAR) and report (5/5 monitoring OK) to the agents (NVDA BUY 107 with stop 175.5 / target 201.9, SPY BUY 22 with 664.75 / 694.45, AAPL HOLD); with --execute and YES, 2 paper bracket orders; with the kill switch on, 0 orders; files _outputs/Lab20_e2e_summary.json, _outputs/Lab20_e2e_log.txt, _outputs/Lab20_decision_trace.json and charts _outputs/Lab20_e2e_pipeline.png and _outputs/Lab20_agent_pipeline.png.

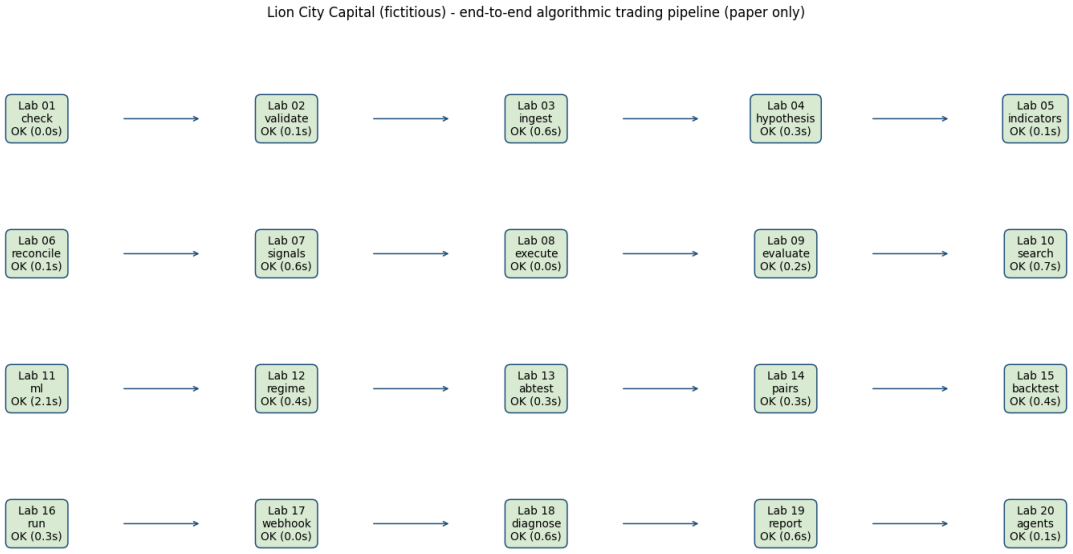


Figure: Lab 20 output (labs_fullcode/_outputs/Lab20_e2e_pipeline.png), offline mode on the bundled data.

Debrief

Agentic AI (K1) chains specialised agents that reason, debate and act, which makes it powerful and risky. Wiring it into an end-to-end pipeline whose gates, deterministic risk limits, kill switch and human approval it cannot override (A7), and evaluating every stage's evidence critically (A10), is how a desk adopts AI and automation responsibly.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Troubleshooting

Symptom	What to do
A stage shows ERROR	Open _outputs/Lab20_e2e_log.txt - the traceback is under that stage; fix it and rerun (python -m pipeline <stage> runs it alone).
Brain shows rule-based although a key is set	Install the anthropic package and check ANTHROPIC_API_KEY in labs_fullcode/.env; failures fall back to rules by design.
All agent decisions are HOLD on a second run	You already hold the positions; agents run with --reset in e2e, or add --reset when running agents alone.
Headlines look unfamiliar	Offline headlines are SYNTHETIC samples for training, not real news; online the Alpaca News API is used.

4.12 Topic Summary

- A7 - Perform coding and configuration of software agents or programs based on selected models or algorithms
- A8 - Conduct tests on the actions taken and outcomes to assess effectiveness of the models
- A9 - Diagnose unintended outcomes produced by statistical models
- A10 - Draw relevant trends and insights from data analysis to support decisions and report accordingly
- K7 - Coding languages for programming of algorithms and signals
- K8 - Potential reasons for unintended outcomes

Assessed by: Practical Performance Task 4 | Written Assessment Q7, Q8

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Assessment Preparation

The assessment mirrors what you have practised. The Written Assessment asks you to explain what you did and why; the Practical Performance asks you to do it again on a new trading scenario. Each lab folder also contains a LabNN_Instructions.pdf with the same steps, AI prompts and checks as this guide.

Question	TSC Ref	Knowledge Statement	Topic	Marks
Q1	K1	Data analytics and modelling business use cases	Topic 1	11
Q2	K2	Types of statistical analyses, data models, algorithms and advanced computational methods	Topic 3	11
Q3	K3	Range and application of various statistical methods and algorithms	Topic 2	11
Q4	K4	Range and application of various types of data models	Topic 3	11
Q5	K5	Usage of analytics platforms and tools	Topic 2	11
Q6	K6	Statistical modelling techniques	Topic 3	11
Q7	K7	Coding languages for programming of algorithms and signals	Topic 2	11
Q8	K8	Potential reasons for unintended outcomes	Topic 4	12
Q9	K9	Forms of data within financial services	Topic 1	11

Task	TSC Ref	Task Title	Related Labs
Task 1	A1, A6	Hypothesis testing: test the trading hypothesis and identify big data applications	Labs 3 and 4
Task 2	A2, A3	Technical indicators and backtesting in the Trading Simulator and TradingView	Labs 5, 6, 7 and 9
Task 3	A4, A5	Machine-learning model and A/B testing of two strategy variants	Labs 11 and 13
Task 4	A7, A8, A9, A10	Multi-agent trading with human approval, forward test, diagnosis and report to the investment committee	Labs 16, 18, 19 and 20

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Formula Sheet

Measure	Formula / definition
Simple return	r_t = P_t / P_(t-1) - 1
CAGR	(ending value / starting value)^(1 / years) - 1
Annualised volatility	standard deviation of daily returns x sqrt(252)
Sharpe ratio	(mean daily return - daily risk-free rate) / std of daily returns x sqrt(252)
Sortino ratio	as Sharpe, but dividing by the standard deviation of negative returns only
Maximum drawdown	min over t of (equity_t / running peak_t - 1)
Calmar ratio	CAGR / |maximum drawdown|
Win rate	profitable trades / total trades
Profit factor	gross profit / |gross loss|
Expectancy	win rate x average win - (1 - win rate) x |average loss|
Exposure	share of bars with a non-zero position
Turnover	total traded value / average capital
Beta / alpha	regression of strategy returns on benchmark returns: slope = beta, intercept = alpha
EMA	EMA_t = a x P_t + (1 - a) x EMA_(t-1), a = 2 / (n + 1)
RSI	100 - 100 / (1 + average gain / average loss), Wilder smoothing over 14 bars
Bollinger Bands	SMA_20 +/- 2 x rolling standard deviation_20
Cost per trade	commission + (half-spread + slippage) x shares x price + regulatory fee on sells
Welch's t statistic	(mean_B - mean_A) / sqrt(s_A^2 / n_A + s_B^2 / n_B)

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Troubleshooting Cheat-Sheet

Sign	Most likely cause	First diagnostic step
401 / 403 from Alpaca	Wrong or live keys used against the paper endpoint	Regenerate paper keys; check .env names APCA_API_KEY_ID / APCA_API_SECRET_KEY
Script says OFFLINE mode	No keys found in the environment or .env	Expected if you have no account - the bundled data is used
Empty DataFrame of bars	Weekend / holiday range, or symbol not on the IEX feed	Widen the date range; try SPY; check the feed setting
Order rejected	Market closed, insufficient buying power, PDT rule, bad quantity	Read the order's status and reject reason; check the market clock
Backtest looks too good	Look-ahead (same-bar execution) or no costs	Check the signal is shifted one bar; re-run with costs
Great in-sample, poor out-of-sample	Overfitting from parameter search	Compare the heatmap's neighbours; test on the hold-out period
Price chart shows a sudden 50% drop	Unadjusted stock split	Use adjusted data (adjustment='all' on Alpaca)
Times are off by hours	UTC versus New York versus Singapore time	Convert timestamps explicitly with tz_convert
Pine script will not compile	Missing //@version=6 or v5 syntax	Read the Pine Editor console line number; check function names
Webhook not received	No public URL, wrong secret, or free TradingView plan	Test with send_test_alert.py locally first

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Glossary

Term	Meaning
Algorithmic trading	Using computer programs that follow predefined rules on price, timing, volume or models to generate and execute orders automatically, with limited or no human intervention.
AI agent	A special class of agentic AI that stands by 24/7 and works all the time: always on, woken by schedules, events and messages, with its own long-term memory.
Agentic AI	The capability of AI to plan and act: it calls tools, uses skills and memory, and follows instructions to complete multi-step work when it is asked.
Alpaca	API-first US broker with commission-free US equities and ETFs and a free paper-trading environment; used for market data, orders and forward testing.
TradingView	Browser-based charting platform with Pine Script, a Strategy Tester and alerts.
Pine Script	TradingView's programming language for indicators and strategies; version 6 is current.
Paper trading	Simulated trading with virtual money against real market prices.
Backtest	Replay of a strategy on historical data to estimate how it would have performed.
Forward test	Running a strategy in real time on unseen data, usually on paper, before committing capital.
Look-ahead bias	Using information in a test that was not available at the time of the decision.
Overfitting	Fitting a model or parameters to noise, so that it fails on new data.
Survivorship bias	Testing only on assets that still exist, ignoring those that failed or were delisted.
Slippage	The difference between the expected price of a trade and the price at which it is filled.
Bracket order	An entry order with an attached take-profit and stop-loss order.
Walk-forward analysis	Repeatedly training on a past window and testing on the next unseen window.
A/B test	A statistical comparison of two variants to decide whether one is genuinely better.
Regime	A market state (for example calm-trending or high-volatility) with distinct statistical behaviour.
Kill switch	A control that immediately stops an algorithm from sending orders.
Tearsheet	A one-page summary of a strategy's performance, risk and costs.
MAS FEAT	The Monetary Authority of Singapore's principles of Fairness, Ethics, Accountability and Transparency for AI and data analytics in finance.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

References

- OpenClaw - https://en.wikipedia.org/wiki/OpenClaw
- Turing Post - What is Hermes Agent (Nous Research) - https://www.turingpost.com/p/hermes
- TechCrunch - OpenAI launches Dots (29 Sep 2026) - https://techcrunch.com/2026/09/29/openai-launches-dots-its-bubbly-agentic-avatar/
- D. E. Shaw & Co. - https://en.wikipedia.org/wiki/D._E._Shaw_%26_Co.
- Two Sigma - https://en.wikipedia.org/wiki/Two_Sigma
- Man AHL - who we are - https://www.man.com/ahl
- IFR - Jane Street thrives amid record trading - https://www.ifre.com/bonds/2246615/jane-street-thrives-amid-record-trading-bonanza
- IFR - Flow market maker: Citadel Securities - https://www.ifre.com/ifr-awards/2327839/flow-marketmaker-citadel-securities
- TradingView - Paper Trading main functionality - https://www.tradingview.com/support/solutions/43000516466-paper-trading-main-functionality/
- Alpaca - Trade with Alpaca on TradingView - https://alpaca.markets/learn/trade-with-alpaca-on-tradingview
- Alpaca - 24/5 trading - https://docs.alpaca.markets/us/docs/245-trading
- Alpaca - crypto spot trading (24/7) - https://docs.alpaca.markets/us/docs/crypto-trading
- Vaswani et al. (2017) Attention Is All You Need - https://arxiv.org/abs/1706.03762
- Xiao et al. (2024) TradingAgents - https://arxiv.org/abs/2412.20138
- TradingAgents GitHub - https://github.com/TauricResearch/TradingAgents
- Open Finance Lab: AgenticTrading - https://github.com/Open-Finance-Lab/AgenticTrading
- Treleaven, Galas & Lalchand (2013) Algorithmic Trading Review - https://cacm.acm.org/research/algorithmic-trading-review/
- MathWorks: algorithmic trading - https://www.mathworks.com/discovery/algorithmic-trading.html
- ScienceDirect Topics: algorithmic trading - https://www.sciencedirect.com/topics/economics-econometrics-and-finance/algorithmic-trading
- Pepperstone: algorithmic trading - https://pepperstone.com/en-gb/learn-to-trade/glossary/algorithmic-trading
- IG: algorithmic trading - https://www.ig.com/en/trading-platforms/algorithmic-trading
- Equiti: guide to algorithmic trading - https://www.equiti.com/sc-en/news/trading-ideas/guide-to-algorithmic-trading/
- DBS: what is algorithmic trading (2026) - https://www.dbs.bank.in/in/treasures/articles/learning-centre/algorithmic-trading
- SEC & CFTC: Findings on the 6 May 2010 market events - https://www.sec.gov/news/studies/2010/marketevents-report.pdf
- SEC: Knight Capital market access charges (2013) - https://www.sec.gov/news/press-release/2013-222
- Nasdaq: Regulation NMS FAQ - https://nasdaqtrader.com/content/marketregulation/regnms/regnms_faqs.pdf
- SEC: decimalisation completed 9 Apr 2001 - https://www.sec.gov/news/speech/spch509.htm
- Overdahl & McMillan (1998) NYSE Rule 80A - https://ideas.repec.org/a/ucp/jnlbus/v71y1998i1p27-53.html
- Commission Delegated Regulation (EU) 2017/589 (MiFID II RTS 6) - https://eur-lex.europa.eu/eli/reg_del/2017/589/oj
- Regulation Asia: SGX RegCo algorithmic trading guide (2020) - https://www.regulationasia.com/sgx-regco-publishes-regulatory-guide-on-algorithmic-trading/
- Alpaca: commission-free API brokerage (2018) - https://alpaca.markets/blog/commission-free-api-stock-algorithmic-trading/
- Wu et al. (2023) BloombergGPT - https://arxiv.org/abs/2303.17564
- TradingView: Pine Script v6 has landed (Dec 2024) - https://www.tradingview.com/blog/en/pine-script-v6-has-landed-48830/
- MAS: FEAT Principles (Nov 2018) - https://www.mas.gov.sg/news/media-releases/2018/mas-introduces-new-feat-principles-to-promote-responsible-use-of-ai-and-data-analytics
- MAS: Guidelines on AI Risk Management (Oct 2026) - https://www.mas.gov.sg/regulation/guidelines/guidelines-on-artificial-intelligence-risk-management-for-financial-institutions

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

- Markowitz (1952) Portfolio Selection, Journal of Finance - https://www.math.hkust.edu.hk/~maykwok/courses/ma362/07F/markowitz_JF.pdf
- Cboe - The creation of listed options (1973) - https://www.cboe.com/insights/posts/the-creation-of-listed-options-at-cboe
- Black-Scholes model history - https://www.macroption.com/black-scholes-history/
- Princeton Newport Partners (Thorp) - https://en.wikipedia.org/wiki/Princeton_Newport_Partners
- UCI Libraries - Edward O. Thorp career exhibit - https://exhibits.lib.uci.edu/thorp/career
- MSCI - Factor indexing through the decades (Barra) - https://www.msci.com/downloads/web/msci-com/research-and-insights/paper/factor-indexing-through-the-decades/factor-indexing-through-the-decades.pdf
- Renaissance Technologies - https://en.wikipedia.org/wiki/Renaissance_Technologies
- Pairs trade (Bamberger, Tartaglia, Morgan Stanley) - https://en.wikipedia.org/wiki/Pairs_trade
- Gatev, Goetzmann & Rouwenhorst - Pairs Trading (NBER w7032) - https://nber.org/papers/w7032
- Long-Term Capital Management - https://en.wikipedia.org/wiki/Long-Term_Capital_Management
- Khandani & Lo - What Happened to the Quants in August 2007? - https://www.nber.org/system/files/working_papers/w14465/w14465.pdf
- Island ECN - https://en.wikipedia.org/wiki/Island_ECN
- UNSW BusinessThink - speed bumps and HFT (Spread Networks, microwave) - https://www.businessthink.unsw.edu.au/articles/how-speed-bumps-are-curbing-high-frequency-traders
- IEX (350-microsecond speed bump; SEC approval 2016) - https://en.wikipedia.org/wiki/IEX
- SEC (2010) Concept Release on Equity Market Structure, Release 34-61358 - https://www.sec.gov/files/rules/concept/2010/34-61358.pdf
- Lopez-Lira & Tang - Can ChatGPT Forecast Stock Price Movements? - https://arxiv.org/abs/2304.07619
- Glasserman & Lin - Look-ahead bias in GPT sentiment - https://arxiv.org/abs/2309.17322
- Alpha-GPT - human-AI interactive alpha mining - https://arxiv.org/abs/2308.00016
- FinMem - LLM trading agent with layered memory - https://arxiv.org/abs/2311.13743
- FinAgent - multimodal foundation agent for trading - https://arxiv.org/abs/2402.18485
- StockAgent - LLM stock trading in simulated markets - https://arxiv.org/abs/2407.18957
- QuantAgent - multi-agent LLMs for high-frequency trading - https://arxiv.org/abs/2509.09995
- AI Hedge Fund (open source) - https://github.com/virattt/ai-hedge-fund
- Chain-of-Alpha (2025) - LLM alpha mining, reviews Alpha-GPT and AlphaAgent - https://arxiv.org/abs/2508.06312

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.